Lecture Notes on Angewandte Mathematik für das Web

Andreas Beham

Wintersemester 2026/27

Contents

1 Foundations of Graph Theory **6**
- 1.1 Undirected Graphs . . . 6
- 1.2 Directed Graphs . . . 9
- 1.3 Isomorphism of Graphs . . . 10
- 1.4 Graph Families . . . 11
- 1.5 Subgraphs . . . 12
- 1.6 Weighted Graphs . . . 13

2 Walks and Connectedness **14**
- 2.1 Connected Components, Bridges, Distance, and Diameter . . . 15
  - 2.1.1 Tarjan's Algorithm for Finding Strongly Connected Components . . . 17
  - 2.1.2 Topological Sorting . . . 19

3 Hamiltonian and Eulerian Graphs **21**
- 3.1 Hamiltonian Graphs . . . 21
  - 3.1.1 Traveling Salesperson Problem . . . 22
- 3.2 Euler Graphs . . . 23
  - 3.2.1 Fleury's Algorithm . . . 25
  - 3.2.2 Hierholzer's Algorithm . . . 25
  - 3.2.3 Chinese Postman Problem . . . 26

4 Transitive Closure **28**
- 4.1 Transitive Closure via Matrix Multiplication . . . 28
- 4.2 Warshall Algorithm . . . 30

5 Shortest Paths **31**
- 5.1 Shortest Path via Breadth-First Search . . . 32
- 5.2 Floyd Algorithm . . . 34
- 5.3 Bellman-Moore-Ford Algorithm . . . 35
- 5.4 Dijkstra's Algorithm . . . 38
  - 5.4.1 Proof of Correctness . . . 40
- 5.5 A* Algorithm . . . 41
- 5.6 Bidirectional Search . . . 43
- 5.7 Iterative Deepening A* . . . 46

6 Minimum Spanning Trees **48**
- 6.1 Trees and Arborescences . . . 48
- 6.2 Prim's Algorithm . . . 49
- 6.3 Kruskal's Algorithm . . . 52
- 6.4 Why the Greedy Choices Work: An Exchange Argument . . . 52
- 6.5 Chu-Liu/Edmonds Algorithm . . . 53

7 Adversarial Search and the Minimax Algorithm **57**
- 7.1 Problem Setting . . . 57
- 7.2 A Small Counterexample to Greedy Play . . . 58
- 7.3 Algorithm . . . 58
- 7.4 Alpha-beta pruning . . . 59
- 7.5 Depth-limiting search . . . 60
- 7.6 Expectimax . . . 61

8 PageRank and Centrality **62**
8.1 From Web Pages to a Mathematical Model . . . 62
8.2 Recommendations and the Random Surfer . . . 63
8.3 Existence, Uniqueness, and Convergence . . . 64
8.4 Dangling Pages, Rank Traps, and Teleportation . . . 65
8.5 Computing PageRank with the Power Method . . . 66
8.5.1 The Relationship to Transitive Closure . . . 67
8.5.2 The Computation on Our Web Graph . . . 67
8.6 Other Meanings of Centrality . . . 68
8.6.1 Degree Centrality . . . 68
8.6.2 Closeness Centrality . . . 69
8.6.3 Betweenness Centrality . . . 69
8.6.4 Eigenvector Centrality . . . 70

9 Flow Networks **71**
9.1 Maximum Flow Problem . . . 71
9.1.1 From a Transportation Network to a Flow . . . 71
9.1.2 Cuts Give an Upper Bound . . . 73
9.1.3 The Residual Graph: What Can We Still Change? . . . 73
9.1.4 Ford-Fulkerson Algorithm . . . 75
9.1.5 Why the Path Choice Matters . . . 77
9.1.6 Edmonds-Karp Algorithm . . . 77
9.2 Bipartite Matching . . . 78
9.2.1 From Matching to Maximum Flow . . . 79
9.2.2 Stable Marriage Problem . . . 80

10 Minimum-Cost Flow **83**
10.1 Supplies and Demands . . . 83
10.2 Residual Costs: Undoing a Decision Has a Price . . . 84
10.3 Klein's Cycle-Canceling Algorithm . . . 85
10.4 A More General Model . . . 86
10.5 A Glimpse of Linear Programming . . . 87
10.5.1 The Same Example in Excel Solver . . . 88

11 Graph Coloring **89**
11.1 Vertex Coloring and the Chromatic Number . . . 89
11.1.1 When Two Colors Suffice . . . 90
11.1.2 Back to Maps . . . 90
11.2 Planning a Sports Competition . . . 90
11.2.1 The Same Idea in Other Applications . . . 92
11.3 Greedy Coloring . . . 92
11.3.1 One Graph, Two Orders . . . 93
11.3.2 What Does Greedy Coloring Guarantee? . . . 94
11.4 Finding a Coloring and Finding the Optimum . . . 95

Choosing an Algorithm **96**

List of Figures

1 Is this a graph? . . . 6
2 A 2-regular graph with 4 vertices. . . . 8
3 Are these two graphs "the same"? . . . 10
4 Some special graphs. . . . 11
5 Two cycles sharing vertices A and D. All four vertices belong to one strongly connected component. . . . 19

6 Replacing each strongly connected component by one vertex produces a DAG. Both (X, Y, Z) and (X, Z, Y) are topological orderings. 20
7 The dodecahedron graph, which was used in Hamilton's Icosian Game. 21
8 The Seven Bridges of Königsberg (source: Wikimedia Commons, author: User:Twotwos, 17 September 2025), altered by adding labels to the bridges. 23
9 Different graphs for the Seven Bridges of Königsberg 23
10 Off-diagonal connections in the Boolean powers A^1 to A^6 and B^1 to B^3. These are graph drawings; diagonal entries are not shown. 29
11 Warshall's algorithm on a directed three-cycle. Dashed blue arrows and bold blue matrix entries show the connections added at each stage. Matrix rows and columns are ordered $1, 2, 3$. 30
12 Graph with negative cycle 32
13 A directed graph for comparing shortest-path algorithms. The numbers beside the edges are their weights. 33
14 Schematic drawing of vertex expansion in Dijkstra's vs A* algorithm 41
15 Pathological case where Algorithm 10 fails as the heuristic is not consistent. 42
16 Schematic drawing of vertex expansions in bidirectional search 44
17 Pathological example where the naive stopping condition fails to find the shortest path 44
18 Example of a tree (left) and a non-tree (right) 49
19 Comparison of arborescence and directed acyclic graph 50
20 The same undirected triangle: a shortest-path tree from A (left) and a minimum spanning tree (right). Solid colored edges are selected; the dashed edge is omitted. 51
21 Example of a directed graph where the greedy edge selection fails 53
22 Example of contracting a cycle 55
23 Choosing the root with $M = 55$: contract $\{a, b\}$, then $\{ab, c\}$; expand by entering at c, then at a. Removing R leaves $c \to a \to b$ of cost 4. 56
24 Minimax example 58
25 A small web graph. An arrow points from the page containing a link to its target. 63
26 Three problems of following only hyperlinks: a dangling page, a closed group of pages, and periodic motion. 65
27 The same web graph with its PageRank scores for $\alpha = 0.85$. 68
28 Two groups connected through a vertex of low degree. The interpretation of importance depends on the centrality measure. 69
29 The network from the introductory flow example in the slides. Capacities leaving s sum to 7, but the sink can receive at most 5. The right-hand drawing gives a feasible flow of value 5; labels there mean flow/capacity. 72
30 One original edge gives two possible adjustments. A forward residual edge permits adding flow; a backward residual edge permits cancelling flow. Residual labels give the maximum allowed adjustment. 74
31 All original capacities are one. After choosing s, A, B, t, no forward-only augmentation is possible. The residual path s, B, A, t cancels $A \to B$ and produces a flow of value two. 75
32 A legal but unfortunate path choice. Alternately using and cancelling the middle edge increases the flow by only one unit per augmentation. Large outer capacities do not help this choice of paths. 77
33 A bipartite graph and its flow network. Orange edges show a matching of size two and the corresponding flow. Every network edge has capacity one; the crossing of two drawn edges is not a vertex. 79
34 Improving a flow of fixed value by a negative residual cycle. The backward residual edges undo the expensive shipment. 85
35 Two models for four teams. Model A represents teams as vertices and games as edges. Model B represents games as vertices and conflicts as edges; its vertex colors specify days. The numbers identify colors even without a color printout. 91

36 The same graph colored greedily in two different orders. Interleaving corresponding vertices uses four colors; visiting one whole row first uses two. Color numbers are shown explicitly. Edge crossings are not additional vertices. 93

List of Algorithms

1 Tarjan's Algorithm for Finding Strongly Connected Components 18
2 Topological Sorting (Toposort) for DAGs 20
3 Fleury's Algorithm for Finding an Eulerian Trail or Cycle 26
4 Hierholzer's Algorithm for Finding an Eulerian Cycle 26
5 Algorithm for Computing the Transitive Closure 30
6 Shortest Path via Breadth-First Search 33
7 Algorithm for Computing All Pairs Shortest Paths 34
8 Bellman–Moore–Ford with a FIFO Queue 36
9 Dijkstra's Algorithm 38
10 A* Algorithm 42
11 A* Algorithm without Closed Set 43
12 Bidirectional Dijkstra 45
13 Iterative Deepening Depth-First Search (IDDFS) 46
14 Iterative Deepening A* (IDA*) 47
15 Prim's Algorithm (Minimum Spanning Tree) 50
16 Kruskal's Algorithm (Minimum Spanning Tree) 52
17 Recursive Minimax Algorithm 59
18 Alpha-beta Minimax 60
19 Depth-limited Minimax 61
20 PageRank by Power Iteration 66
21 Ford-Fulkerson Method 76
22 Edmonds-Karp Algorithm 78
23 Gale–Shapley with A Proposing 81
24 Klein's cycle-canceling algorithm 85
25 Greedy Vertex Coloring (First Fit) 93

1 Foundations of Graph Theory

What is a graph? Maybe you have seen graphs already. Typically these are drawn as some points that are interconnected by lines. Each line connects exactly two points. In this course, we will however need a formal definition and then reason about them. In Figure 1 you can see a drawing that already shows some complexity.

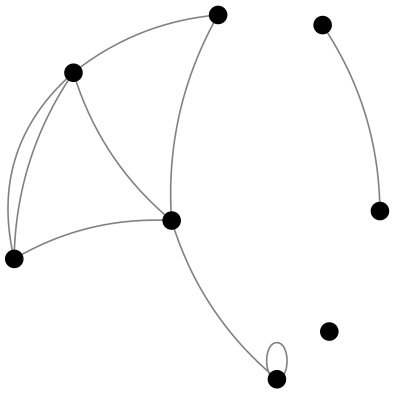


Figure 1: Is this a graph?

Beside the simple fact of "points connected by line", we can at least observe in Figure 1:

- There is a point that is not connected by any line
- There are two points connected, but not to the rest
- There are two points that are connected by two lines
- There is a point that is connected to itself

While mathematical descriptions may already be challenging to read, they are even harder to write. Do try to write down the formal definitions yourself. You will probably appreciate the definitions given here much more if you have thought about it yourself - even if they weren't perfect. Seeing where you made errors is a great way to learn.

Pause and Think – Quick Check

What properties can you deduce from this drawing? Describe what you see in the language of mathematics yourself before continuing!

1.1 Undirected Graphs

The points seem like atomic (non-decomposable) entities seemingly without further properties, except that we can probably identify each of them. To describe the points, we will therefore use the mathematical notation of a set. We call the set V the set of *vertices* (dt. Knoten, Ecken oder Punkte). Also a line seems like an identifiable, but otherwise atomic entity. We will therefore use the set E to describe the *edges* (dt. Kanten) - which represent the lines in the figure. Obviously, the set of edges is *somehow related* to the set of vertices, because an edge always ends at a point - a relationship that we have to express in mathematical terms.

We start with the simple case: no loops and at most one edge between any two vertices. We will return to parallel edges when modeling the bridges of Königsberg.

Definition 1.1. A simple undirected graph is a tuple $G = (V, E)$ where V is the finite set of vertices and $E \subseteq [V]^2$ is the set of edges.

Notation

Here $[V]^2 = \{\{u, v\} \mid u, v \in V,\ u \neq v\}$ denotes the set of all two-element subsets of V. An edge is an unordered pair: $\{u, v\} = \{v, u\}$, so it has no direction.

Pause and Think – Work It Out

Certainly, you can express $[V]^2 = \{\{u, v\} \mid u, v \in V \wedge u \neq v\}$, can you think of a general expression for $[V]^n$ with $n \in \mathbb{N}$? Hint, use the notion of the *power set* $\mathcal{P}(V)$.

Background

Remember, a set is a collection of objects in which each object appears only once. There is no order in a set, thus, e.g., there is no "first element". The power set is the set of all possible subsets of a set. In a tuple objects may appear multiple times and there is order. Sets are enclosed by "{" and "}", tuples by "(" and ")" in these notes. In programming, ordered sequences are often represented by lists or arrays.

Unless stated otherwise, our undirected graphs will be simple.
So that we have our definition of what a graph is, we can now start to reason about them. For this purpose, we want to explore some properties of undirected graphs and define them. Considering that we have two sets, we can reason about the "size" of a graph in two ways. We can say it has a lot of vertices and we can say it has a lot of edges.

Definition 1.2. The *order* (dt. Ordnung) of a graph $G = (V, E)$ is the number of vertices, i.e., $|V|$. The *size* (dt. Größe) of a graph is the number of edges, i.e., $|E|$.

For simple undirected graphs, the size is also bound by the order, because $|E| \leq |[V]^2|$.
Next, we observe that, in our simple graph, each edge connects exactly two vertices and that thus, we want to express that a vertex has other vertices as *neighbors*. We observe that some vertices have more neighbors than others and thus, we want to express the quantity of neighbors.

Pause and Think – Work It Out

Can you think of a way to express the fact that two vertices are connected by an edge? How would you express this in mathematical terms? It will be "obvious" if you read on, but still not so easy to come up with it yourself. Mathematical fluency can be acquired by practicing it - reading and writing.

Definition 1.3. We say two vertices $u, v \in V$ with $u \neq v$ are *adjacent* (dt. benachbart) and use the relationship symbol $\sim$ we say $u \sim v \Leftrightarrow \exists e \in E : u \in e \wedge v \in e$. We can also say an edge $e \in E$ and a vertex $v \in V$ are *incident* (dt. verbunden) to each other if $v \in e$.

We can use the "$\in$" operator on e, because remember E is a set of sets and e is a member of E and thus e is a set.
So, we have a formalism to express neighborship among vertices. With this definition we can now express the quantity of neighbors for a vertex.

Definition 1.4. The *degree* of a vertex v is the number of edges incident to it, i.e., $\deg(v) = |\{e \in E \mid v \in e\}|$.

Background

For our simple graph it makes no difference if we use the amount of "adjacent vertices" or the amount of "incident edges" within the definition of the degree. However, would we be talking about multi-graphs, there is a substantial difference.

Now, we want to express the neighborhood of a vertex.

Definition 1.5. The set of all vertices adjacent to a vertex v is called the *neighborhood* of v and is denoted by $N(v) = \{u \in V \mid u \sim v\}$.

For simple graphs it holds that $|N(v)| = \deg(v)$, because each edge connects exactly two vertices. However, in multi-graphs the degree of a vertex can be larger than the size of its neighborhood. A vertex without any neighbors has degree 0 and is called an *isolated vertex* (dt. isolierter Knoten).

In addition, there is also a maximal and a minimal degree of a graph which is the respective maximum and minimum of the degrees of all vertices in the graph.

Definition 1.6. The *maximum degree* (dt. Maximalgrad) of a graph $G = (V, E)$ is denoted $\Delta(G) = \max_{v \in V} \deg(v)$, and the *minimum degree* (dt. Minimalgrad) is denoted $\delta(G) = \min_{v \in V} \deg(v)$.

There are also graphs for which $\Delta(G) = \delta(G)$. These are called *regular graphs* (dt. reguläre Graphen).

Definition 1.7. A graph is regular when $\exists r \in \mathbb{N} : \forall v \in V : \deg(v) = r \Leftrightarrow \Delta(G) = \delta(G) = r$. Such a graph is called an *r-regular graph* (dt. r-regulärer Graph).

Pause and Think – Work It Out

Think of a 3-regular graph with 6 vertices. What would it look like? Can you also draw a 3-regular graph with 5 vertices?

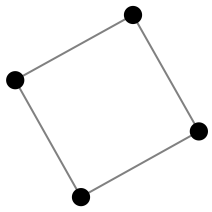


Figure 2: A 2-regular graph with 4 vertices.

If you tried to draw a 3-regular graph with 5 vertices, you find that, no matter what you try, it just doesn't seem possible. Is there really no way to construct a 3-regular graph with 5 vertices? Let's look at this mathematically.

Given a simple undirected graph $G = (V, E)$ it must hold that

Theorem 1.8.

$$\sum_{v \in V} deg(v) = 2 \cdot |E|$$

Proof. Each edge is counted exactly two times: one time for each end of the edge and there are exactly two ends for each edge. There is no edge without ends in V. If we count each edge exactly two times, we have counted twice the number of edges. □

Since this number is a multiple of two, it must be an *even number*. Our hypothetical graph with a degree of 3 for each of its 5 vertices would amount to a total degree of 15 - an odd number. Such a "graph" would violate Theorem 1.8 and thus, cannot exist. But, we can make an even stronger statement:

Theorem 1.9 (Handshaking Lemma)**.** *In a simple undirected graph, there must be an even number of vertices with an odd degree. More formally, let $G = (V, E)$ be a simple undirected graph. Then it holds that $|\{v \in V \mid deg(v) \notin \mathbb{N}_e\}| \in \mathbb{N}_e$, where $\mathbb{N}_e$ is the set of even numbers (by definition $0 \in \mathbb{N}_e$).*

Proof. We already know that $\sum_{v \in V} \deg(v)$ is an even number by Theorem 1.8. We can split this sum into two sums, one over all vertices with even degree and one over all vertices with odd degree:

$$\sum_{v \in V} \deg(v) = \sum_{v \in V | \deg(v) \in \mathbb{N}_e} \deg(v) + \sum_{v \in V | \deg(v) \notin \mathbb{N}_e} \deg(v)$$

We know that if $z = x + y$ and $z \in \mathbb{N}_e$, then x and y must have the same parity.
We see that the first sum on the right-hand side results in an even number, because it is a sum of even numbers (the vertex degrees are even).
Thus, the second sum must also result in an even number. But, the second expression sums only odd numbers (the vertex degrees are odd).
A sum of odd numbers is even if and only if there is an even amount of odd numbers in the sum.
Thus, there must always be an even number of vertices with an odd degree. □

Again we see that a 3-regular graph with 5 vertices violates the Handshaking Lemma. There are an odd number of vertices (5) that have an odd degree (3).

1.2 Directed Graphs

We formally introduce a directed graph as a 2-tuple $G = (V, E)$ where V is the set of vertices and E is the set of edges, similar to undirected graphs.

Pause and Think – Work It Out

Try to think about how *you* would express the directedness of an edge! Obviously, an edge has to be "something with two vertices" and we need to clearly express that one is the start and the other is the end.

To express the directedness, we say that an edge is a tuple (pair) of two vertices. As you recall, in a tuple the order matters and (u, v) and (v, u) are different tuples. We define that the first element in the pair is the start vertex (dt. Startknoten) and the second element is the end vertex (dt. Endknoten).

Definition 1.10. The edges in a graph E are defined as a set of ordered pairs of vertices, i.e., $E \subseteq V \times V$. We say an edge $e = (u, v)$ is directed from u to v and we write $u \to v$.

Note, that the cross product $V \times V$ also includes pairs of the same vertex (u, u), i.e., self-loops. Thus, in contrast to simple undirected graphs, the simple directed graph does not exclude self-loops.

Pause and Think – Quick Check

Answer for yourself: Does this definition of directed graphs also allow for multiple edges between two vertices? Why? Why not?

We also have a more distinguished relationship between vertices in directed graphs. Given a directed graph $G = (V, E)$ and an edge $e = (u, v) \in E$ with $u \neq v \wedge u, v \in V$ we say the vertex u is the *predecessor* (dt. Vorgänger) of v and v is the *successor* (dt. Nachfolger) of u.
Similar to the neighborhood, we now have the *successors* and *predecessors* of a vertex.

Definition 1.11. The *successors* of a vertex v in a directed graph $G = (V, E)$ is the set of all vertices that an edge originating from v points to, i.e., $Suc(v) = \{u \in V \mid (v, u) \in E\}$.

The *predecessors* of a vertex v in a directed graph $G = (V, E)$ is the set of all vertices from which an edge points to v, i.e., $Pre(v) = \{u \in V \mid (u, v) \in E\}$.

Likewise, we must also refine the notion of the degree of a vertex. In directed graphs, we have two kinds of degrees: the *in-degree* (dt. Eingangsgrad) and the *out-degree* (dt. Ausgangsgrad). For undirected graphs Suc and Pre are identical to the neighborhood N.

Definition 1.12. The *in-degree* of a vertex v is the number of edges directed into it, i.e., $\text{indeg}(v) = |\{(a, b) \in E \mid v = b\}|$.

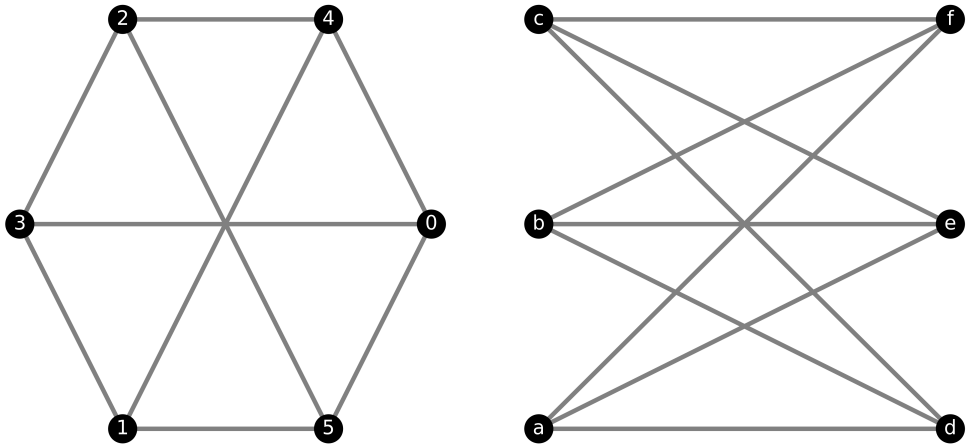
The *out-degree* of a vertex v is the number of edges directed out of it, i.e., $\text{outdeg}(v) = |\{(a, b) \in E \mid v = a\}|$.

Pause and Think – Quick Check

According to this definition, how does a loop affect the in-degree and out-degree of a vertex? The answer is in the definition, you just have to parse it. Mathematical definitions need to be exact and unambiguous.

1.3 Isomorphism of Graphs

One question that we have escaped so far is when two graphs are "equal". Naturally, true equality can only be achieved when the vertex set and the edge set are exactly the same, however often two graphs are just labelled differently, but still look the same. Consider the two graphs shown in Figure 3, are they "the same"?



(a) (b)

Figure 3: Are these two graphs "the same"?

We can perceive that two graphs are "the same" when we can describe one of them with the labels of the other. In mathematics, we say that there is a morphism that transforms one graph into the other.

Definition 1.13. Two graphs $G = (V, E)$ and $H = (U, F)$ are called *isomorphic* (dt. isomorph) if there exists a bijective function $f : V \to U$ such that for every pair of vertices $a, b \in V$, it holds that $a \sim b \Leftrightarrow f(a) \sim f(b)$ (trivially it must hold that $|V| = |U| \wedge |E| = |F|$).

We can see that in this definition, the isomorphic graphs preserve neighborhood relationships. That is, if two vertices are adjacent in one graph, their images under the mapping function f are also adjacent in the other graph. While this also implies that the degrees of corresponding vertices are the same, it is not sufficient to just compare the degree sequences of two graphs to determine isomorphism.

Determining whether two graphs are isomorphic is unfortunately a rather difficult problem and is known as the *graph isomorphism problem*. While this may look rather innocent and easy at first, its complexity class is actually still unknown. If we assume $n = |V| = |U|$, there are $n!$ possible bijective functions between the vertex sets of the two graphs. Naturally, this number grows extremely fast, at just 20 vertices, there are already $2,432,902,008,176,640,000$ possible

mappings. Thus, a brute-force approach by checking every possible mapping is entirely infeasible for even moderately sized graphs.
László Babai announced a major breakthrough in 2015 and presented it at STOC in 2016: graph isomorphism can be decided in quasipolynomial time $\exp((\log n)^{O(1)})$, where n is the number of vertices in each of the two graphs [1]. Here $\exp(x) = e^x$, and the $O(1)$ in the exponent stands for a fixed constant. Taking log to be the natural logarithm, $e^{\log n} = n$, but raising $\log n$ to a power changes the growth considerably. For example,

$$e^{(\log n)^2} = (e^{\log n})^{\log n} = n^{\log n}.$$

This example grows faster than every fixed power of n, yet much slower than 2^n. The quasipolynomial bound therefore does not give us a linear-time or even a polynomial-time algorithm. It remains unknown whether the general graph-isomorphism problem has a polynomial-time algorithm or is NP-complete.

1.4 Graph Families

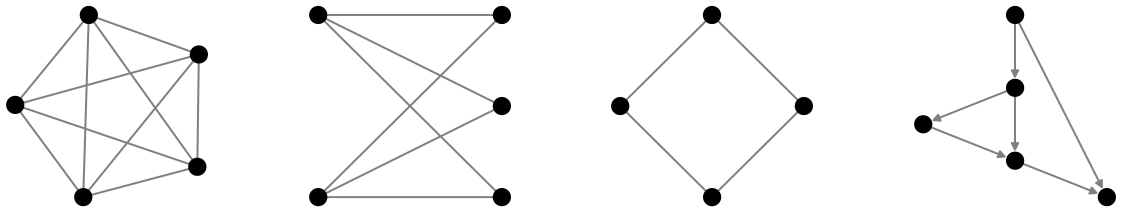
In the literature, there are some special graphs with specific properties that are explicitly named. Among those are:

Complete graphs (dt. vollständige Graphen): A complete graph is a simple undirected graph where each vertex is adjacent to every other vertex. Since the edges are determined we use K_n to denote a complete graph with n vertices. The complete graph K_n is $(n-1)$-regular. The K_0 with no vertices, as well as K_1 with no edges also exist as trivial cases.

Bipartite graphs (dt. bipartite Graphen): A bipartite graph is a simple undirected graph whose vertices can be divided into two disjoint sets U and V such that every edge connects a vertex in U to a vertex in V. There are no edges between vertices within the same set. A *complete bipartite graph* is a bipartite graph where every vertex in U is connected to every vertex in V. We use $K_{m,n}$ to denote a complete bipartite graph where $|U| = m$ and $|V| = n$.

Cycles (dt. Kreise): A cycle is a simple undirected graph where the vertices can be arranged in a circle such that each vertex is adjacent to exactly its two neighboring vertices in the circle. We use C_n to denote a cycle with n vertices. Every cycle C_n is 2-regular and every C_n with $n = 2 \cdot k,\ k \in \mathbb{N}$, i.e., an even number of vertices, is also a bipartite graph.

DAGs (dt. azyklischer gerichteter Graph): A **d**irected **a**cyclic **g**raph is a special subfamily of directed graphs that *contain no cycles*, i.e., starting at vertex v and following a sequence of directed edges we cannot eventually loop back to v again. DAGs can be found in many applications, such as representing dependencies in task scheduling, data processing pipelines, and version control systems.



(a) K_5 (b) $K_{2,3}$ (c) C_4 (d) A DAG

Figure 4: Some special graphs.

Figure 4 shows some examples of these special graphs. Consider the property of a cycle being bipartite. In C_4 (Figure 4c) and even cycles in general, we can apply a simple coloring rule. As we navigate the edges, we will color the vertices with an alternating scheme of two colors until all vertices are colored. If we never observe a conflict, e.g. color an already colored vertex with the

opposite color then the graph is bipartite. This works for C_4, but doesn't for C_5, because we reach the starting vertex with the opposite color.

Theorem 1.14. *An undirected graph is bipartite if and only if it contains no cycles of odd length.*

There are many other named graphs, such as the Petersen graph. However, we will not explore them here.

1.5 Subgraphs

We may find that sometimes we want to reason only about a part of a graph. Consider a global supply chain where vertices denote suppliers, producers, storage facilities, and consumers that are connected by edges that denote the flow of materials using some means of transportation. In this graph, if we want to reason only about all vertices and edges that are related to a specific material, we may want to have the ability to "extract" a part of the graph. This is what we call a *subgraph*.

Definition 1.15. A subgraph (dt. Teilgraph) $H = (V_H, E_H)$ of a graph $G = (V, E)$ is a graph where $V_H \subseteq V$ and $E_H \subseteq E$.

Since we say that H is a graph, it must also satisfy the properties of a graph that we have defined above. Thus, we don't need to explicitly state that $E_H \subseteq [V_H]^2$, respectively $E_H \subseteq V_H \times V_H$ for directed graphs, because this is already implied when we say "this is a graph".

Pause and Think – Work It Out

Sometimes we want to say, that a subgraph is formed by considering only a subset of the vertices and have it retain all edges in the original graph between these vertices. Likewise, we may want to form a subgraph from a subset of the edges and have it retain all vertices that are incident to these edges. Try to express this in mathematical terms, before reading on.

A subgraph formed from a subset of the vertices is called *vertex-induced subgraph* (dt. knoteninduzierter Teilgraph) and a subgraph formed from a subset of the edges is called *edge-induced subgraph* (dt. kanteninduzierter Teilgraph).

Definition 1.16. A *vertex-induced subgraph* $H = (V_H, E_H)$ of a graph $G = (V, E)$ is a subgraph where $V_H \subseteq V$ and $E_H = \{\{u, v\} \in E \mid u \in V_H \wedge v \in V_H\}$.

An *edge-induced subgraph* $H = (V_H, E_H)$ of a graph $G = (V, E)$ is a subgraph where $E_H \subseteq E$ and $V_H = \{v \in V \mid \exists e \in E_H : v \in e\}$.

Notation

We use the notation $\langle V \rangle$ to describe a vertex-induced subgraph and $\langle E \rangle$ to describe an edge-induced subgraph. Thus, we can also write $H = \langle V_H \rangle$ or $H = \langle E_H \rangle$, depending on which subgraph we want. The angle brackets can thus be seen as an operator (function if you want) that produce a graph from a given set.

We use the same operator for vertex sets and edge sets and so the semantics matter. This is a bit sloppy. To be more precise we would have to introduce two operators, e.g. $\langle_v \cdot \rangle_v$ for vertex-induced subgraphs. However, it would make notation more complex to read and is it really clearer?
Subgraphs that contain the same set of vertices have a special name. They are called *spanning subgraphs* (dt. spannender Teilgraph). We'll come back to these in a later section.

Definition 1.17. A subgraph $H = (V_H, E_H)$ of a graph $G = (V, E)$ is called a *spanning subgraph* if $V_H = V$ and $E_H \subseteq E$.

Pause and Think – Quick Check

Is every graph a subgraph of itself? Why? Why not?

1.6 Weighted Graphs

In many applications, edges have numbers associated with them, so called *weights*. For instance, in a transportation network the edges may represent roads between cities (the vertices) and the weight may represent the distance or travel time between the cities. In social networks, edges may represent connections between people (the vertices) and the weight may represent the number of interactions.

Pause and Think – Work It Out

How would you express weights mathematically? Think about it before reading on. It's trivial once you see it, but perhaps not as easy to come up with it yourself. Sharpen your mathematical expression skills by trying it out yourself.

Often students suggested to extend the definition of an edge, e.g., an edge $e = \{u, v, w\}$. This is a typical programmer's solution and perhaps that was your choice as well? But, from a mathematical standpoint it's not as elegant. How to identify the weight? Using something like "the weight is that element in the edge-set that is not a vertex" is perhaps possible, but rather complicated. For directed graphs, we can distinguish the elements, but we'd have to address the properties by index and remember that index 3 is the weight (as in a tuple-based edge list). A more general, simpler and more elegant method of expressing a property of an object mathematically is by using a *function* that maps from the set of objects to the set of property-values.

Definition 1.18. A *weighted graph* is a tuple $G = (V, E, \omega)$ where V is the set of vertices, E is the set of edges, and $\omega : E \to Z$ is a weight function that maps edges to a number from the numeric set Z - where $Z \in \{\mathbb{N}, \mathbb{Z}, \mathbb{R}, \ldots\}$ - to each edge.

For some algorithms having positive weights ($Z = \mathbb{N}$ or $Z = \mathbb{R}^+$) is a requirement, sometimes integrality is necessary for performance reasons or to obtain integral results.
For example, if an edge $e = \{A, B\}$ represents a road taking twelve minutes, write $\omega(e) = 12$. The endpoints stay in e; the function supplies its travel time. A second function could assign a toll to that same edge.

Digression

In programming, this connects to object-oriented programming (OOP) and data-oriented design (DoD). OOP commonly groups an object's properties in a struct; many such objects form an *array of structs*. DoD may instead use a *struct of arrays*, with an object's identity acting as an index into those arrays. This can improve cache efficiency when processing the same property of many objects. Mathematical functions offer a related way to associate properties with otherwise atomic objects. Entity-component-system (ECS) architectures in games also combine well with data-oriented techniques. These are useful connections, but the best representation depends on the application.

2 Walks and Connectedness

We have already created quite some formalism to reason about graphs. However, there is an aspect that we still cannot describe. Remember the drawing in Figure 1? We recognized that there were some vertices that seemed "separated" and were not connected to the others. We cannot express this property with the definitions we have. Maybe you think the degree of a vertex could be used. However, the degree is a property of a single vertex. That some vertices are not connected to others seems more like a property of the graph as a whole.
We have to invent new terms and concepts and define them formally if want to reason about such properties.

Pause and Think – Work It Out

Again, try to think about it, before reading on. How would you approach this? What concepts do we need to introduce? How are these defined? Try it out yourself and have some fun with math. Don't worry if you don't get it correct or right. Maybe you got the right idea, but struggled to describe it, maybe it was the wrong idea overall. But, as you do this, you've probably learned quite a lot! Potentially, the inventors of graph theory also struggled, before they found the right definitions. This is a creative process.

Definition 2.1. A walk (dt. Wanderung) in a graph is a tuple $W_{a,b} = (S_V, S_E)$ where $S_V = (v_0, v_1, \ldots, v_n)$ is a sequence of vertices and $S_E = (e_1, e_2, \ldots, e_n)$ is a sequence of edges such that

$$v_i \in V \quad \forall i \in \{0, 1, \ldots, n\} \tag{1}$$
$$e_i \in E \quad \forall i \in \{1, 2, \ldots, n\} \tag{2}$$
$$e_i = \{v_{i-1}, v_i\} \quad \forall i \in \{1, 2, \ldots, n\} \tag{3}$$
$$v_0 = a \tag{4}$$
$$v_n = b \tag{5}$$

In this definition, $a = v_0$ is the start vertex and $b = v_n$ is the end vertex of the walk. A walk is also possible to describe in directed walks, but since edges are tuples, we have to say $e_i = (v_{i-1}, v_i)$ in Equation (3). Additionally, since a walk is determined by either S_V or S_E alone for simple graphs, we can also use the notation of a walk and specify only one of the sequences.

Background

Two aspects to consider in this digression. First, note that $v_0, v_1, \ldots$ and such are not by themselves vertices, but rather variables. Previously we used variables like u and v to represent elements of the set V. Here, we additionally associate a position in the sequence with each variable - denoted by the index.

Pause and Think – Quick Check

Now we're almost there. We have defined a new object and its properties. If you ever worry if something is a walk, then you need to look at the definition. Can you already see how we can express connectedness now?

We will call the concept that some vertices cannot be reached from others by following edges *connectedness* (dt. Zusammenhang) and define it formally using the concept of a walk.

Definition 2.2. A graph $G = (V, E)$ is called *connected* (dt. zusammenhängend) if for *every* pair of vertices there exists at least one walk. That is, $\forall u, v \in V : u \neq v \; \exists \, W_{u,v} = (S_V, S_E)$ is a walk.

Definition 2.3. Given a walk between vertices u and v: $W_{u,v} = (S_V, S_E)$ the length of the walk $\mathcal{L}_1(W_{u,v}) = |S_E|$ is the number of edges in the sequence. The *trivial walk* $W = ((u), ())$, for which the start and end vertex are the same, is the shortest possible walk and has a length of 0.

Definition 2.4. A walk where start and end are the same is called *closed* (dt. geschlossen), while the opposite, namely a walk that ends at a different vertex than where it started, is called *open* (dt. offen).

Understanding the definition of a walk, we see that there is no restriction on whether we can revisit a vertex or use the same edge multiple times in a walk. We could simply create a very long walk by jumping back and forth between two vertices. For instance, $S_V = (a, b, a, b, a, b, a, b, a, b, a)$ of a closed walk has length 10 if $a \sim b$. Sometimes, placing restrictions on the occurrences of an edge or vertex is desired however, and thus we want to introduce:

Definition 2.5. A *trail* (dt. Weg) is a walk in which no edge is repeated:
$\forall e_i, e_j \in S_E : \; e_i = e_j \Leftrightarrow i = j$.

Definition 2.6. A *path* (dt. Pfad oder einfacher Weg) is a trail in which no vertex is repeated:
$\forall v_i, v_j \in S_V : \; v_i = v_j \Leftrightarrow i = j$.

The trivial walk is the only closed *path*. We have already talked about Cycles earlier. A closed trail where the start and end vertex are the only vertices that occur twice also forms a cycle. Also from this definition it is clear that every path is also a trail and every trail is also a walk. This also means that if there exists a path between two vertices, then there also exists a walk between them.

Definition 2.7. We define the operator $\circ$ to denote the concatenation of tuples. Consider $A = (a_0, a_1, \ldots, a_n)$ and $B = (b_0, b_1, \ldots, b_m)$. Their concatenation is $A \circ B = (a_0, a_1, \ldots, a_n, b_0, b_1, \ldots, b_m)$.

We also use $\circ$ for walks, but only when the end of the first equals the start of the second. If $S_V^1 = (v_0, \ldots, v_n)$ and $S_V^2 = (v_n, u_1, \ldots, u_m)$, then

$$W_1 \circ W_2 = ((v_0, \ldots, v_n, u_1, \ldots, u_m),\; S_E^1 \circ S_E^2).$$

The shared vertex occurs once at the join; all edges are retained. For example, joining the walks (a, b) and (b, c) gives (a, b, c). Ordinary tuple concatenation, as used for appending entries to algorithmic lists, still keeps every entry.

Definition 2.8. The set of all possible walks in a graph G is denoted as $\mathcal{W}(G)$.

Pause and Think – Quick Check

Can it be said that $\mathcal{W}(G)$ is finite or infinite for finite graphs? Why? Why not?

2.1 Connected Components, Bridges, Distance, and Diameter

Following these definitions, we can now express mathematically the fact that a graph is "separated" into parts. And we do call the graph *disconnected* (dt. unzusammenhängend) as opposed to a connected (dt. zusammenhängend) graph.
However, we note that there may be more to connectedness than it just being a binary property. Some graphs have multiple disconnected parts and we also want to put a name to these "parts". Consider that we can define a relation R between any two vertices $u, v \in V$ whether there exists a path between them.

$$uRv \Leftrightarrow \exists\, W : W \text{ is a path from } u \text{ to } v$$

A relation can have many properties, the relation R is

- *reflexive* (dt. reflexiv): uRu for all $u \in V$,
- *symmetric* (dt. symmetrisch): if uRv, then vRu for all $u, v \in V$, and
- *transitive* (dt. transitiv): if uRv and vRw, then uRw for all $u, v, w \in V$.

It is reflexive, because for every vertex $u \in V$ we can construct a trivial walk $W = (u)$. It is symmetric, because, we can reverse the path (this only applies to undirected graphs). Finally, concatenate the two paths to obtain a walk from u to w. It may repeat vertices elsewhere even though the join is correct. Whenever a vertex occurs twice, erase the closed portion between those occurrences. Repeating this step gives a path from u to w (the trivial path if $u = w$), proving transitivity.
Given these three properties, we say R is an equivalence relation (dt. Äquivalenzrelation). We can use R to partition the set of vertices into equivalence classes (dt. Äquivalenzklassen). Each equivalence class is a set of vertices that are all reachable from each other, i.e., for every vertex u in the equivalence class there exists a path to every other vertex v in the equivalence class.

Definition 2.9. A *connected component* (dt. Zusammenhangskomponente) of a graph $G = (V, E)$ is an equivalence class of the relation R.

Pause and Think – Work It Out

Is connectedness also guaranteed in subgraphs? Is there anything we can claim about the number of connected components in a subgraph?

For a connected graph, there exists only one connected component. Removing an edge can increase the number of connected components, adding an edge can decrease the number of connected components.

Definition 2.10. A *bridge* (dt. Brücke) in a graph $G = (V, E)$ is an edge $e \in E$ such that removing it from the graph increases the number of connected components, i.e., $|C(G)| < |C(H)|$ with $H = (V, E \setminus e)$ where $C(G)$ is the set of connected components of the graph G.

Why This Matters

Bridges are important in network design and network analysis. They represent critical connections in a network, and their removal can lead to network fragmentation. In the context of the internet, for example, bridges can represent crucial links between different networks or regions. Identifying and protecting bridges is essential for maintaining network reliability and robustness. In supply chains, the discovery of bridges in the supply network is also highly relevant for risk management.

Definition 2.11. The *distance* (dt. Abstand) $d(u, v)$ between two vertices $u, v \in V$ in a graph $G = (V, E)$ is the length of the shortest path between u and v, i.e., $d(u, v) = \min_{W_{u,v}} \{\mathcal{L}(W_{u,v}) \mid W_{u,v} \text{ is a path from } u \text{ to } v\}$.

Definition 2.12. The *diameter* (dt. Durchmeser) of a graph $G = (V, E)$ is the longest distance between any two vertices in the graph. It is denoted as $d(G) = \max_{u,v \in V} d(u, v)$.

Returning to what we said earlier, we defined *walks*, *trails*, and *paths*. We now are able to more formally define *cycles*.

Definition 2.13. A *cycle* is a closed trail $S_V = (v_0, v_1, \ldots, v_{n-1}, v_0)$ where $\forall_{i,j \in \{1,2,\ldots,n-1\}} : i \neq j \Rightarrow v_i \neq v_j \wedge v_i \neq v_0$.

The length of the shortest cycle in a graph G is called the *girth* (dt. Taillenweite) of the graph and is denoted as $g(G)$, while the length of the longest cycle is called the *circumference* (dt. Umfang) of the graph and is denoted as $c(G)$. For graphs that have no cycles, $g(G) = \infty$ and $c(G) = 0$ by definition.

Theorem 2.14. *Every undirected graph $G = (V, E)$ has a path of at least length $\delta(G)$.*

Proof. Assume a *longest* path $S_V = (v_0, v_1, \ldots, v_n)$ in G where n is the length of the path. For S_V to be a longest path means that every neighbor of v_n must be in that path, or otherwise the path wasn't the longest as we could very easily extend it by appending that neighbor. Thus, $n \geq deg(v_n) \geq \delta(G)$. □

Theorem 2.15. *Every undirected graph G with $\delta(G) \geq 2$ has a cycle of at least length $\delta(G) + 1$.*

Proof. From Theorem 2.14 we know that there exists a path of at least $\delta(G)$. Again, assume a longest path $S_V = (v_0, v_1, \ldots, v_n)$. The vertex v_n has at least two neighbors, because $\delta(G) \geq 2$ (given). Choose $i \in \mathbb{N}$ with $i < n$ to be the smallest i such that $v_i \sim v_n$. v_i is then the neighbor of v_n that we first visit in the path. Then, we can form a cycle by going back from v_n to v_i, i.e., $C = (v_i, v_{i+1}, \ldots, v_n, v_i)$. This cyle contains *all* neighbors of v_n (as i was the first neighbor). In additon, C has one more edge (which closes the loop) an thus, the length of the cycle must be at least $\delta(G) + 1$. □

As we have seen in the definition of connected components, we required our relation to be symmetric. However, for directed graphs, symmetry is not guaranteed. When there is a path from u to v, we cannot assume that a $v - u$ path exists as well. Thus, for directed graphs, we have to use a different definition of connectedness.

Definition 2.16. Given a directed graph $G = (V, E)$, we can construct an undirected graph $H = (V, X)$ where $X = \{\{u, v\} \in [V]^2 \mid (u, v) \in E \vee (v, u) \in E\}$. We say G is *weakly connected* (dt. schwach zusammenhängend) if H is connected.

In a weakly connected directed graph, we cannot reach every vertex from every other. Thus, if we want to express this stronger notion of connectedness, we also have to introduce a stronger definition of connectedness for directed graphs.

Definition 2.17. A directed graph $G = (V, E)$ is called *strongly connected* (dt. stark zusammenhängend) if for every pair of vertices $u, v \in V$ there exists a path from u to v as well as a path from v to u.

Similarly, we can create an equivalence relation R given that we require both paths and therefore explicitly stating that the relation is symmetric. The equivalence classes are then called *strongly connected components* (dt. starke Zusammenhangskomponenten).
Note that the definition, doesn't say how we obtain the components, nor whether we can compute them efficiently. We require algorithms for this purpose.

Pause and Think – Programming Exercise

Can you think of an algorithm to find all connected components of a directed graph? Try it yourself, implement it and see whether it works and how it performs. Maybe, you find a completely new and highly efficient approach that the world doesn't yet know about! Tell the world! Maybe you failed in coming up with an algorithm or it performs rather poorly - it doesn't matter. You still learned quite a lot about the problem and its challenges and this knowledge will help you understand existing algorithms.
You are not convinced that new algorithms can be invented? Graph theory is still an active research field and more efficient algorithms are still being discovered. One example is a deterministic almost-linear-time algorithm published in 2023 for exact maximum flow and minimum-cost flow on directed graphs with polynomially bounded integral demands, costs, and capacities [2]. But perhaps you first want to deepen your understanding of the field of graph theory.

2.1.1 Tarjan's Algorithm for Finding Strongly Connected Components

Tarjan's algorithm as presented in Algorithm 1 efficiently finds all strongly connected components. These components help us locate cycles in a directed graph. In some applications, cycles are undesirable, e.g., when we schedule tasks with dependencies. If the dependency graph has a cycle, we cannot find a valid schedule. Such dependency graphs occur in applications such as transactions in a database or web services.

Algorithm 1 Tarjan's Algorithm for Finding Strongly Connected Components

Input: A directed graph $G = (V, E)$
Output: The strongly connected components of G

```
1: index ← 0
2: S ← an empty stack
3: ∀_{v∈V} node_index[v] ← undefined
4: ∀_{v∈V} node_link[v] ← undefined
5: onStack ← {}                                          ▷ An empty set
6: SCCs ← ()                                             ▷ An empty list (of lists) - the result
7: for v ∈ V do
8:     if node_index[v] = undefined then
9:         StrongConnect(v)
10:    end if
11: end for
12: function StrongConnect(v)
13:    node_index[v] ← index
14:    node_link[v] ← index
15:    index ← index + 1
16:    push(S, v)
17:    onStack ← onStack ∪ {v}
18:    for (v, w) ∈ E do                                 ▷ All successors of v
19:        if node_index[w] = undefined then
20:            StrongConnect(w)
21:            node_link[v] ← min(node_link[v], node_link[w])
22:        else if w ∈ onStack then
23:            node_link[v] ← min(node_link[v], node_index[w])
24:        end if
25:    end for
26:    if node_link[v] = node_index[v] then
27:        scc ← ()                                      ▷ An empty list
28:        repeat
29:            w ← pop(S)
30:            onStack ← onStack \ {w}
31:            scc ← scc ∘ (w)
32:        until w = v
33:        SCCs ← SCCs ∘ (scc)
34:    end if
35: end function
36: return SCCs
```

The algorithm is recursive and embeds a depth-first search (DFS). Its recursion stack tracks the current DFS path. The separate stack S holds discovered vertices not yet assigned to a component; a vertex can remain in S after its recursive call has finished. The arrays store discovery indices and lowlink values ($node_{link}$ in Algorithm 1). A lowlink summarizes how far back the DFS subtree can connect to a vertex still in S, using tree edges followed by at most one edge to that stack. When a vertex is found that has the same index and lowest reachable index, a strongly connected component is identified, and all vertices in that component are popped from the stack.

Example: two cycles in one component. Consider the graph in Figure 5. It contains the cycles $A \to B \to C \to D \to A$ and $A \to D \to A$. The vertices A and D can reach each other, but they do not form a strongly connected component of their own: a component contains *all* vertices mutually reachable with its members. Here B and C belong to the same component, because the longer cycle lets us travel from any of the four vertices to any other. Thus the only strongly connected component is $\{A, B, C, D\}$.

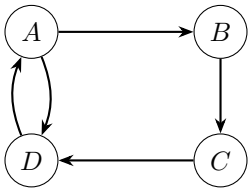


Figure 5: Two cycles sharing vertices A and D. All four vertices belong to one strongly connected component.

Start the DFS at A and explore $A \to B$ before $A \to D$. The vertices receive indices $0, 1, 2, 3$ in the order A, B, C, D. When D finds the edge back to A, its lowlink becomes 0. Returning through the recursive calls passes this value to C and then B:

Step	Lowlink update	Stack S (bottom to top)
Discover A, B, C, D	Initially $0, 1, 2, 3$	(A, B, C, D)
Examine $D \to A$	$node_{link}[D] \leftarrow 0$	(A, B, C, D)
Return from D to C	$node_{link}[C] \leftarrow 0$	(A, B, C, D)
Return from C to B	$node_{link}[B] \leftarrow 0$	(A, B, C, D)
Finish A	Lowlink and index are both 0	$()$

The calls for D, C, and B finish without removing these vertices from S: their lowlinks are smaller than their own indices. Back at A, the remaining edge $A \to D$ changes nothing. Only when A finishes do lowlink and index agree, so the algorithm pops D, C, B, A together and returns one component.

Let's analyze the time complexity of Tarjan's algorithm. The algorithm performs a depth-first search (DFS) on the graph, which takes $O(V + E)$ time, where V is the number of vertices and E is the number of edges in the graph. Each vertex is visited exactly once, and each edge is followed up on exactly once during the DFS traversal.

Implementation Note

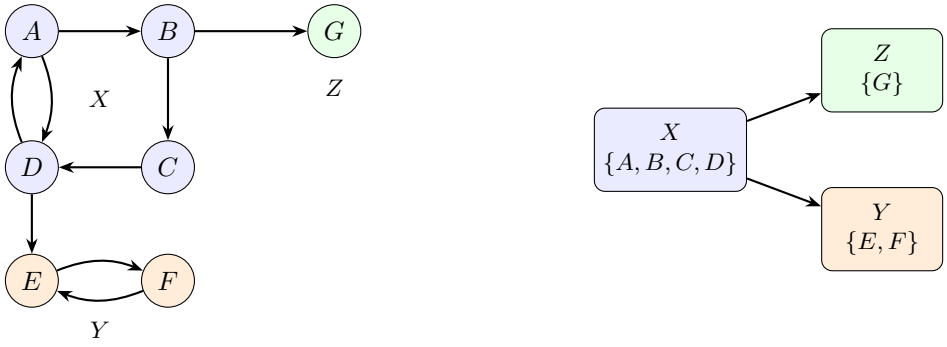
$O(V + E)$ is linear in the input size when the graph is represented by adjacency lists. For sparse graphs (dt. dünn besetzte Graphen), $E = O(V)$, so it is also linear in the number of vertices. For dense graphs, $E = \Theta(V^2)$, making it quadratic in vertex count while remaining linear in the adjacency-list input size. Scanning a full adjacency matrix instead costs $O(V^2)$. Why doesn't the V-part in the O-notation vanish? Why don't we just say that the algorithm is $O(E)$? Because we want to express that, even for a graph with no edges at all, we still have to visit all the vertices at least once.

2.1.2 Topological Sorting

If we examine the output of Tarjan's algorithm, we may note an additional property. If we consider the sequence of returned components as $(c_1, c_2, \ldots, c_n)$, we observe that any vertex in c_1 may have only edges to other vertices in c_1, but not to vertices in c_2 or higher. Any vertex in the second component c_2 returned may have edges only to vertices in c_1 and c_2, but not to c_3 or higher. In general, a vertex in component c_i with $1 \leq i \leq n$ may have edges only to vertices of component c_j with $j \leq i$.

To see this ordering more clearly, replace each strongly connected component by one vertex. Draw an edge between two of these vertices whenever an edge in the original graph leads from one component to the other. The resulting *condensation graph* is a DAG: a cycle between components would make them mutually reachable, so they would form a single component. A *topological ordering* lists the vertices of a DAG so that every edge points from an earlier vertex to a later one. Tarjan returns the components in reverse topological order; reversing that list gives a topological ordering of the condensation graph.

Example: ordering components. Extend the previous graph by adding $D \to E$, $E \to F$, $F \to E$, and $B \to G$, as shown in Figure 6. Its components are $X = \{A, B, C, D\}$, $Y = \{E, F\}$, and $Z = \{G\}$. The condensation graph has just two edges, $X \to Y$ and $X \to Z$. Therefore X must come first, while Y and Z can appear in either order: both (X, Y, Z) and (X, Z, Y) are valid topological orderings. The original graph has cycles, so we can order its components this way, but cannot topologically order its individual vertices.



(a) Original graph with components X, Y, Z.

(b) Condensation graph: X comes first.

Figure 6: Replacing each strongly connected component by one vertex produces a DAG. Both (X, Y, Z) and (X, Z, Y) are topological orderings.

For example, if Tarjan starts at A, follows the longer cycle first, and explores $D \to E$ before returning to B, it finishes Y first. It then visits G from B and finishes Z, before finally returning X. Its output (Y, Z, X) reverses to the valid topological ordering (X, Z, Y).
However, if we know that the graph is a DAG, we can use a simpler algorithm to obtain the topological order (or reversed order if we invert the direction of all edges). Algorithm 2 presents a variant of Kahn's algorithm. It is conceptually simple, however not as powerful as Tarjan's algorithm and its complexity is still $O(V + E)$.

Algorithm 2 Topological Sorting (Toposort) for DAGs

Input: A directed acyclic graph $G = (V, E)$
Output: A topological order of the vertices in G

$L \leftarrow ()$ ▷ An empty list - the result
$\forall_{v \in V}\, \mathrm{id}(v) \leftarrow \mathrm{indeg}(v)$ ▷ Map of in-degrees
$\mathrm{S} \leftarrow \{v \in V \mid \mathrm{id}(v) = 0\}$ ▷ Set of all vertices with no incoming edges
while $S \neq \emptyset$ **do**
    Pick any $v \in S$
    $S \leftarrow S \setminus \{v\}$
    $L \leftarrow L \circ (v)$
    for $(v, u) \in E$ **do** ▷ All successors of v
        $\mathrm{id}(u) \leftarrow \mathrm{id}(u) - 1$
        if $\mathrm{id}(u) = 0$ **then**
            $S \leftarrow S \cup \{u\}$
        end if
    end for
end while
if $|\{v \in V \mid \mathrm{id}(v) > 0\}| > 0$ **then**
    Error: Graph has at least one cycle
end if
return L

On the condensation graph in Figure 6, Kahn's algorithm starts with $S = \{X\}$, since only X has indegree zero. After removing X and its outgoing edges, both Y and Z have indegree zero. Choosing Y next gives (X, Y, Z); choosing Z next gives (X, Z, Y).

Pause and Think – Work It Out

What do you think: Is the topological order of any DAG unique or do there exist multiple orderings? Can you imagine a non-trivial DAG for which there is a unique topological order? Can you draw a DAG that has multiple topological orders?

3 Hamiltonian and Eulerian Graphs

3.1 Hamiltonian Graphs

Hamiltonian graphs are named after the Irish mathematician Sir William Rowan Hamilton, who, among others, invented a puzzle known as the Icosian Game in the 19th century. The game presented the dodecahedron (Figure 7) - a special graph. The challenge was to find in this graph, what was later called, a Hamiltonian cycle.

Definition 3.1. A *Hamiltonian path* (dt. Hamiltonscher Pfad) in a graph $G = (V, E)$ is a path that visits each vertex exactly once.

Definition 3.2. A *Hamiltonian cycle* (dt. Hamiltonscher Zyklus) in a graph $G = (V, E)$ is a cycle that visits each vertex exactly once.

A graph that contains a Hamiltonian cycle is called *Hamiltonian* (dt. hamiltonsch). To find such a cycle may appear an innocent task, and for some graphs it is rather simple. For instance for K_n any of the $n!$ permutations of vertices constitutes a Hamiltonian cycle. Also, for certain grids it is easy to find such cycles. However, for general graphs no known efficient algorithm exists to determine whether a graph is Hamiltonian or to find such a cycle. Deciding whether a Hamiltonian *path* exists in a general graph is NP-complete; finding one is the corresponding search problem.
Several famous puzzles and problems are based on Hamiltonian cycles, e.g., the lawn mower problem or the traveling salesman problem (TSP). The TSP is especially hard as it does not only require finding a Hamiltonian cycle, but also among all such cycles the one with the minimum cost (e.g., distance, time, or money) when summing all edge-weights visited in the cycle. Solving the TSP is one of the most intensively studied problems in optimization and there exist many heuristics and approximation algorithms, but still pose a challenge even today.

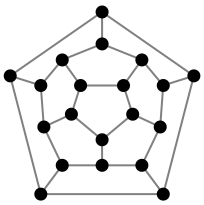


Figure 7: The dodecahedron graph, which was used in Hamilton's Icosian Game.

There are some *sufficient conditions* that guarantee the existence of a Hamiltonian cycle in a graph. One such condition is given by Dirac's theorem.

Theorem 3.3 (Dirac's Theorem)**.** *Let $G = (V, E)$ be a simple graph with $|V| = n \geq 3$.*
$\delta(G) \geq \frac{n}{2} \Rightarrow G$ is Hamiltonian.

Background

A condition is *sufficient* if it guarantees the property when it is met, but the property may still hold when the condition is not met. For instance, in the 10-cycle $\delta(C_{10}) = 2 \not\geq \frac{10}{2}$, but by definition it is Hamiltonian.
A condition is *necessary* if the property cannot hold when the condition is not met (the condition is required or necessary), but the property may not hold even when the condition is

met. For instance, $n \geq 3$ is a necessary condition in Dirac's Theorem, a graph with less than three vertices cannot be Hamiltonian. Sufficient and necessary are related to implications in logic, e.g. $P \Rightarrow Q$ is a sufficient condition for Q, while $P \Leftarrow Q$ is a necessary condition for Q. If both hold, i.e., $P \Leftrightarrow Q$, then the condition is both necessary and sufficient.

Pause and Think – Quick Check

Why does K_2 not have a cycle?

The existence of a Hamiltonian path is also related to topological sorting.

Theorem 3.4. *A directed acyclic graph (DAG) has a unique topological order if and only if it contains a Hamiltonian path.*

This means determining whether a DAG is Hamiltonian is rather easy, because it is efficient to compute the topological order as we have already seen. Based on this order, we can then check whether there exists an edge between each consecutive pair of vertices in the order. If this is the case, we have found a Hamiltonian path and we know that there does not exist another topological order. Otherwise, we know that there exist multiple topological orders and that we cannot find a Hamiltonian path.

Let $T = (v_1, v_2, \ldots, v_n)$ be the topological order of a DAG $G = (V, E)$ with n vertices. If $\forall_{i \in \{1,2,\ldots,n-1\}} : (v_i, v_{i+1}) \in E$, then T is a Hamiltonian path in G and the topological order is unique.

3.1.1 Traveling Salesperson Problem

The Traveling Salesperson Problem (TSP) is a famous optimization problem in graph theory and combinatorial optimization. More formally the problem can be defined as follows:

Definition 3.5 (Traveling Salesperson Problem)**.** Given a weighted Hamiltonian graph $G = (V, E)$ with a weight function $w : E \to \mathbb{R}^+$, the goal is to find a Hamiltonian cycle C such that the total weight of the edges in C is minimal, i.e., minimize $\sum_{e \in C} w(e)$.

As we have discussed it may be difficult to find even one Hamiltonian cycle, especially in a sparse graph, finding the one with least cost is even more challenging. For a complete graph, there are $\frac{(n-1)!}{2}$ possible Hamiltonian cycles, which a brute-force algorithm could enumerate. This is not a requirement on every exact algorithm. The TSP is NP-hard, meaning that there is no known polynomial-time algorithm to solve it for all instances. Finding fast approximate algorithms is still a hard challenge. Those algorithms, which are often combinations of various techniques, are of high practical usefulness as the TSP has many applications in logistics and manufacturing.

3.2 Euler Graphs

Leonhard Euler (1707-1783) is considered to be the inventor of graph theory and the "Seven Bridges of Königsberg" problem is often cited as the first problem in graph theory. The problem is based on the city of Königsberg (now Kaliningrad, Russia) which was divided by the Pregel river and had seven bridges connecting its different parts in 1736 AD. The problem was to find out *whether one can walk through the city crossing each bridge exactly once*?

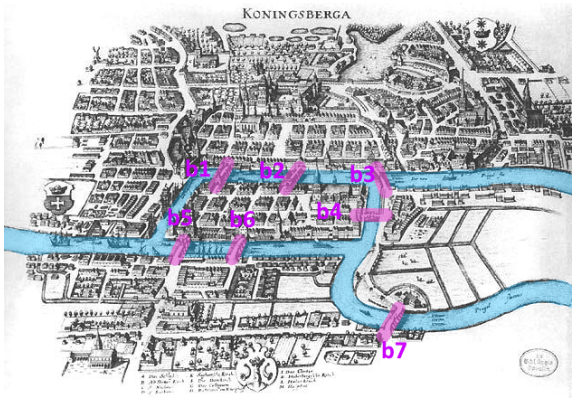
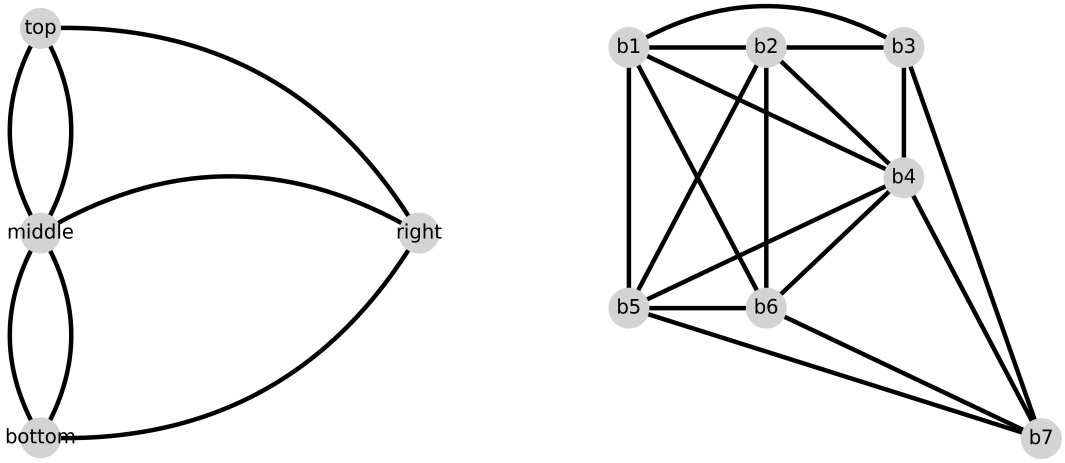


Figure 8: The Seven Bridges of Königsberg (source: Wikimedia Commons, author: User:Twotwos, 17 September 2025), altered by adding labels to the bridges.

First, Euler had to abstract the problem and create a mathematical model. That was the birth of graph theory. If we look at the picture in Figure 8, we can see that there are four landmasses (middle island, right island, top riverbank and bottom riverbank). The seven bridges (b_1 to b_7) are connecting these landmasses. Abstraction means that we ignore details that are not relevant to the problem and that we then come up with a mathematical description or model. Clearly, the houses, city walls, or streets on the landmasses are not relevant, neither is the length of the bridge, nor the shape of the river. However, we still have quite some options and I want to show both a correct and a wrong model of this problem. In practice, devising a meaningful and correct abstraction is not always straight-forward or easy.

In Figure 9a we modeled the four landmasses as vertices and the seven bridges as edges, let's call this *Model A*. In Figure 9b we chose the bridges as vertices and the existence of a non-bridge-crossing path as edges between them, let's call this *Model B*.



(a) Model A

(b) Model B

Figure 9: Different graphs for the Seven Bridges of Königsberg

Definition 3.6. An undirected multigraph with possible loops is a triple $G = (V, E, i)$, where V and E are finite sets and $i : E \to [V]^2 \cup [V]^1$ assigns each edge its endpoints. Here $[V]^1$ is the set of one-element subsets of V. Different edges can have the same endpoints; a singleton describes a loop.

Model A uses this multigraph convention: different bridges may connect the same landmasses. To work with a simple graph instead, put a new vertex in the middle of each bridge and replace that bridge by two edges. Replacing an edge $\{u, v\}$ by a path $u - x - v$ through a new vertex x is called *subdividing* the edge. Each bridge gets its own midpoint, so the resulting graph has no parallel edges. Each midpoint has degree two, and a traversal entering it must leave along its other edge. Crossing these two edges therefore corresponds to crossing the original bridge, so the algorithms below can be applied to this graph as well.

At first, both models seem to be valid. In Model A we look for a way to traverse each edge exactly once, in Model B we seemingly look for a way to visit each vertex exactly once. That's just what we learned, right? That's a Hamiltonian cycle! And even if that's a hard problem, it seems easy in this case. So, let's try this approach on Model B first. We can easily find a Hamiltonian cycle in Model B, for instance $(b_1, b_5, b_7, b_6, b_4, b_3, b_2, b_1)$. Great! So does this mean that we can conclude: Yes it's possible to create a tour that crosses each bridge exactly once? Well, let's analyze the tour, we seemingly started at the top riverbank and crossed b_1 to reach the middle island, then crossed b_5 to the bottom riverbank, then crossed b_7 to the right island and then crossed b_6 which.... erm, which connects the middle island to the bottom riverbank, but we're on the right island. Well, that's not possible. We can't cross b_6 at this point. So is the edge from b_6 to b_7 wrong? Well, no, if we crossed from the right island to the bottom riverbank, it would be ok. Does that mean we need a directed graph? Hmm, but then the edge would still point in both directions. Now you may be tempted to just find a different Hamiltonian cycle, perhaps that's just not the right one? However, you will find out that any Hamiltonian cycle in Model B will have the same problem: You can't just cross a certain bridge. The reason is that in Model B we have lost important information about the direction of travel and this "direction" is not just possible to model with a directed graph. The realization is that having bridges as vertices means that we don't need to actually cross the bridge. We modeled that we just visited the bridge, e.g., touched it, but that doesn't mean we crossed it. Thus, Model B is not the right abstraction of the original problem, or in other words: we can't answer the question based on Model B.

Pause and Think – Quick Check

If even Model B is incorrect, you should understand *why* it's incorrect given the reasoning above. We can still learn a lot from models that are wrong - when you at least recognize that they are wrong and correctly deduce why they are wrong. Otherwise, you make the same mistakes again in a different context.

Repairing Model B. We can make the Hamiltonian-cycle approach work by representing each bridge with three new vertices instead of one. For every original edge $e = \{u, v\}$, create a path $u_e - m_e - v_e$. Its endpoints represent the two ends of that bridge; endpoints belonging to different bridges remain separate vertices. Then connect every pair of bridge endpoints that lie on the same landmass. The original landmass vertices are no longer needed.

The middle vertex m_e has exactly two neighbors, so a Hamiltonian cycle must use both edges of $u_e - m_e - v_e$. This forces it to cross the entire bridge. The connections between bridge endpoints then let it continue along another bridge on the same landmass. Thus a Hamiltonian cycle in the new graph describes an Eulerian cycle in the original graph. Conversely, an Eulerian cycle gives a Hamiltonian cycle by following each bridge's three-vertex path and connecting successive bridges at their shared landmass.

This is a correct model, but it offers little help in finding a tour: for m original bridges it has $3m$ vertices, plus the edges connecting endpoints on each landmass. We would still need to solve a Hamiltonian-cycle problem. Working directly with Model A gives us a much simpler method, as we will see next.

So let's turn our attention to Model A. Perhaps we can solve the problem there. In Model A,

we are looking for a trail that visits each edge exactly once. So, is this possible? Euler proved that, based on Model A, it is not possible that such a trail exists.

Theorem 3.7. *An undirected connected graph $G = (V, E)$ contains an Eulerian trail (an open trail that visits each edge exactly once) if and only if it has exactly two vertices of odd degree. The trail must start and end at these two vertices.*

Theorem 3.8. *An undirected connected graph $G = (V, E)$ contains an Eulerian cycle (a closed trail that visits each edge exactly once) if and only if all vertices of G have even degree.*

A graph for which an Eulerian cycle exists is called *Eulerian*.

Proof. Necessity. In a closed trail using every edge, each arrival at a vertex is paired with a departure along a different edge. Hence every degree is even. In an open Eulerian trail, the start has one extra departure and the end one extra arrival; exactly those two vertices have odd degree.

Sufficiency. Suppose all degrees are even. Starting at a vertex with an edge, follow unused edges until no continuation is possible. Parity prevents getting stuck anywhere except the start, so this produces a closed trail. If edges remain unused, connectedness ensures that a vertex of the current trail is incident to an unused edge. The unused degrees are still even. Construct another closed trail there and splice it into the first one. Each step uses more edges, so the process terminates with an Eulerian closed trail. This is Hierholzer's idea.

If exactly two vertices a, b have odd degree, add an auxiliary edge between them (allowing a parallel edge for this argument). All degrees are now even. Construct an Eulerian closed trail and delete the auxiliary edge, starting the remaining sequence just after it. This gives an open Eulerian trail between a and b. □

We observe that the graph in Model A has odd degree for all four vertices. Thus, according to Theorems 3.7 and 3.8, there can exist neither an Eulerian cycle nor an Eulerian trail in this graph. We can thus say, that this problem has no solution.

The implication of these theorems is that we can easily check whether we can find an Eulerian trail or cycle in such a graph. We just need to look at all vertex degrees and count the odd ones. If there are more than two, we know that no Eulerian trail exists. If there are exactly two, we know that an Eulerian trail exists, and if there are none, we know that an Eulerian cycle exists. Compared to the Hamiltonian path or cycle problem, Eulerian cycle problems are easy. Still, the theorem only tells us about the existence of such a trail or cycle, but we still need an algorithm to actually find such a trail or cycle.

3.2.1 Fleury's Algorithm

The idea of Fleury's algorithm as shown in Algorithm 3 is to start at any vertex (respectively one of the two odd vertices) and to traverse the edges one after another. If you tried that yourself, you may have noticed that this may get you stuck at some point, because you choose an edge that provides no path back to the remaining edges. Thus, in Fleury's algorithm, we have to avoid choosing *bridges* as the edge to follow along until there's no other possibility.
Naturally, the avoidance of bridges comes at a significant cost, because it is computationally not trivial to determine whether an edge is a bridge or not. We will have to perform some kind of connectivity test for each edge we consider. Such a test can involve running a depth-first-search (DFS). Otherwise, in this algorithm we're removing one edge in each iteration. Because a DFS runs in $O(V + E)$ the time complexity of Fleury's algorithm is $O(E^2)$ (the $V \cdot E$ part is dominated by the E^2 part and thus omitted).

3.2.2 Hierholzer's Algorithm

Hierholzer's algorithm was published in 1873, after his death in 1871. It constructs and splices trails as in the sufficiency proof above, without testing bridges. With all degrees even, an initial maximal trail closes at its start. With two odd vertices, start at one and finish at the other; additional closed trails can be spliced in at vertices with unused edges.

Algorithm 4 implements this recursively. The remaining-edge set F and output list K are shared across recursive calls. Vertices are appended after their remaining edges have been explored,

Algorithm 3 Fleury's Algorithm for Finding an Eulerian Trail or Cycle

Input: An *connected* undirected graph $G = (V, E)$ with either zero or two vertices of odd degree
Output: An Eulerian trail or cycle in G

if $\exists\, v_0 \in V : \deg(v_0)$ is odd **then**
    $v \leftarrow v_0$
else
    $v \leftarrow$ any vertex in V
end if
$F \leftarrow E$ ▷ Copy of the edge set
$K \leftarrow (v)$ ▷ Current trail, starting with v
while $F \neq \emptyset$ **do**
    $e = \{v, u\} \leftarrow$ an edge incident to v that is not a bridge in (V, F), unless it is the only edge at v
    $F \leftarrow F \setminus \{e\}$
    $v \leftarrow$ the other endpoint of edge e
    $K \leftarrow K \circ (v)$
end while
return K

constructing the trail in reverse order, which is valid in an undirected graph. With adjacency lists and constant-time edge removal or marking, each edge is processed a constant number of times, giving $O(E)$ time for a connected graph with edges. This is why Hierholzer is generally preferred to Fleury.

Algorithm 4 Hierholzer's Algorithm for Finding an Eulerian Cycle

Input: An *connected* undirected graph $G = (V, E)$ with either zero or two vertices of odd degree
Output: An Eulerian trail or cycle in G

$v \leftarrow$ an odd-degree vertex if exists otherwise any vertex in V
$F \leftarrow E$ ▷ Copy of the edge set
$K \leftarrow ()$ ▷ Current trail, initially empty
VISIT(V, F, v, K)
return K
procedure VISIT(V, F, v, K)
    $N \leftarrow \{u \in V : \{v, u\} \in F\}$
    for $u \in N$ **do**
        if $\{v, u\} \in F$ **then** ▷ Edge may have been removed in recursive call
            $F \leftarrow F \setminus \{\{v, u\}\}$
            VISIT(V, F, u, K)
        end if
    end for
    $K \leftarrow K \circ (v)$
end procedure

3.2.3 Chinese Postman Problem

The Chinese Postman Problem (CPP), or Route Inspection Problem, asks for a minimum-weight *closed walk* traversing every edge at least once. It was introduced by Meigu Guan in 1960. Postal delivery, garbage collection, and street sweeping provide natural applications: roads are edges and intersections are vertices.

If every degree is even, an Eulerian closed trail uses each road once. Otherwise, some roads must be repeated. Repeating a route changes the parity of its endpoints, so the task is to choose additional traversals that make all degrees even at minimum extra cost. Roads can have different lengths, making the choice matter. Unlike the NP-hard TSP optimization problem, the connected undirected CPP with non-negative weights can be solved in polynomial time. Other route-inspection variants can be NP-hard.

Pause and Think – Challenge

How would you approach solving the CPP? Can you devise an algorithm yourself? Can you prove its optimality?

4 Transitive Closure

The transitive closure turns indirect connections into direct ones: it adds an edge from u to v whenever we can reach v from u by following one or more edges. It therefore lets us read reachability directly from a graph or its adjacency matrix.

Definition 4.1. For a directed graph $G = (V, E)$, its *transitive closure* is $G^+ = (V, E^+)$, where

$$E^+ = \{(u, v) \in V \times V \mid \text{a positive-length walk from } u \text{ to } v \text{ exists}\}.$$

Its *reflexive transitive closure* also allows the trivial walk, so it contains (v, v) for every $v \in V$.

In an undirected graph, all vertices in the same connected component can reach each other. Its transitive closure therefore connects every pair of distinct vertices in that component, making the component a clique. For the matrix calculation, we enter each undirected edge in both directions. Following an edge and coming back also gives a positive-length walk from a vertex to itself, so the corresponding diagonal entry is 1. In the drawings below, we leave out these self-loops to keep the connections between vertices easy to see.

Background

A clique is a set of pairwise adjacent vertices. Computing a maximum clique in an arbitrary graph is NP-hard; deciding whether a clique of at least a given size exists is NP-complete. In the undirected graph obtained by drawing the reachability relation without self-loops, the largest components are its maximum cliques, so this special case is easy. A clique in this graph need not have been a clique in the original graph.

There are several algorithms to compute the transitive closure of a graph. We will examine solving it via matrix multiplications as well as using the Floyd-Warshall algorithm.

4.1 Transitive Closure via Matrix Multiplication

You have already learned about matrix multiplication in the context of linear algebra. An adjacency matrix A of a graph $G = (V, E)$ is a square matrix of size $|V| \times |V|$ that represents the edges of the graph.

$$a_{ij} = \begin{cases} 1 & \text{if } v_i \sim v_j \\ 0 & \text{otherwise} \end{cases}$$

Definition 4.2. The Boolean product $C = A \odot B$ of two adjacency matrices A and B is defined as $c_{ij} = \bigvee_{k=1}^{n} (a_{ik} \wedge b_{kj})$, where $\wedge$ is the logical AND operation and $\vee$ is the logical OR operation.

The Boolean power A^i describes walks of exactly i edges, including walks that repeat vertices. For $n = |V| \geq 1$, positive-length reachability is

$$T^+ = A \vee A^2 \vee \cdots \vee A^n.$$

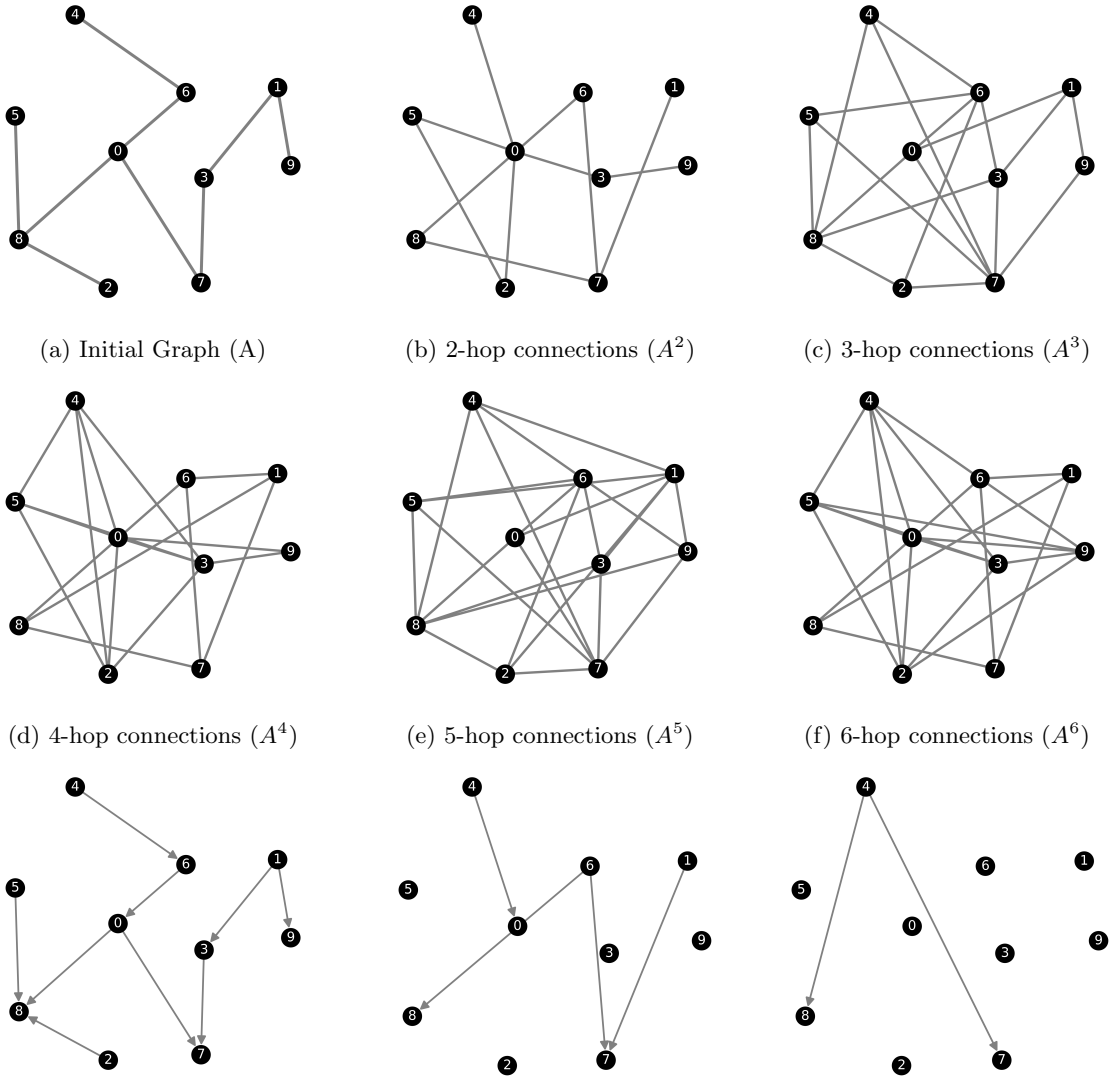
Why include the nth power? For different endpoints, removing closed portions leaves a path of at most $n - 1$ edges. For equal endpoints, a shortest nonempty return walk can be a cycle of length n. If trivial walks are allowed, the identity matrix already supplies the diagonal, giving

$$T^* = I \vee A \vee A^2 \vee \cdots \vee A^{n-1}.$$

For example, in the directed cycle $1 \to 2 \to 3 \to 1$, neither A nor A^2 has a diagonal entry, but $A^3 = I$. Thus $A \vee A^2$ misses all three positive-length returns.

Figure 10 shows an example graph and its adjacency matrices A^1 to A^6, as well as B^1 to B^3 of a directed graph. Note that while vertex v_1 and v_3 are connected in the initial undirected graph, v_1 can reach v_7 and itself in exactly two hops. The drawings suppress self-loops, so only the connection to v_7 is drawn in A^2. Since v_1 and v_3 may be connected via this walk that is three edges long: (v_1, v_3, v_1, v_3), these are connected again in A^3. In this graph, there is no even-length walk between

v_1 and v_3, thus these are not connected in even powers of the adjacency matrix. Furthermore, as in this undirected graph every vertex can reach every other, drawing its reachability relation without self-loops gives a complete graph. In the directed graph, there are no cycles and thus B^4 would already be the null matrix.



(a) Initial Graph (A) (b) 2-hop connections (A^2) (c) 3-hop connections (A^3)

(d) 4-hop connections (A^4) (e) 5-hop connections (A^5) (f) 6-hop connections (A^6)

(g) Initial Graph (B) (h) 2-hop connections (B^2) (i) 3-hop connections (B^3)

Figure 10: Off-diagonal connections in the Boolean powers A^1 to A^6 and B^1 to B^3. These are graph drawings; diagonal entries are not shown.

In practice, the computation of the transitive closure via matrix multiplications is often not efficient, especially for large graphs. More efficient algorithms, such as the Floyd-Warshall algorithm, are often preferred.

4.2 Warshall Algorithm

Algorithm 5 computes the adjacency matrix of the transitive closure with three nested loops, giving a time complexity of $O(V^3)$. Its central idea is to discover longer connections by considering one intermediate vertex at a time. In Section 5, we will use the same idea for all-pairs shortest paths in the *Floyd Algorithm*.

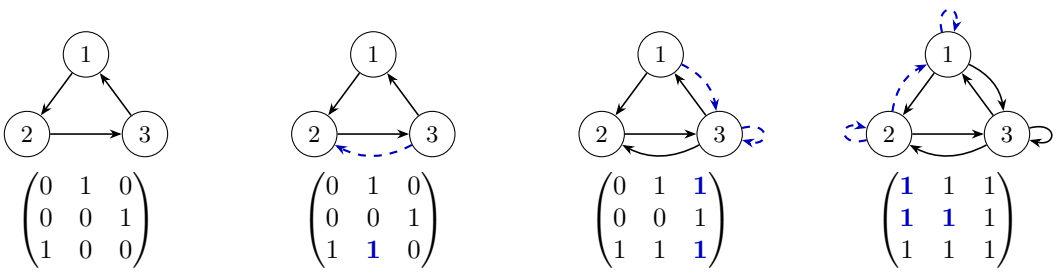
Number the vertices $v_1, \ldots, v_n$. Initially, $d^{(0)}$ is the adjacency matrix: we know only the direct edges. After stage k, $d^{(k)}[i][j] = 1$ if there is a positive-length walk from v_i to v_j whose intermediate vertices all belong to $\{v_1, \ldots, v_k\}$. The endpoints can be any vertices.

At stage k, we check whether a connection from v_i to v_j can go through v_k. If we already know how to get from v_i to v_k and from v_k to v_j, joining those two walks gives a connection from v_i to v_j. We keep all connections found earlier as well. Therefore

$$d^{(k)}[i][j] = d^{(k-1)}[i][j] \vee \big(d^{(k-1)}[i][k] \wedge d^{(k-1)}[k][j]\big).$$

The outer loop over k makes each vertex available as an intermediate vertex in turn. Algorithm 5 stores the successive stages in one matrix.

Figure 11 follows the algorithm on the cycle $1 \to 2 \to 3 \to 1$. At $k = 1$, the walk $3 \to 1 \to 2$ adds the connection $3 \to 2$. At $k = 2$, we find $1 \to 3$ and $3 \to 3$; the latter combines the known connection $3 \to 2$ with the edge $2 \to 3$. Finally, $k = 3$ adds $2 \to 1$, $1 \to 1$, and $2 \to 2$. Every vertex can now reach every vertex, including itself, so the final matrix consists entirely of ones.



(a) Initial matrix $d^{(0)}$. (b) After $k = 1$. (c) After $k = 2$. (d) After $k = 3$.

Figure 11: Warshall's algorithm on a directed three-cycle. Dashed blue arrows and bold blue matrix entries show the connections added at each stage. Matrix rows and columns are ordered $1, 2, 3$.

Algorithm 5 Algorithm for Computing the Transitive Closure

Input: A graph $G = (V, E)$
Output: The adjacency matrix of the transitive closure of G

```
d[·][·] ← 0                                   ▷ Zero-initialize the adjacency matrix
for (u, v) ∈ E do
    d[u][v] ← 1                               ▷ For undirected graphs, symmetry needs to be ensured
end for
for k ∈ V do
    for i ∈ V do
        for j ∈ V do
            d[i][j] ← d[i][j] ∨ (d[i][k] ∧ d[k][j])
        end for
    end for
end for
return d
```

5 Shortest Paths

The problem of finding shortest paths in graphs is a fundamental problem in computer science and graph theory. It has numerous applications in various fields including also computer games. In this section, we will explore different algorithms for finding shortest paths and some of the limitations. A fundamental difference between the TSP discussed in Section 3.1 and shortest path is that we don't need to visit *all* vertices in the graph. We will see that the shortest path problem can be solved efficiently such that we can apply it to very large graphs with reasonable computational resources.

To formally define the shortest path problem, we first need to define the *length* of a path in a graph. We have already introduced the length of a walk in Definition 2.3, but used only the number of edges in the walk. Typically, shortest path problems are found in *weighted graphs*, and we seek to minimize the total weight of all edges in the path, rather than the number of edges. We thus introduce a variant of our length function.

Definition 5.1. Given a (u, v)-walk between vertices u and v: $W_{u,v} = (S_V, S_E)$ in a weighted graph with weight function $\omega : E \to Z$, the length of the walk is defined as $\mathcal{L}_2(W_{u,v}) = \sum_{e \in S_E} \omega(e)$. Again, the empty and trivial walk have a length of 0.

Given a start and end vertex in a weighted graph, the *shortest path problem* consists of finding a walk between the two vertices such that the length of the walk is minimal. If there are multiple minimal walks, any of them is a valid solution.
Not all shortest path problems are easy to solve however. Negative-weight edges introduce a complexity that the most efficient algorithms cannot handle. The obstruction to any shortest path problem are negative cycles reachable from the source from which the target is also reachable. In a negative cycle each round-trip reduces the overal length of the path. Figure 12 depicts a directed graph with such a cycle. In an undirected graph, even one negative edge allows a negative closed walk by traversing it in both directions; though it is not necessarily a cycle in its earlier definition.

Background

Note, that we explicitly use the definition of *walk* for the definition of the shortest *path* problem. That's strange! Well, let me explain it this way: With positive edge weights, any part of a walk that returns to the same vertex *can be removed*, because removing it will actually decrease the walk's length. The detour is always more expensive. Consequently, any shortest walk in such a graph *is also a path*. With non-negative weights, or directed negative edges but no negative cycles, removing closed portions does not increase the cost. Thus at least one shortest walk is a path.
The distinction matters when negative cycles are allowed. If a negative cycle can be traversed on the way from the start to the destination, *no shortest walk exists*. But the suggestion, that we should then "just" forbid repeating vertices and force what we usually obtain (a path) leads us to a very different problem. Of course a shortest path (a factual path) exists whenever the destination is reachable, BUT finding it is NP-hard in general. Why? Well, think about a cyclic graph where all weights are -1. Finding a shortest $s - t$ path in this graph is actually the longest path problem: Visiting as much vertices as possible also visits as much edges as possible and each edge lowers the length of the path. Again, you can see the connection to finding a Hamiltonian path.

Shortest path algorithms rely on the following *optimality principle* (dt. Optimalitätsprinzip):

Theorem 5.2 (Optimality Principle for Shortest Paths)**.** *Let $W_{s,t}$ be a shortest walk from s to t in a weighted graph. Split this walk at any occurrence of a vertex v into a prefix $W_{s,v}$ and a suffix $W_{v,t}$. Then $W_{s,v}$ is a shortest walk from s to v, and $W_{v,t}$ is a shortest walk from v to t. In particular,*

$$\mathcal{L}_2(W_{s,t}) = \mathcal{L}_2(W_{s,v}) + \mathcal{L}_2(W_{v,t}).$$

Proof. If either part could be replaced by a shorter walk between the same endpoints, this replacement would also shorten the entire walk from s to t, contradicting its optimality. □

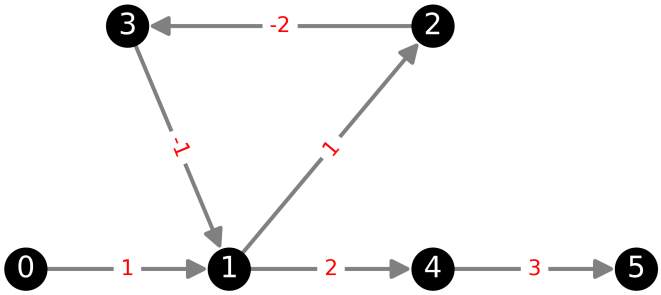


Figure 12: Graph with negative cycle

The principle applies whenever a shortest walk exists. If a negative cycle can be visited on a walk from s to t, repeatedly traversing that cycle produces walks of ever smaller total weight, so no shortest walk exists.

Background

This property is called optimal substructure: an optimal solution contains optimal solutions to its subproblems. It allows shortest-path algorithms to build solutions from optimal routes between intermediate vertices.

5.1 Shortest Path via Breadth-First Search

Breadth-first search finds paths with the fewest edges. For unweighted graphs, or graphs whose edges all have the same non-negative weight, these are also shortest paths by weight. Algorithm 6 stores the number of edges from the start vertex.

A simple breadth-first search (BFS) finds all unweighted distances in linear time with adjacency lists. The pseudocode of the algorithm is given in Algorithm 6. The time complexity of the algorithm is $O(V + E)$ as each vertex and edge is processed at most once. It maintains a list of vertices A that includes all vertices which have the same distance to the start vertex. For each vertex in the current level, all neighbors are explored. If they were not visited before $\text{dist}(u) = \infty$, their distance is updated and they are added to the list B, i.e., vertices of the next level. After all vertices in the current level have been processed, the current level A is advanced to the next level B and the process continues until no more vertices are left to explore. The algorithm returns the distance from the start vertex to all other vertices as well as a predecessor list W that can be used to reconstruct the actual shortest paths.

Algorithm 6 Shortest Path via Breadth-First Search

Input: An unweighted graph $G = (V, E)$ and a start vertex s
Output: Distances measured in numbers of edges from s, and predecessors for reconstructing paths

```
1: lvl ← 0
2: A ← (s)
3: W(s) ← (s)
4: dist(s) ← 0
5: for all v ∈ V \ {s} do
6:     dist(v) ← ∞
7:     W(v) ← ()
8: end for
9: while |A| > 0 do
10:    lvl ← lvl + 1
11:    B ← ()
12:    for all v ∈ A do
13:        for all u ∈ Suc(v) do                ▷ for all successors of v
14:            if dist(u) = ∞ then
15:                dist(u) ← lvl
16:                W(u) ← (v)
17:                B ← B ∘ (u)                  ▷ append u to B
18:            end if
19:        end for
20:    end for
21:    A ← B
22: end while
23: return dist, W
```

We apply Algorithm 6 to the directed graph in Figure 13, starting at s. For now, ignore the edge weights and count only the edges traversed. Process successors in alphabetical order whenever there is a choice.

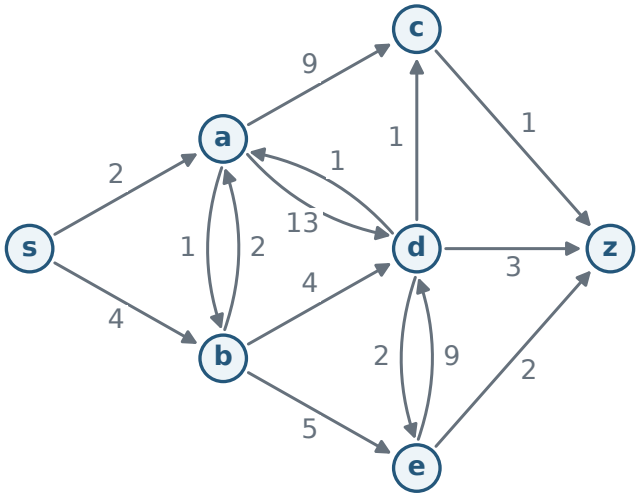


Figure 13: A directed graph for comparing shortest-path algorithms. The numbers beside the edges are their weights.

Distance from s	0	1	2	3
Vertices first reached	s	a, b	c, d, e	z

After processing s, the current list is $A = (a, b)$. Processing a discovers c and d; processing b adds e. Thus the next list is $A = (c, d, e)$. Vertex c then discovers z, giving the path (s, a, c, z) with three edges. Each vertex receives its distance when it is first discovered.
Now consider the weights. The path (s, a, c, z) costs $2 + 9 + 1 = 12$, whereas (s, a, b, d, c, z) costs $2+1+4+1+1 = 9$, despite using five edges. For unequal weights, minimizing the number of edges need not minimize their total weight. We therefore need algorithms that can improve a distance after first discovering a vertex.

Digression

A special case of a graph for which BFS is most efficient is its application to a tree. We will discuss trees in Chapter 6.1. Just because an algorithm is not suitable to solve a general problem, doesn't mean it can't be the best to solve a special problem. Even, if it is a general algorithm such as BFS.

5.2 Floyd Algorithm

The Warshall algorithm, which we have already seen in Section 4.2, can be modified to compute the all pairs shortest path problem (APSP) which finds not only a single shortest path between two vertices, but the shortest paths between *all pairs* of vertices in a weighted graph. In the modified code given in Algorithm 7, we use the operations min and + instead of logical or and logical and. The time complexity remains $O(V^3)$. The algorithm can be applied to graphs with negative weights.

The intermediate-vertex interpretation is exactly the one used for Warshall: retain the best old route, or combine a route to k with one from k. For example, if the current entries are $d[A][C] = 8$, $d[A][B] = 2$, and $d[B][C] = 3$, allowing B as an intermediate vertex changes $d[A][C]$ to $\min(8, 2 + 3) = 5$. The endpoints A, C need not have been processed as intermediate vertices yet.

Algorithm 7 Algorithm for Computing All Pairs Shortest Paths

Input: A weighted graph $G = (V, E, \omega)$ that contains no negative cycles
Output: A matrix of shortest path distances between all pairs of vertices in G

```
1:  d[·][·] ← ∞                                          ▷ initialize the distance matrix
2:  for (u, v) ∈ E do
3:      d[u][v] ← ω((u, v))          ▷ For undirected graphs, symmetry needs to be ensured
4:  end for
5:  for v ∈ V do
6:      d[v][v] ← 0                                          ▷ Distance to self is zero
7:  end for
8:  for k ∈ V do
9:      for i ∈ V do
10:         for j ∈ V do
11:             d[i][j] ← min (d[i][j], d[i][k] + d[k][j])
12:         end for
13:     end for
14: end for
15: return d
```

Furthermore, the algorithm returns only the shortest distances between pairs of vertices, it does not compute the actual paths. In order to obtain the paths we need to keep track of the predecessors. We can initialize a predecessor matrix $pred[\cdot][\cdot]$ such that for every edge $(u, v) \in E$ we set $pred[u][v] = u$. Whenever we update a distance $d[i][j]$ via an intermediate vertex k, we also update the predecessor $pred[i][j] = pred[k][j]$. After the algorithm has completed, we can reconstruct the shortest path from u to v by backtracking through the predecessor matrix starting from v until we reach u.

5.3 Bellman-Moore-Ford Algorithm

Bellman–Moore–Ford computes shortest paths from one start vertex s to all other vertices. As in BFS, we initially set $\text{dist}(s) = 0$ and all other distances to ∞. With unequal edge weights, however, a newly discovered route may improve a distance that is already finite. If

$$\text{dist}(u) + \omega((u, v)) < \text{dist}(v),$$

we replace $\text{dist}(v)$ by this smaller value and remember u as the predecessor of v. This operation is called *relaxation* (dt. Entspannung). The known walk to u need not already be shortest; any improvement can help us discover a better route to v.

Background

We stated above, that Bellman–Moore–Ford findes shortest paths from s to *all other* vertices. In fact, we cannot stop the algorithm early. We do not know if we have found the shortest path to a certain node t, unless we have confirmed that no further relaxation is possible and no further node remains to be visited (either by discovering it first time or relaxing a neighbor). At this point, the algorithm has found shortest paths to all other vertices.

When the distance to a vertex improves, we must examine its outgoing edges to see whether the improvement also gives shorter routes to its successors. We keep the vertices whose outgoing edges still need to be examined in a queue Q. The queue follows the first-in, first-out (FIFO) rule: remove the vertex that has been waiting longest and append newly improved vertices at the end. A vertex already waiting does not need a second entry. The set *onQueue* records which vertices are waiting. After a vertex has been removed and processed, a later improvement can add it to the queue again.

Implementation Note

In practice you don't always need to use an unordered (hash-based) or ordered set. If each of the nodes has a 0-based index, you can use bitsets which are much more space efficient.

Algorithm 8 groups this work into iterations of the outer loop. At the beginning of each iteration, it stores the number q of vertices currently in the queue and then processes exactly q entries. Newly appended entries wait until the next iteration of the outer loop.

Algorithm 8 Bellman–Moore–Ford with a FIFO Queue

Input: A weighted graph $G = (V, E, \omega)$ and a start vertex s
Output: Distances and predecessors, or a message that a negative cycle is reachable from s

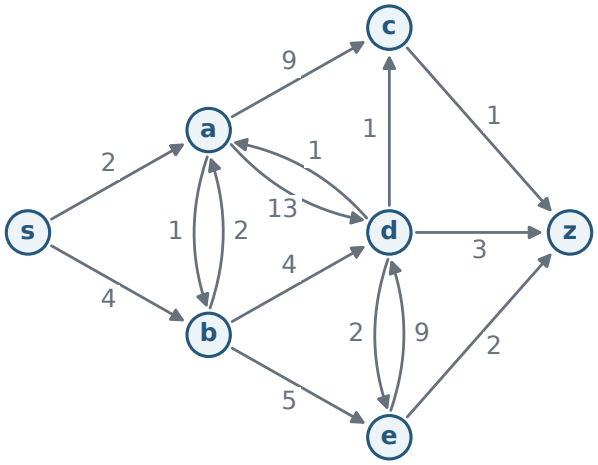
```
 1: dist(s) ← 0; W(s) ← (s)
 2: for all v ∈ V \ {s} do
 3:     dist(v) ← ∞; W(v) ← ()
 4: end for
 5: Q ← (s); onQueue ← {s}
 6: for k = 1 to |V| do
 7:     q ← |Q|                          ▷ Number of vertices currently in the queue
 8:     for j = 1 to q do
 9:         u ← dequeue(Q); remove u from onQueue
10:         for all v ∈ Suc(u) do
11:             if dist(u) + ω((u, v)) < dist(v) then
12:                 if k = |V| then
13:                     return "negative cycle reachable from s"
14:                 end if
15:                 dist(v) ← dist(u) + ω((u, v)); W(v) ← (u)
16:                 if v ∉ onQueue then
17:                     enqueue(Q, v); add v to onQueue
18:                 end if
19:             end if
20:         end for
21:     end for
22:     if Q is empty then
23:         return dist, W
24:     end if
25: end for
26: return dist, W
```

How many iterations of the outer loop are needed? By the end of iteration k, every vertex v satisfies $\text{dist}(v) \leq \mathcal{L}_2(W)$ for every walk W from s to v with at most k edges. To see this, split such a walk at its last edge (u, v). By the preceding iteration, the distance to u is at most the cost of the prefix. Once this distance is available, u either has already propagated it or is waiting to do so in the current iteration. Relaxing (u, v) therefore gives a distance to v no larger than the cost of W.

Without a reachable negative cycle, a shortest path uses at most $|V| - 1$ edges. Thus all reachable distances are correct after at most $|V| - 1$ iterations of the outer loop. An improvement in iteration $|V|$ implies that a negative cycle is reachable from s. Each vertex is processed at most once per iteration of the outer loop. With adjacency lists and constant-time tests for queue membership, the total running time is $O(VE + V)$, including initialization. Negative edge weights are allowed; they do not invalidate the relaxation argument.

Algorithm 8 detects only negative cycles reachable from the start vertex s. A cycle that cannot be reached from s cannot affect walks starting there. To detect negative cycles anywhere in the graph, initialize every distance to zero and put every vertex into Q, with $onQueue = V$. This is equivalent to first processing an artificial source with zero-cost edges to every vertex. We will use such a search when improving minimum-cost flows in Section 10.

We return to the graph in Figure 13, this time using its edge weights. Successors are again processed in alphabetical order. The table shows the queue and distances after examining all outgoing edges of the indicated vertex u. The next vertex to be removed is written at the left of Q, and $\text{dist}(s) = 0$ throughout.



Outer iteration k	u	Q afterward	a	b	c	d	e	z
–	–	(s)	∞	∞	∞	∞	∞	∞
1	s	(a, b)	2	4	∞	∞	∞	∞
2	a	(b, c, d)	2	3	11	15	∞	∞
2	b	(c, d, e)	2	3	11	7	8	∞
3	c	(d, e, z)	2	3	11	7	8	12
3	d	(e, z, c)	2	3	8	7	8	10
3	e	(z, c)	2	3	8	7	8	10
4	z	(c)	2	3	8	7	8	10
4	c	(z)	2	3	8	7	8	9
5	z	$()$	2	3	8	7	8	9

When a improves the distance to b from 4 to 3, b is still waiting, so its queue entry is sufficient. Later, c is processed with distance 11 and supplies a route to z of cost 12. Processing d then improves the distance to c to $7 + 1 = 8$. Since c has already been removed, it must be appended again. Its second processing improves the distance to z from 10 to 9, which also requires processing z again. Altogether, there are nine removals from the queue: $s, a, b, c, d, e, z, c, z$. Repeated processing

is necessary to propagate these improvements, even though all weights in this example are positive.

5.4 Dijkstra's Algorithm

Dijkstra's algorithm, one of the most popular algorithms, finds shortest paths in graphs with non-negative weights. In the Bellman–Moore–Ford example, c and z had to be processed again because a shorter route was found later. Dijkstra chooses the next vertex by its smallest currently known distance from s. With non-negative edge weights, this choice guarantees that the distance is already shortest. We call such a vertex *finalized* and add it to the set F. Its outgoing edges then need to be examined only once. We prove this guarantee below.

The time complexity of Dijkstra's algorithm depends strongly on two operations:

1. $|V|$ times we have to obtain the vertex v^* with the smallest distance in line 9
2. $|E|$ times we have to perform an update to the distances (decrease-key operation) in line 13

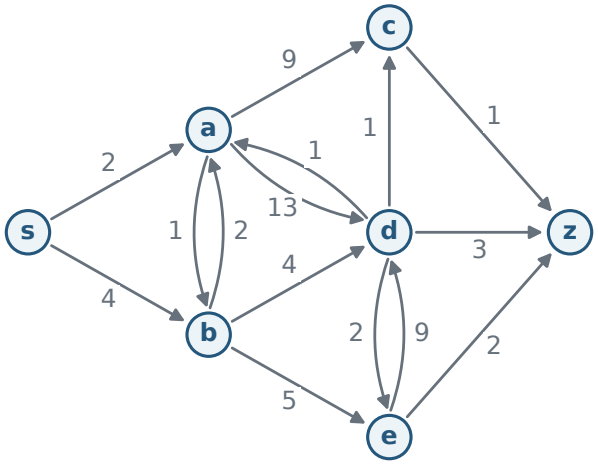
Using a simple list, argmin is $O(V)$, while decrease-key is $O(1)$, so the overall complexity would be $O(V^2)$. With a binary heap, both argmin and decrease-key are $O(\log V)$, so the overall complexity would be $O((V + E) \log V)$. However, with a Fibonacci heap, it can be improved to $O(E + V \log V)$ since decrease-key is amortized $O(1)$ for Fibonacci heaps. The pseudocode of Dijkstra's algorithm is given in Algorithm 9. This means that solving the APSP problem with Dijkstra's algorithm is more efficient than using the Floyd-Warshall algorithm for sparse graphs. Much of the actual efficiency of algorithms depend on the implementation and computer architecture though and actual size of the graph. But more importantly, it is trivial to parallelize solving the APSP with Dijkstra's algorithm.

Algorithm 9 Dijkstra's Algorithm

Input: A weighted graph $G = (V, E, \omega)$ with non-negative weights and a start vertex s
Output: Shortest distances from s to other vertices, and a predecessor list to reconstruct the shortest paths

```
 1: F ← {}
 2: dist(s) ← 0
 3: W(s) ← (s)
 4: for all v ∈ V \ {s} do
 5:     dist(v) ← ∞
 6:     W(v) ← ()
 7: end for
 8: while V ≠ F do                          ▷ Alternatively stop when target vertex z ∈ F
 9:     v* ← argmin_{v∈V\F} dist(v)         ▷ get vertex with smallest distance
10:     F ← F ∪ {v*}
11:     for all v ∈ Suc(v*) \ F do          ▷ for all successors of v* not in F
12:         if dist(v*) + ω((v*, v)) < dist(v) then
13:             dist(v) ← dist(v*) + ω((v*, v))   ▷ Decrease-key operation
14:             W(v) ← (v*)
15:         end if
16:     end for
17: end while
18: return dist, W
```

We apply Algorithm 9 to the same graph in Figure 13, starting at s. Each finite value $\text{dist}(v)$ is the length of a walk we have found; ∞ means that no walk to v is known yet. The table shows the distances after finalizing each vertex and examining its outgoing edges. When distances are equal, we choose alphabetically.



After finalizing	s	a	b	c	d	e	z
Initially	0	∞	∞	∞	∞	∞	∞
s	0	2	4	∞	∞	∞	∞
a	0	2	3	11	15	∞	∞
b	0	2	3	11	7	8	∞
d	0	2	3	8	7	8	10
c	0	2	3	8	7	8	9
e	0	2	3	8	7	8	9
z	0	2	3	8	7	8	9

When a is finalized, relaxation changes b's predecessor from s to a, since $2 + 1 < 4$. After b, Dijkstra selects d with distance 7 before c with distance 11. Processing d improves the distance to c to 8, before c is finalized. Thus c's outgoing edges are examined only with its final distance. The shortest route to z is (s, a, b, d, c, z) with cost 9. All seven vertices are processed once, compared with the nine queue removals in the BMF example. Discovering z at cost 10 is not a reason to stop; finalizing it at cost 9 is.

If the weights represent travel times, the route takes nine time units. Its consecutive vertices follow edges of the graph, so it is feasible. To establish that no faster route exists, we need the correctness argument for Dijkstra's algorithm and its assumption of non-negative weights.

The algorithm shown in Algorithm 9 can be terminated as soon as a certain target vertex z has been added to the set of finalized vertices F, since at that point the shortest path from s to z has been found.

Digression

The advantage of terminating early is reflected less in theoretical results, but can have a big practical impact. For instance, when there's a huge graph, but shortest paths are computed only among close nodes it's much more important to terminate early. Theoretical runtime bounds assume significantly large arbitrary graphs with arbitrary endpoints. My advice is to always measure, also a Fibonacci heap is theoretically quite efficient, but in practice for certain applications and graphs it may be slower than a binary heap. It all depends.

Pause and Think – Quick Check

The actual target vertex isn't used anywhere in Dijkstra's algorithm, what does that mean for the search from s to a target vertex z?

Dijkstra's algorithm is the basis for A* search as we will see next, as well as Prim's algorithm for minimum spanning trees which we will explore later. Thus, if we know Dijkstra's algorithm by heart, we can also solve spanning tree problems with a small modification.

5.4.1 Proof of Correctness

Pause and Think – Challenge

How would you prove the correctness of Dijkstra's algorithm? What proof techniques have you learned?

Proof. Let $\text{dist}(v)$ be the distance from the start vertex s to vertex v as computed by Dijkstra's algorithm and let $\delta(s, v)$ be the actual shortest path distance from s to v. We want to show that for all vertices $v \in V$, $\text{dist}(v) = \delta(s, v)$ when v is added to the set F of finalized vertices.

We will prove the correctness of Dijkstra's algorithm using induction. We assume a graph $G = (V, E, \omega)$ with non-negative weights and a start vertex s.

Base Case: The first finalized vertex is s, so after that first selection $F = \{s\}$ and $\text{dist}(s) = 0$. This is correct since the length of the trivial walk has a length of 0, thus $\text{dist}(s) = \delta(s, s) = 0$.

Every finite value $\text{dist}(v)$ is the cost of an actual walk from s, so it cannot be smaller than the shortest distance. If the smallest distance among vertices outside F is ∞, no remaining vertex is reachable: a first edge leaving the reachable finalized vertices would already have supplied a finite distance.

Inductive Step: When the smallest distance among vertices outside F is finite, by our induction hypothesis (IH) we can assume that all vertices in F have correct shortest distances from s: $\forall_{v \in F}\ \text{dist}(v) = \delta(s, v)$.
We need to show that when we add v^* to F, then $\text{dist}(v^*) = \delta(s, v^*)$.
We aim to show that there does not exist a shorter path P_{s,v^*}. By contradiction, let's assume, the path P is indeed shorter and thus it must hold:

$$\mathcal{L}_2(P_{s,v^*}) < \text{dist}(v^*)$$

The path P must start at $s \in F$ and must end at $v^* \notin F$. Let u be the first vertex on P not in F, and let x be the vertex immediately before u on P. Then $x \in F$, $u \notin F, (x, u) \in E, P_{s,v^*} = (s, \ldots, x, u, \ldots, v^*)$.
Let $P_{s,u}$ be the prefix of P ending at u and $P_{s,x}$ be the prefix ending at x. By IH we know that due to non-negative weights:

$$\text{dist}(x) = \delta(s, x) \leq \mathcal{L}_2(P_{s,x}), \text{ hence } \text{dist}(x) + \omega((x, u)) \leq \mathcal{L}_2(P_{s,u}) \leq \mathcal{L}_2(P_{s,v^*})$$

When x was added to F, the distance to u must have been updated by the algorithm, thus:

$$\text{dist}(u) \leq \text{dist}(x) + \omega((x, u))$$

We know that v^* has the smallest distance among all vertices not in F.
Since $u \notin F$ and since we chose v^*, we know that

$$\text{dist}(v^*) \leq \text{dist}(u)$$

Combining the inequalities we observe the following contradiction:

$$\text{dist}(v^*) \leq \text{dist}(u) \leq \text{dist}(x) + \omega((x, u)) \leq \mathcal{L}_2(P_{s,v^*}) < \text{dist}(v^*)$$

Thus, our assumption that there exists a shorter path to v^* must be false, and we conclude that $\text{dist}(v^*)$ is indeed the shortest distance from s to v^*. □

The downside of Dijkstra's algorithm is that it expands vertices in all directions equally, instead of focusing on the direction towards the target vertex. This means that still quite a large part of the graph needs to be explored if the start and end vertices are far apart. This is shown in Figure 14. In the next section we will explore the A* algorithm which overcomes this limitation by using a heuristic to guide the search towards the target vertex.

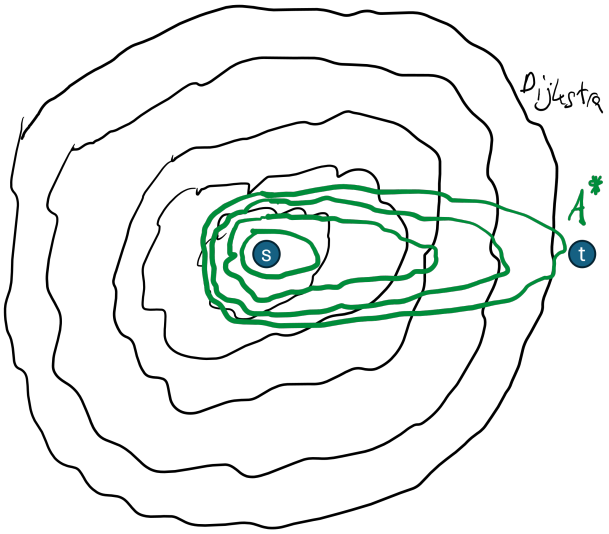


Figure 14: Schematic drawing of vertex expansion in Dijkstra's vs A* algorithm

5.5 A* Algorithm

The idea of the A* algorithm is to prioritize the exploration of vertices that are more likely to lead to the target vertex. But this means that we need to somehow integrate into the algorithm a quantification of the "direction" towards the target vertex. This is done via a *heuristic function* $h : V \to Z_0^+$ (Z is the numeric set of edge weights, but is restricted to the non-negative domain).

While we can't use any values for h, because these could mislead the search, we also don't need to have the actual shortest distances. The heuristic function estimates the cost from a vertex v to the target vertex z. We can still guarantee to make an optimal greedy decision as with Dijkstra, if we can prove that h is *monotonic*, also called *consistent* (dt. monoton bzw. konsistent).

Definition 5.3. A heuristic function $h : V \to Z_0^+$ is *monotonic* or *consistent* if for every edge $(u, v) \in E$ it holds that $h(u) \leq \omega((u, v)) + h(v)$ and $h(z) = 0$.

You may have already heard in other lectures or in other sources that the heuristic h must not *overestimate* the actual shortest path distance from v to z. This property is called *admissibility*. It can be shown that monotonicity implies admissibility, but the reverse is not necessarily true. Monotonicity is a stronger condition than admissibility and is required for the algorithm that uses a "closed set" F like Dijkstra's algorithm to be optimal.

The pseudocode of the A* algorithm using a closed set which requires h to be a consistent heuristic is given in Algorithm 10.

Pause and Think – Quick Check

Can you spot the difference between Dijkstra's algorithm and A*?

In Figure 15 we see a pathological case where A* with closed set would fail to find the shortest path from s to z. We can verify that h is admissible, however we note that h is not consistent:

$$h(c) \not\leq h(b) + \omega((b, c))$$
$$6 \not\leq 2 \quad + 1$$

Algorithm 10 A* Algorithm

Input: A weighted graph $G = (V, E, \omega)$ with non-negative weights, a start vertex s, a target vertex z, a consistent heuristic function h

Output: Shortest distances from s to z, and a predecessor list to reconstruct the shortest path

$F \leftarrow \{\}$ ▷ Often called "closed set"
$\text{dist}(s) \leftarrow 0$
$W(s) \leftarrow (s)$
for all $v \in V \setminus \{s\}$ **do**
    $\text{dist}(v) \leftarrow \infty$
    $W(v) \leftarrow ()$
end for
while $z \notin F$ **do**
    $v^* \leftarrow \text{argmin}_{v \in V \setminus F} \text{dist}(v) + h(v)$
    $F \leftarrow F \cup \{v^*\}$
    for all $v \in Suc(v^*) \setminus F$ **do** ▷ for all successors of v^* not in F
        if $\text{dist}(v^*) + \omega((v^*, v)) < \text{dist}(v)$ **then**
            $\text{dist}(v) \leftarrow \text{dist}(v^*) + \omega((v^*, v))$ ▷ Note that h is not used here
            $W(v) \leftarrow (v^*)$
        end if
    end for
end while
return dist, W

Pause and Think – Work It Out

Why does A* fail in this example?

What happens, after the search has closed a: $\text{dist}(b) = 5$ and $h(b) = 2$, thus the total estimated distance for b is 7. On the other hand, $\text{dist}(c) = 3$ and $h(c) = 6$, thus the total estimated distance for c is 9. Because, b is smaller it will be chosen next and added to the closed list F. When c will be chosen in the next iteration, b is already in F and thus it will not be recognized that there is a shorter distance to it - the path (s, a, c, b) has a distance of 4. In this example, A* as given in Algorithm 10 will find the incorrect path (s, a, b, z).

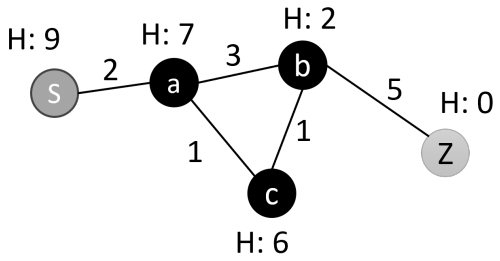


Figure 15: Pathological case where Algorithm 10 fails as the heuristic is not consistent.

But we can use a variant of A* when we have an admissible, but not consistent heuristic. In this variant, we do not use a closed set F, as we cannot guarantee that we'll find a shorter path later. Instead, the idea is to propagate shorter distances similar to the Bellman-Ford algorithm - however using a priority queue instead of a FIFO queue. Thus, we may re-expand vertices multiple times, but it will still find the optimal path as long as the heuristic is admissible. The pseudocode of this version of A* is given in Algorithm 11. It is however not as efficient as the version with a closed set, as vertices may be expanded multiple times.

In order to reduce the number of re-expansions in this second variant of A*, one can use a technique called *pathmax* which updates the heuristic values during the search to ensure that they

Algorithm 11 A* Algorithm without Closed Set

Input: A weighted graph $G = (V, E, \omega)$ with non-negative weights, a start vertex s, a target vertex z, an admissible heuristic function h

Output: Shortest distances from s to z, and a predecessor list to reconstruct the shortest path

```
1: dist(s) ← 0
2: W(s) ← (s)
3: for all v ∈ V \ {s} do
4:     dist(v) ← ∞
5:     W(v) ← ()
6: end for
7: Q ← priority queue initialized with (s, dist(s) + h(s))
8: while Q is not empty do
9:     v* ← extract-min(Q)
10:    if v* = z then
11:        break                                  ▷ stop search when target is reached
12:    end if
13:    for all v ∈ Suc(v*) do                     ▷ for all successors of v*
14:        if dist(v*) + ω((v*, v)) < dist(v) then
15:            dist(v) ← dist(v*) + ω((v*, v))
16:            W(v) ← (v*)
17:            insert/update (v, dist(v) + h(v)) in Q
18:        end if
19:    end for
20: end while
21: return dist, W
```

are non-decreasing along any path. This can help to reduce the number of times a vertex is re-expanded, improving the efficiency of the algorithm. The pathmax update is performed as follows: whenever a vertex v is expanded, for each neighbor u of v, we update the heuristic value of u as follows:

$$h(u) \leftarrow \max(h(u), h(v) - \omega((v, u)))$$

In our example, this would have partially helped: when a was closed, then $h(b) \leftarrow \max(2, 7-3) = 4$. This means the estimated distance of b the same as that of c, so it depends on the order in the priority queue whether b (wrong choice) or c (correct choice) is expanded next.

With a consistent heuristic, constant-time heuristic evaluations, and the same data structures, A* without re-expansion has Dijkstra's worst-case time bound. This does not cover reopening vertices with an inconsistent heuristic. In practice, a useful heuristic can substantially reduce exploration; when none is available, Dijkstra remains relevant.

5.6 Bidirectional Search

Without a suitable heuristic, we can still search from both endpoints simultaneously. Figure 16 sketches the possible saving when nearby vertices occupy roughly circular regions: two disks of half the radius have half the area of one full disk. Two small pizzas can be less than one big pizza! This geometric intuition can explain fewer expansions on spatial graphs; it is not a guarantee for arbitrary graph structures.

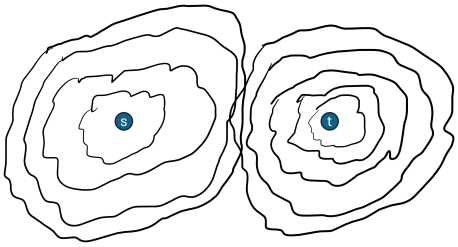


Figure 16: Schematic drawing of vertex expansions in bidirectional search

Pause and Think – Work It Out

You have learned about Dijkstra's algorithm, think about this question for yourself for a moment: When should the bidirectional search stop, what's the right condition for "met in the middle"?

It turns out that the optimal stopping condition for bidirectional search is a bit tricky. The naive approach is to stop the search as soon as we have the same vertex in the set F of finalized vertices from both directions. But, while that is a possible path from s to t, it is not necessarily the shortest one. In Figure 17 we see an example where the naive stopping condition fails to find the shortest path. In both directions, the shortest path to vertex "c" would be found before (s, a, b) or (t, b, a) is found. However, the path via (a, b) with a cost of 20 is significantly shorter than the one via c with a cost of 26.

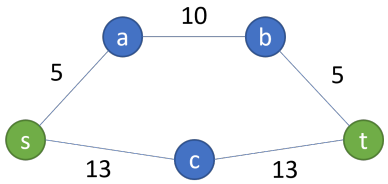


Figure 17: Pathological example where the naive stopping condition fails to find the shortest path

A complete route supplies an upper bound S. Let a, b be the smallest currently known distances among the unfinalized vertices in the forward and backward searches, respectively. Once $a+b \geq S$, Algorithm 12 can stop, provided its upper-bound updates have recorded the connections between the searches. Finding a route can precede proving it optimal. Each search excludes only its own finalized set. Backward search follows predecessors and measures distances to t.

Here join drops the duplicate meeting vertex, erases any other closed portions, and saves a copy of the resulting route; S is its cost. Non-negative weights make cycle erasure safe. We use $\infty + x = \infty$ and $\infty \geq \infty$, so unreachable targets are handled too.

Algorithm 12 Bidirectional Dijkstra

Input: A weighted graph $G = (V, E, \omega)$ with non-negative weights, a start vertex s, and a target vertex t

Output: Shortest distance from s to t, and the shortest path

if $s = t$ **then**
  return $0, (s)$
end if
$F_{s\to t} \leftarrow \{\}$
$F_{t\to s} \leftarrow \{\}$
for all $v \in V$ **do**
  $\text{dist}_{s\to t}(v) \leftarrow \infty$
  $W_{s\to t}(v) \leftarrow ()$
  $\text{dist}_{t\to s}(v) \leftarrow \infty$
  $W_{t\to s}(v) \leftarrow ()$
end for
$\text{dist}_{s\to t}(s) \leftarrow 0$
$W_{s\to t}(s) \leftarrow (s)$
$\text{dist}_{t\to s}(t) \leftarrow 0$
$W_{t\to s}(t) \leftarrow (t)$
$S \leftarrow \infty$ ▷ Initialization of upper bound
$P \leftarrow ()$
while $F_{s\to t} \neq V \wedge F_{t\to s} \neq V$ **do**
  $v^*_{s\to t} \leftarrow \text{argmin}_{v\in V\setminus F_{s\to t}} \text{dist}_{s\to t}(v)$
  $u^*_{t\to s} \leftarrow \text{argmin}_{u\in V\setminus F_{t\to s}} \text{dist}_{t\to s}(u)$
  if $\text{dist}_{s\to t}(v^*_{s\to t}) + \text{dist}_{t\to s}(u^*_{t\to s}) \geq S$ **then** ▷ The termination condition when the shortest path is found
    break
  end if
  $F_{s\to t} \leftarrow F_{s\to t} \cup \{v^*_{s\to t}\}$
  $F_{t\to s} \leftarrow F_{t\to s} \cup \{u^*_{t\to s}\}$
  for all $v \in Suc(v^*_{s\to t}) \setminus F_{s\to t}$ **do**
    if $\text{dist}_{s\to t}(v^*_{s\to t}) + \omega(v^*_{s\to t}, v) < \text{dist}_{s\to t}(v)$ **then**
      $\text{dist}_{s\to t}(v) \leftarrow \text{dist}_{s\to t}(v^*_{s\to t}) + \omega(v^*_{s\to t}, v)$
      $W_{s\to t}(v) \leftarrow W_{s\to t}(v^*_{s\to t}) \circ (v)$
      if $v \in F_{t\to s} \wedge \text{dist}_{s\to t}(v) + \text{dist}_{t\to s}(v) < S$ **then**
        $P \leftarrow \text{join}(W_{s\to t}(v), W_{t\to s}(v))$
        $S \leftarrow \mathcal{L}_2(P)$
      end if
    end if
  end for
  for all $u \in Pre(u^*_{t\to s}) \setminus F_{t\to s}$ **do** ▷ Predecessors of $u^*_{t\to s}$
    if $\text{dist}_{t\to s}(u^*_{t\to s}) + \omega(u, u^*_{t\to s}) < \text{dist}_{t\to s}(u)$ **then** ▷ Length $u \to u^*_{t\to s}$!
      $\text{dist}_{t\to s}(u) \leftarrow \text{dist}_{t\to s}(u^*_{t\to s}) + \omega(u, u^*_{t\to s})$
      $W_{t\to s}(u) \leftarrow (u) \circ W_{t\to s}(u^*_{t\to s})$ ▷ prepend u
      if $u \in F_{s\to t} \wedge \text{dist}_{t\to s}(u) + \text{dist}_{s\to t}(u) < S$ **then**
        $P \leftarrow \text{join}(W_{s\to t}(u), W_{t\to s}(u))$
        $S \leftarrow \mathcal{L}_2(P)$
      end if
    end if
  end for
end while
if $S = \infty$ **then**
  return $\infty, ()$ ▷ No path exists
end if
return S, P

5.7 Iterative Deepening A*

Depth-first search (DFS) is a common graph algorithm. It requires less memory than breadth-first search (BFS) or best-first search (like Dijkstra or A*), but in the case of finding shortest paths has to search through an exponential number of paths. Thus, DFS is a rather ill-suited algorithm for shortest path problems. We also know of a DFS-variant called Iterative Deepening Depth-First Search (IDDFS) which performs a series of *depth-limited* DFS searches with increasing depth limits until the target vertex is found. Algorithm 13 shows the pseudocode of IDDFS, which looks a bit more complex than standard DFS due to maintaining the current path.

Algorithm 13 Iterative Deepening Depth-First Search (IDDFS)

Input: A graph $G = (V, E)$, a start vertex s, a target vertex z
Output: A path from s to z, if it exists

```
1: for dlim = 0 to ∞ do
2:     found, remaining, path ← DepthLimitedDFS(G, s, z, dlim)
3:     if found then
4:         return path
5:     end if
6:     if not remaining then
7:         return null                                   ▷ No path exists
8:     end if
9: end for
10: procedure DepthLimitedDFS(G, s, z, dlim)
11:     remaining ← False
12:     depth ← 0
13:     stack ← ((s, depth))
14:     path ← ()
15:     while |stack| > 0 do
16:         (v, d) ← pop(stack)
17:         while |path| > d do          ▷ Remove previous vertex, including siblings
18:             remove last element from path
19:             depth ← depth − 1
20:         end while
21:         path ← path ∘ (v)
22:         depth ← d
23:         if v = z then
24:             return True, *, path
25:         end if
26:         for all u ∈ Suc(v) \ {x | x occurs in path} do
27:             if depth < dlim then
28:                 stack ← stack ∘ ((u, depth + 1))
29:             else
30:                 remaining ← True
31:             end if
32:         end for
33:     end while
34:     return False, remaining, ()          ▷ target not found within depth limit
35: end procedure
```

The combination of DFS with the optimality of A* is called Iterative Deepening A* (IDA*) algorithm. This algorithm is similar to IDDFS, but instead of using a depth limit, it uses a distance limit based on the estimated total distance $f(v) = \text{dist}(v) + h(v)$. The pseudocode of IDA* is given in Algorithm 14. The main advantage of IDA* is that it uses less memory than A*. Again, a consistent heuristic makes it more efficient.

Algorithm 14 Iterative Deepening A* (IDA*)

Input: A weighted graph $G = (V, E, \omega)$ with non-negative weights, a start vertex s, a target vertex z, an admissible or consistent heuristic function h
Output: Shortest distance from s to z, and the shortest path

```
1: limit ← h(s)
2: while True do
3:     found, l, path ← DepthLimitedAStar(G, s, z, limit, h)
4:     if found then
5:         return path
6:     end if
7:     if l = ∞ then
8:         return null                                   ▷ No path exists
9:     end if
10:    limit ← l
11: end while
12: procedure DepthLimitedAStar(G, s, z, limit, h)
13:    min_exceed ← ∞
14:    stack ← ((s, 0, (s)))                            ▷ (vertex, g-value, path)
15:    while |stack| > 0 do
16:        (v, g, path) ← pop(stack)
17:        f ← g + h(v)
18:        if f > limit then
19:            min_exceed ← min(min_exceed, f)
20:            continue
21:        end if
22:        if v = z then
23:            return True, *, path                      ▷ found target
24:        end if
25:        for all u ∈ Suc(v) do
26:            if u ∉ path then                          ▷ Avoid cycles
27:                stack ← stack ∘ ((u, g + ω((v, u)), path ∘ (u)))
28:            end if
29:        end for
30:    end while
31:    return False, min_exceed, ()                      ▷ target not found within limit
32: end procedure
```

6 Minimum Spanning Trees

We have already heard about what a spanning subgraph is in Section 1.5. A spanning tree is a special subgraph which adheres to additional constraints. Spanning trees have many applications in computer science. The spanning tree protocol (STP) is used in Ethernet networks to prevent broadcast loops and have packets circulate indefinitely. In distributed systems, spanning trees are used for efficient broadcasting and aggregation of information. They also play a crucial role in network design, where the goal is to connect a set of vertices with the minimum total edge weight, known as the minimum spanning tree problem - which we will cover in this section.

6.1 Trees and Arborescences

Definition 6.1. A tree (dt. Baum) is a Graph that is connected, acyclic, and undirected.

Because of these requirements, it has several interesting properties:

- There is exactly one unique path between any two vertices in the tree.
- A tree with n vertices has exactly $n-1$ edges: $|E| = |V| - 1$.
- Adding any edge to a tree creates a cycle.
- Removing any edge from a tree disconnects the graph.

What if the graph is not connected? There is the notion of a forest and let's define it.

Definition 6.2. A forest (dt. Wald) is an undirected acyclic graph where each connected component is a tree.

Pause and Think – Quick Check

Given the definitions, answer for yourself: Is a tree a forest?

So we're now able to define what a spanning tree is and introduce the minimum spanning tree problem.

Definition 6.3. Let $G = (V, E)$ be a connected undirected graph. A **spanning tree** (dt. Spannbaum) of G is a subgraph $T = (V, E_T)$ with $E_T \subseteq E$ such that T is a tree.

Definition 6.4. If G is weighted graph with a weight function $\omega : E \to \mathbb{R}$, a **minimum spanning tree (MST)** (dt. minimaler Spannbaum) is a spanning tree $T = (V, E_T)$ that minimizes $\sum_{e \in E_T} \omega(e)$ among all spanning trees of G.

Background

We already heard about the TSP as a famous problem of finding the shortest Hamiltonian cycle. Certainly a cycle is not a tree, but if we break the cycle at any edge, e.g., the most expensive one, the result is a path. A path in an undirected graph fulfills all criteria of a tree. A Hamiltonian path is also a spanning tree, because by definition Hamiltonian paths include all vertices. Hamiltonian paths are however only a subset of all spanning trees, because it imposes a greater restriction, i.e., that the tree is also a path.
Nevertheless, we can argue that any Hamiltonian path must be at least as long as a minimum spanning tree. Thus, the cost of a minimum spanning tree is a lower bound for the cost of a Hamiltonian path. As we will see later, MSTs are rather easy to compute (especially compared to the TSP). So, it can tell us something about how short a Hamiltonian path can possibly become. This may not sound very helpful, but is a very useful property in the design of algorithms for solving hard problems such as the TSP.

Figure 18 shows an example of a tree and a non-tree. The graph on the left is a tree as it is connected and acyclic. The graph on the right is not a tree as it contains the cycle (a, b, e, d, a).

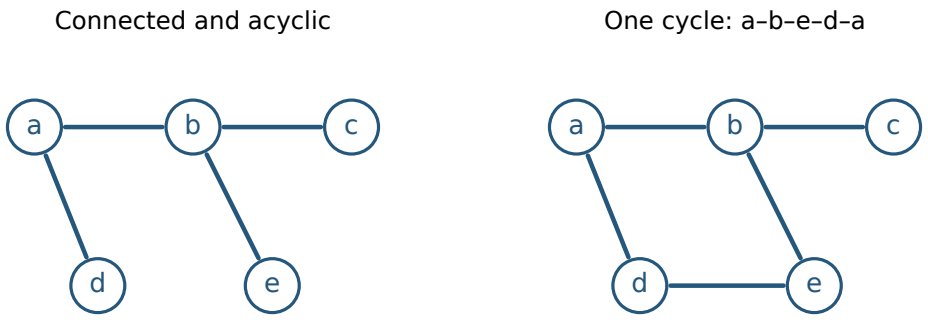


Figure 18: Example of a tree (left) and a non-tree (right)

For directed graphs, the problem is a bit different. A directed graph may even be free of cycles when it has more than $|V| - 1$ edges. Also, for a non-trivial directed graph the property that there is a path from each vertex to each other cannot be maintained when that graph must remain acyclic. Still, the problem of finding a minimum cost set of edges such that we can reach all vertices remains an interesting challenge. However, we have to introduce a single, so called, "root" vertex from which we want to reach all other vertices.

Definition 6.5. A connected, directed graph $G = (V, E)$ is called an **arborescence** (rooted tree) (dt. Wurzelbaum) if there is a vertex $r \in V$ (root) such that the following properties hold:

1. Every vertex $v \in V$ is reachable from r.
2. The root has in-degree zero.
3. Every other vertex has in-degree one.

These conditions imply a unique directed path from the root to each vertex and exclude cycles. Unique simple root-to-vertex paths alone would not suffice: $r \to a \to b \to a$ has such unique paths but also a cycle. It follows that every arborescence is a directed acyclic graph (DAG for directed acyclic graph), but not every DAG is an arborescence. Figure 19 shows an arborescence compared to a more general directed acyclic graph. The root vertex is labeled r and the leaf vertices are labeled $a, b, c, \ldots$. In the DAG, one can see that the path from r to b is not unique. Both graphs are cycle-free. It is noticeable that in an arborescence each vertex can have any number of successors, but at most one predecessor, whereas in a DAG each vertex can also have any number of predecessors.

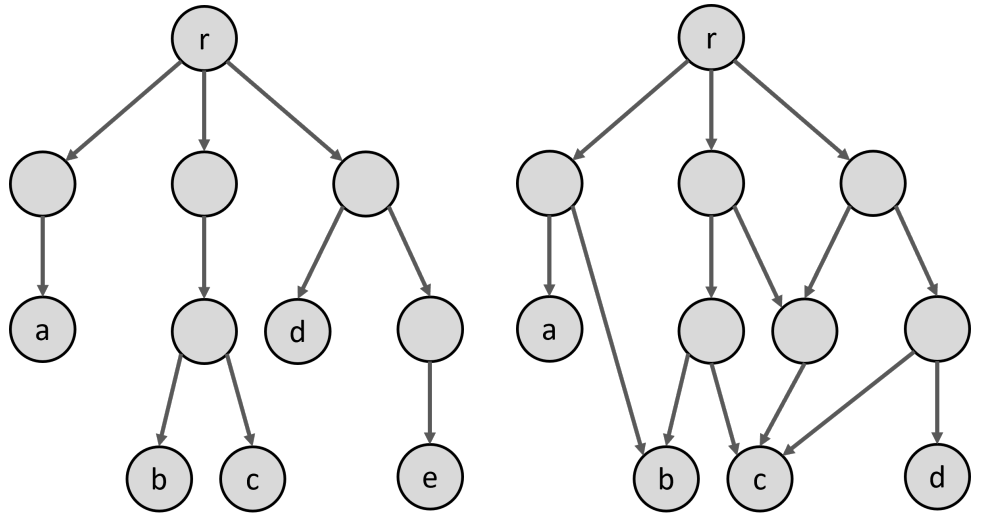
Background

The term "tree" (computer science) is not the same as the term "tree" (mathematics). While in computer science we often envision a tree as a directed graph, in mathematics, a "tree" is strictly the term that describes this special class of *undirected* graphs. Different communities may have different words for the same thing, much like in natural languages and mathematics, after all, is just another language. The only thing we can do is to know the differences and be aware of them when we talk to each other.

6.2 Prim's Algorithm

Prim's algorithm constructs a minimum spanning tree by growing a tree from an arbitrary start vertex. At each step it adds the cheapest edge that connects a new vertex to the current tree. Conceptually, this is only a minor variant of Dijkstra's algorithm: we keep the same algorithmic structure, but the update is changed from

$$\text{dist}(v) \leftarrow \text{dist}(v^*) + \omega((v^*, v))$$



(a) Arboreszenz (b) DAG

Figure 19: Comparison of arborescence and directed acyclic graph

to

$$\text{dist}(v) \leftarrow \omega((v^*, v)).$$

In other words, $\text{dist}(v)$ stores the cheapest *edge* that connects v to the current tree, not a path length.

Algorithm 15 Prim's Algorithm (Minimum Spanning Tree)

Input: A connected undirected weighted graph $G = (V, E, \omega)$ and a start vertex s
Output: A minimum spanning tree as a predecessor list W

```
1:  F ← {}
2:  dist(s) ← 0
3:  W(s) ← (s)
4:  for all v ∈ V \ {s} do
5:      dist(v) ← ∞
6:      W(v) ← ()
7:  end for
8:  while V ≠ F do
9:      v* ← argmin_{v∈V\F} dist(v)
10:     F ← F ∪ {v*}
11:     for all v ∈ N(v*) \ F do                ▷ neighbors of v*
12:         if ω((v*, v)) < dist(v) then
13:             dist(v) ← ω((v*, v))
14:             W(v) ← (v*)
15:         end if
16:     end for
17: end while
18: return W
```

The data structures are crucial for performance: Prim's algorithm uses the same two operations as Dijkstra (extract-min and decrease-key) and thus the same time complexity. With a binary heap it runs in $\mathcal{O}((V + E) \log V)$, with a Fibonacci heap in $\mathcal{O}(E + V \log V)$, and with a simple list in $\mathcal{O}(V^2)$.
If Dijkstra's algorithm is run to completion (i.e., not terminated early after reaching a target vertex z), the predecessor list also defines a spanning tree rooted at s (the *shortest-path tree*). However, this tree is not necessarily a *minimum* spanning tree, because it minimizes distances from s, not

the total edge weight.

Shortest Paths and Cheapest Connections

Suppose we want to connect three sites A, B, C by cable. The undirected edges have weights $\omega(\{A, B\}) = 2$, $\omega(\{A, C\}) = 2$, and $\omega(\{B, C\}) = 1$. If travel starts at A, the direct routes to B and C each cost 2; going via the other site costs 3. Dijkstra therefore chooses the two edges incident to A, with total tree weight 4.

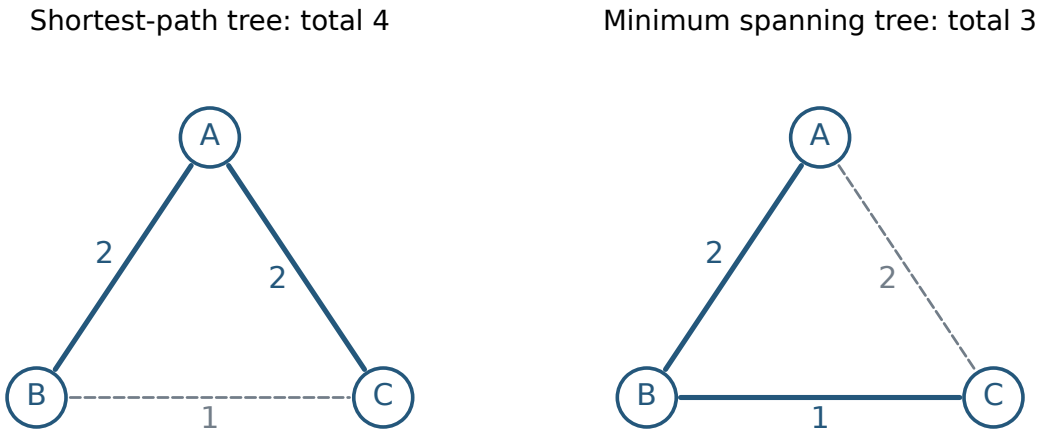


Figure 20: The same undirected triangle: a shortest-path tree from A (left) and a minimum spanning tree (right). Solid colored edges are selected; the dashed edge is omitted.

For installing cable, however, choose $\{A, B\}$ and $\{B, C\}$. Their total weight is 3, the sum of the two cheapest edges, so no spanning tree can cost less. This is an MST, although its route from A to C costs 3 instead of the graph's shortest distance 2. Choosing $\{A, C\}$ and $\{B, C\}$ gives a second MST. An MST need not preserve all shortest paths.

Prim starting at A may break the initial tie in favor of B. It then replaces C's connection cost 2 by the edge cost 1 from B. Dijkstra instead compares 2 with the *path* cost $2 + 1 = 3$ and makes no change. The update describes a different objective, even though the algorithms look almost identical.

The symbolic argument from the triangle makes the distinction precise. Write

$$x = \omega(\{A, B\}), \qquad y = \omega(\{A, C\}), \qquad z = \omega(\{B, C\}).$$

Assume the rest of the graph attaches to this triangle only at A, and reaching A costs d_0. Replacing $\{A, C\}$ by $\{B, C\}$ lowers the tree cost exactly when

$$x + z < x + y \quad \Longleftrightarrow \quad z < y.$$

But the direct route to C remains shorter than the route via B exactly when

$$d_0 + y < d_0 + x + z \quad \Longleftrightarrow \quad y < x + z.$$

Both can hold: $z < y < x + z$, as in $1 < 2 < 2 + 1$. For the whole star at A to be a shortest-path tree, also require $x \leq y + z$; for $\{A, B\}, \{B, C\}$ to be an MST of the triangle, require $x \leq y$ and $z \leq y$. Our $x = y = 2, z = 1$ satisfies these conditions.

Both trees in Figure 20 connect all three sites without a cycle. For cable installation, we minimize the total weight of the selected edges; for journeys starting at A, we minimize the distances from A. The choice of objective determines which tree is optimal.

6.3 Kruskal's Algorithm

Kruskal's algorithm takes the opposite perspective: instead of growing a tree from a single root, it starts with a forest of $|V|$ isolated vertices and repeatedly merges components by the cheapest available edge. It processes edges in increasing order and adds an edge if it does not create a cycle in the current forest.

Algorithm 16 Kruskal's Algorithm (Minimum Spanning Tree)

Input: A connected undirected weighted graph $G = (V, E, \omega)$
Output: A minimum spanning tree T

```
T ← (V, ∅)
F ← edges in E sorted by non-decreasing weight
for all e = (u, v) ∈ F do
    if adding e does not create a cycle in T then
        T ← T ∪ {e}
    end if
end for
return T
```

In a naive implementation, the cycle test can be done with a graph traversal for each edge, which is expensive. Efficient implementations use a disjoint-set data structure (union-find) to maintain components and check whether u and v are already connected in almost constant time. The total time complexity is then dominated by sorting the edges: $\mathcal{O}(E \log E)$.
Comparing time complexities: Kruskal runs in $\mathcal{O}(E \log E)$ with union-find, while Prim has the same complexity as Dijkstra: $\mathcal{O}((V + E) \log V)$ with a binary heap (or $\mathcal{O}(V^2)$ with an adjacency matrix). For sparse graphs both are similar, while for dense graphs Prim's adjacency-matrix implementation can be faster.
On the same triangle, Kruskal first selects $\{B, C\}$ of weight 1, then either edge of weight 2 to attach A. It rejects the remaining edge because that would close a cycle. Prim and Kruskal thus agree on the minimum cost 3, although a tie can give different trees.

6.4 Why the Greedy Choices Work: An Exchange Argument

The useful proof idea is to preserve an invariant: *the edges already chosen can be extended to some MST*. We need not guess that MST in advance.

Theorem 6.6 (Safe edge across a cut)**.** *Let F be a set of edges contained in some MST of a connected undirected weighted graph. Choose a nonempty proper vertex set S such that no edge of F crosses between S and $V \setminus S$. If e is a minimum-weight edge crossing this cut, then $F \cup \{e\}$ is contained in some MST.*

Proof. Take an MST T containing F. If it already contains e, we are done. Otherwise, adding e creates exactly one cycle. What must that cycle do after crossing from S to $V \setminus S$ along e? It must cross back. Let f be another crossing edge on the cycle. Since F has no crossing edges, $f \notin F$.

Remove f. The resulting graph $T' = T + e - f$ is connected and acyclic, hence still a spanning tree, and it contains $F \cup \{e\}$. Because e was a cheapest crossing edge,

$$\omega(T') = \omega(T) + \omega(e) - \omega(f) \leq \omega(T).$$

But T was already minimum, so equality holds and T' is an MST. We have exchanged an edge without making the solution worse. □

For *Prim*, take S to be the vertices of the growing tree. Its next edge is a cheapest crossing edge. For *Kruskal*, take S to be the vertex set of one current component incident to its next accepted edge. A cheaper crossing edge would already have been processed and merged the components, so the accepted edge is cheapest across that cut too. Thus both preserve the invariant. After $|V| - 1$ accepted edges, their spanning tree is itself an MST.

Ties matter: "a cheapest edge is safe" means it belongs to *some* MST extending F, not necessarily every MST. The triangle gives two MSTs. Distinct edge weights guarantee uniqueness: if two different MSTs existed, take the cheapest edge in their symmetric difference, add it to the tree missing it, and remove a heavier edge of the resulting cycle that is absent from the other tree. This contradicts minimality. Distinct weights are sufficient, but not necessary; a graph that is already a tree has a unique spanning tree even if every edge has the same weight.

This argument also explains why MST algorithms allow negative weights. They select an acyclic set of edges and pay each weight once; the repeated backtracking that obstructs shortest walks does not occur.

6.5 Chu-Liu/Edmonds Algorithm

Further Connections

This section extends the minimum-spanning-tree discussion to directed graphs and is conceptually more demanding because a locally optimal selection of incoming edges may create cycles that have to be contracted and resolved recursively.

Similar to the minimum spanning tree, we now want to find a spanning arborescence whose edge weights have a minimal sum. However, Kruskal's and Prim's algorithms cannot be directly applied to directed graphs. In a minimum spanning tree, every vertex can be treated as the root, so it does not matter where one starts. When edges are directed, however, a root must be defined. Moreover, all vertices except the root have exactly one incoming edge. This can be guaranteed neither by Kruskal nor by Prim, i.e., it would require a modification that forbids such edges.

If one still wants to apply an algorithm for undirected graphs, such as a modified Kruskal, to directed graphs, the result is not optimal. The graph in Figure 21 illustrates that the greedy edge selection fails. If one chooses the edge of weight 1 first, only the two edges of weight 9 remain available, because the edges of weight 2 would on the one hand create a cycle and, in any case, increase a vertex' in-degree to 2. The minimum-cost arborescence, however, contains exactly the two edges of weight 2: $\{(c, a), (a, b)\}$.

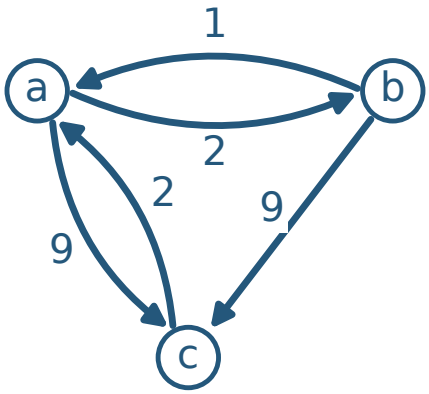


Figure 21: Example of a directed graph where the greedy edge selection fails

The Chu-Liu/Edmonds algorithm can be applied to directed graphs to find a minimum-cost arborescence (if one exists). The algorithm was developed independently by Chu-Liu and Edmonds in 1965 and 1967, respectively.

There are minimum- and maximum-weight formulations; negating the weights converts one to the other. Chu and Liu's 1965 paper explicitly treats shortest arborescences. Here we minimize costs.[1]

[1] `https://www.cs.cmu.edu/~15850/handouts/chu-liu_1965.pdf`

Edmonds Algorithm

With a *specified root* r, we seek a cheapest spanning arborescence directed out of r. Such an arborescence exists exactly when every vertex is reachable from r. Incoming edges to r and self-loops cannot belong to it, so we remove them before starting the algorithm.

If the root should be *chosen by the optimization*, add an artificial root R and an edge (R, v) to every original vertex. This is useful in our example: arbitrarily fixing an original root can change the optimum. Let $n = |V|$, let $B = \max_{e \in E} |\omega(e)|$ (take $B = 0$ if E is empty), and give each artificial edge weight $M > 2nB$.

Why is this enough? An augmented spanning arborescence has n edges. The original-edge contribution to any such tree lies between $-nB$ and nB, so two such contributions differ by at most $2nB$. One extra artificial edge costs more than any possible saving in original edges. Thus the optimum first minimizes the number of artificial edges, then the original cost. If the original graph has a spanning arborescence for some root, exactly one artificial edge is selected. Remove R and that edge; its endpoint is the chosen root. If the optimum needs more than one artificial edge, no single-root spanning arborescence exists in the original graph. Equivalently, one can minimize the pair (number of artificial edges, original cost) lexicographically.

Merely choosing M larger than every individual edge weight is insufficient: the saving from changing several edges can be larger. For our three-vertex example, $B = 9$ and $M = 55$ satisfy the bound. We retain R in the contraction sequence so that this root-selection problem remains a rooted problem at every recursive step.

The algorithm starts by selecting, for each vertex, the *incoming edge with the smallest weight*. Selecting the cheapest incoming edge gives a lower bound, since every non-root vertex needs exactly one incoming edge. The selected edges are not yet guaranteed to form a feasible arborescence. However, this set of edges can contain cycles; an artificial root does not by itself force a cycle. If an original vertex has no incoming original edge, an artificial edge can be selected immediately.

Considering the example in Figure 21, the edges with the smallest incoming weight for each vertex are the edges $\{(b, a), (a, b), (a, c)\}$. It is easy to see that a cycle is present here. Since we allow only one incoming edge into this cycle, we must replace one edge in the cycle, i.e., break the cycle at a vertex. This results in a new problem of the same type as the original. We treat the cycle as a single vertex and thereby obtain a graph with fewer vertices. If there are multiple cycles, we pick one arbitrarily. In the Chu-Liu/Edmonds algorithm, we now construct a new graph in which we replace the vertices in the cycle $C = (V_C, E_C)$ by a single vertex k_C. To incorporate the cycle into the arborescence, the costs of incoming edges into the cycle (i.e., into vertex k_C) must be adjusted to account for the costs of breaking it. We denote by $\text{pre}_c \in V_C$ the predecessor of $c \in V_C$ in the cycle. The weights are assigned as follows:

- Edges with both endpoints outside the cycle retain their weights. Edges internal to the cycle disappear during contraction.
- Outgoing edges retain their weights. For each outside destination separately, keep a cheapest representative, remembering its original endpoints for expansion.
- The weights of all incoming edges into the cycle are adjusted. We denote by $X_u = \{(u, c) \in E \mid c \in V_C\}$ the set of all edges entering the cycle with predecessor $u \notin V_C$, so the cost of $e = (u, k_C)$ is $\omega(e) = \min_{(u,c) \in X_u} \big(\omega((u, c)) - \omega((\text{pre}_c, c))\big)$.

We therefore reduce the costs of incoming edges by the cost of the edge in the cycle that we could replace. Figure 22 shows an excerpt of how the cycle in our example leads to a new vertex. In this case, the cycle consists of $V_C = \{a, b\}$, $E_C = \{(a, b), (b, a)\}$. There is an incoming edge from c: (c, a). Since this edge would replace the edge (b, a) (because a can only have one predecessor), the costs are reduced by the cost of this edge.

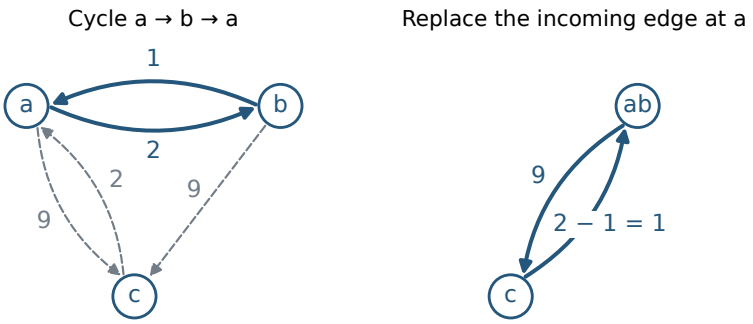


Figure 22: Example of contracting a cycle

The algorithm is now called recursively with the modified graph. If no cycle remains, the base of the recursion is reached and, while unwinding, we can reconstruct the arborescence. The Chu-Liu/Edmonds algorithm thus finds the minimum-cost spanning arborescence. When expanding a cycle, restore the original entering edge (u, c) and all selected cycle edges except (pre_c, c). Restore the saved outgoing representatives too. The full sequence of the example and the artificial root is shown in Figure 23.

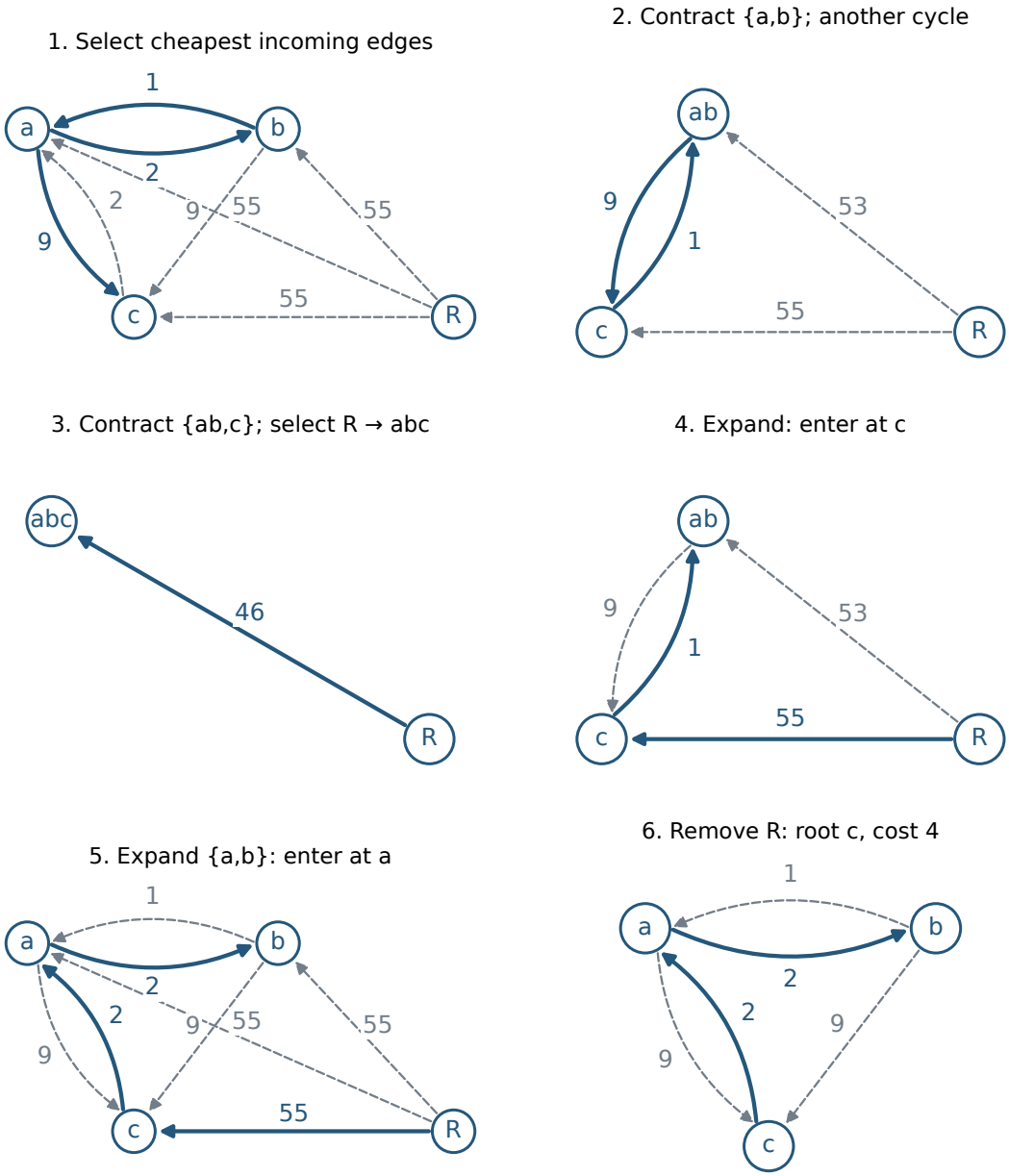


Figure 23: Choosing the root with $M = 55$: contract $\{a, b\}$, then $\{ab, c\}$; expand by entering at c, then at a. Removing R leaves $c \to a \to b$ of cost 4.

7 Adversarial Search and the Minimax Algorithm

So far the path and spanning tree problems we have seen can be generalized as "find a selection of edges that minimizes the sum of edge weights under some constraints". The adversarial search problem is fundamentally different. There are two deciding actors whose interests may conflict. The graph of game states need not be a tree: different move sequences can reach the same position, and moves can sometimes return to an earlier position. For search, we unfold possible histories into a rooted tree. A state reached by different histories can therefore appear in several tree vertices. We must be aware that a set of edges is chosen exactly opposite to our interests. Typically, this is the case for two player games like "Tic-Tac-Toe", connect four, chess, checkers, othello/reversi, and many others. A related category are games such as 2048 where a random unbiased process puts a new block in the board. It's not exactly an opponent as it does not play optimally, but the problem setting is somewhat similar. The goal of a minimax application is to make a decision such that the worst-case outcome is best.

Pause and Think – Challenge

In the web context, it is possible to model an attacker/defender scenario as a game where each responds in turns to the decisions of the other. An automated defense system could decide on a number of countermeasures given a model of the attacker using a minimax approach.

7.1 Problem Setting

Definition 7.1. Consider a deterministic two-player game with perfect information, alternating turns, and zero-sum utility measured from MAX's perspective (MIN's utility is its negative). Assume its histories form a finite rooted game tree G. Games with repetition require termination rules or a finite search horizon; depth-limited evaluation is introduced below. Vertices are typically denoted as states and hence we use s here. Terminal vertices have utilities given by a value function val. Vertices at the same depth share a common theme: At a MAX vertex we choose a successor with maximum value; at a MIN vertex the opponent chooses a successor with minimum value. Those themes alternate as the depth increases. The Minimax value is defined recursively as

$$\text{value}(s) = \begin{cases} \text{val}(s) & \text{if } s \text{ is terminal,} \\ \max\limits_{w \in Suc(s)} \text{value}(w) & \text{if } s \text{ is a MAX vertex,} \\ \min\limits_{w \in Suc(s)} \text{value}(w) & \text{if } s \text{ is a MIN vertex.} \end{cases}$$

Pause and Think – Quick Check

Why do maximum and minimum alternate between consecutive levels of the game tree? Which assumptions about the opponent are hidden in this alternation?

7.2 A Small Counterexample to Greedy Play

Let's consider the rooted tree in Figure 24. That tree has one MAX layer, one MIN layer, and one terminal layer. The edges are the actions that can be taken.

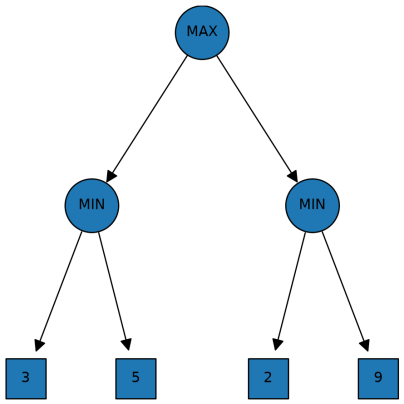


Figure 24: Minimax example

Pause and Think – Work It Out

Perform the value-propagation by hand. Assuming optimal play by both parties what will be the outcome? What if you expect your opponent to play sub-optimally, how could that improve your outcome?

The value represents the advantage of either player. A positive value is an advantage for the MAX player, a negative value is an advantage for the MIN player. In a typical zero-sum game, a winning terminal vertex for the MAX player will have a value of 1, while for the MIN player it will be -1. A draw would be assigned a 0.

Pause and Think – Quick Check

Make a quick list of all the challenges you can think of when applying this in practice for a suitable scenario.

7.3 Algorithm

The minimax algorithm is given in Algorithm 17. It is a recursive implementation whereby the layers alternate between MIN and MAX, meaning that the players each take a turn one after another. This model may apply even if one player has multiple moves - these can still be compressed to one layer, but with a higher number of successor states.
The running time of this algorithm is $\Theta(|V| + |E|) = \Theta(|V|)$ since the number of edges in a rooted tree is similar to the number of vertices. While, this may look very efficient - linear in the amount of vertices, we have to face the fact that the number of such vertices can be extremely high. Consider a branching factor of at most b and the search depth is d, the complexity is $\mathcal{O}(b^d)$. The recursive implementation additionally uses $\Theta(d)$ auxiliary space for the call stack, excluding the space needed to store the game tree itself.
So, a big challenge for this algorithm is the size of the rooted tree. For chess, an estimate of roughly $4.8 \cdot 10^{44}$ concerns legal *positions*, not vertices of the full history tree[2]. For reference, the largest value a 64 bit integer can hold is approximately $1.8 * 10^{19}$. A full evaluation of the tree seems out of question. Here are two common additions to the basic minimax algorithm:

- Pruning the tree, i.e., not descending into every successor
- Depth-limiting the search and computing a utility for non-terminal vertices

[2] `https://github.com/tromp/ChessPositionRanking`

Algorithm 17 Recursive Minimax Algorithm

Input: A rooted game tree $G = (V, E)$ with root $r \in V$ and $T \subset V$ terminal states, a function $\text{val} : T \to \mathbb{Z}$
Output: The value of r under optimal choices

```
1: function MINIMAX(s, maximizingPlayer)
2:     if s ∈ T then                                   ▷ s is terminal
3:         return val(s)
4:     end if
5:     if maximizingPlayer then
6:         best ← −∞
7:         for all w ∈ Suc(s) do
8:             best ← max(best, MINIMAX(w, False))
9:         end for
10:    else
11:        best ← ∞
12:        for all w ∈ Suc(s) do
13:            best ← min(best, MINIMAX(w, True))
14:        end for
15:    end if
16:    return best
17: end function
18: return MINIMAX(r, True)
```

7.4 Alpha-beta pruning

The recursive Minimax algorithm computes the value of every vertex in the rooted tree. However, sometimes we know that this would not influence the search anymore. Why? Consider, that we're currently evaluating the value of a MIN vertex which naturally has a MAX vertex as predecessor. The MAX vertex has already observed a value of 5 after exploring its first successor (and thus the subtree below it). Certainly, we can say that MAX will not choose any other vertex as successor that has a lower value than 5. We can say the value of MAX is now *bounded* to be ≥ 5. Our MIN vertex has also evaluated its first successor (and thus the subtree below it) and has evaluated to a value of 3. The MIN vertex' value is now bounded to be ≤ 3. We can say 3 is an upper bound on the value of the MIN vertex, while 5 is a lower bound. However, which value x that satisfies $5 \leq x \leq 3$ could still possibly influence our decision? Exactly: There is none and thus the remaining successors of this MIN vertex need not be evaluated anymore. For our MIN vertex, none of the remaining successors would influence the decision of the preceding MAX vertex anymore. Consequently, why still evaluate them - if evaluation is that costly? This same example can be drawn for a MIN - MAX pair of course and this applies in every depth of the rooted tree.
In summary, alpha-beta pruning keeps track of two bounds while traversing the tree:

- α is the greatest value that MAX can already guarantee on the current path.
- β is the smallest value that MIN can already guarantee on the current path.

Both bounds are initialized at the root as $-\infty$ and $+\infty$, respectively. At a MAX vertex, α is updated whenever a better child is found; at a MIN vertex, β is updated whenever a smaller child is found. Whenever $\alpha \geq \beta$, the remaining successors can be skipped.

Pause and Think – Quick Check

Look at Algorithm 18 and think about this: How you would be able to increase the number of pruned vertices? Because clearly, the more vertices you can prune the faster you can complete the evaluation of the full rooted tree.

Alpha-beta pruning returns exactly the same value as minimax. Its *worst-case* running time is still the same as the basic minimax algorithm. In practice however, it works much better.

Algorithm 18 Alpha-beta Minimax

Input: A state s, bounds α and β
Output: The minimax value of s

```
1: function AlphaBeta(s, α, β, maximizingPlayer)
2:     if s is terminal then
3:         return val(s)
4:     end if
5:     if maximizingPlayer then
6:         best ← −∞
7:         for all w ∈ Suc(s) do
8:             best ← max(best, AlphaBeta(w, α, β, False))
9:             α ← max(α, best)
10:            if α ≥ β then
11:                return best
12:            end if
13:        end for
14:    else
15:        best ← ∞
16:        for all w ∈ Suc(s) do
17:            best ← min(best, AlphaBeta(w, α, β, True))
18:            β ← min(β, best)
19:            if β ≤ α then
20:                return best
21:            end if
22:        end for
23:    end if
24:    return best
25: end function
26: return AlphaBeta(r, −∞, +∞, True)
```

Nevertheless, the number of pruned vertices strongly depends on the order in which successors are examined. With ideal *move ordering*, the number of examined vertices can be reduced to $\mathcal{O}(b^{d/2})$. Consequently, move ordering is an important practical optimization even though it does not change the result.

Pause and Think – Quick Check

What does *ideal* mean here, what would it look like?

In the example tree in Figure 24 alpha-beta pruning would skip the last vertex. After having completed the left branch of the MAX vertex we observe $\alpha = 3$. After observing the first child of the second MIN vertex, $\beta = 2 \leq \alpha$. Thus this MIN call skips its remaining child; the root finishes after completing its remaining required work.

7.5 Depth-limiting search

While pruning certainly helps evaluating large trees, there are still trees of such size that full traversal is not feasible, at least not in a human lifetime. Depth-limited minimax proposes to stop after a fixed number L of moves. But then, what utility or value would you use? At a depth cutoff, the state is usually non-terminal, so its utility is unknown. For this purpose, we'll introduce an evaluation function h that estimates the value of the state. The function should use only information available in the state and should return larger values for positions that are better for MAX and smaller values for states that are better for MIN.
If the branching factor is at most b, depth-limited search has running time $\mathcal{O}(b^L)$, assuming that h takes constant time to compute. While alpha-beta pruning can be combined with depth limiting,

Algorithm 19 Depth-limited Minimax

Input: A state s, remaining depth ℓ
Output: An estimated minimax value of s

```
1: function DepthLimitedMinimax(s, ℓ, maximizingPlayer)
2:     if s is terminal then
3:         return val(s)
4:     end if
5:     if ℓ = 0 then
6:         return h(s)
7:     end if
8:     if maximizingPlayer then
9:         best ← −∞
10:        for all w ∈ Suc(s) do
11:            best ← max(best, DepthLimitedMinimax(w, ℓ − 1, False))
12:        end for
13:    else
14:        best ← ∞
15:        for all w ∈ Suc(s) do
16:            best ← min(best, DepthLimitedMinimax(w, ℓ − 1, True))
17:        end for
18:    end if
19:    return best
20: end function
21: return DepthLimitedMinimax(r, L, True)
```

it does not affect the worst-case runtime bound $\mathcal{O}(b^L)$. Still, pruning and move ordering reduces the number of evaluated states substantially in practice.
In addition, in Algorithm 19 we can see that it may return both val and h from this function and they're subsequently compared by min and max. So, these two functions need to operate on the same perspective and scale, h must approximate the value of a state given optimal play.
The main risk for the depth-limited variant is the *horizon effect*. An event that is important to the decision may lie just beyond the cutoff, causing the evaluation function to miss it or to postpone it artificially. A common mitigation in chess is quiescence search: after reaching the nominal depth limit, continue exploring forcing or unstable moves, such as captures or checks, until the position is quiet enough for h to give a more reliable estimate.

7.6 Expectimax

We mentioned the game of 2048[3] above. What if the opponent does not play against us, but simply acts randomly? Expectimax replaces the MIN vertices with chance vertices. Instead of taking the minimum, we compute the average of the successor values weighted by their probabilities. At MAX vertices we still choose the maximum. Consequently, we choose the move with the *best expected outcome* rather than the best worst-case outcome.
Depth-limiting works just as before, but h should now estimate the expected outcome under the given probabilities. Alpha-beta pruning does not apply directly: at a chance vertex, every successor with non-zero probability contributes to the average. Pruning is still possible if we have bounds on the remaining successor values that guarantee they cannot change our decision.

[3] `https://en.wikipedia.org/wiki/2048_(video_game)`

8 PageRank and Centrality

So far, we have mostly treated vertices as atomic objects: we can identify them and connect them, but we have not asked what makes one vertex more important than another. We have searched for paths, selected edges for spanning trees, and computed reachability. Naturally, we have already assigned a property to individual vertices, namely their degree. Now, however, the properties of the vertices themselves become our main interest. Which vertex occupies an important position in a graph? And what does "important" even mean?

In this chapter, we first study this question for web pages. We will turn hyperlinks into a mathematical model, explain why a suitable ranking exists, and compute it with a simple iterative algorithm. Afterward, we will consider other meanings of importance, called *centrality measures* (dt. Zentralitätsmaße). These will reconnect us with vertex degrees and shortest paths. We need a little linear algebra along the way, but our goal is to understand what the mathematical results allow us to do, rather than prove those results.

8.1 From Web Pages to a Mathematical Model

In the second half of the 1990s, Larry Page and Sergey Brin developed the ideas behind PageRank at Stanford. Their 1998 paper describes Google as a search engine that uses both page content and the structure of hyperlinks [3]. A separate report by Page, Brin, Motwani, and Winograd develops PageRank in more detail [4]. The interesting observation is that a hyperlink provides information beyond the text of a page: somebody chose to link to another page. Perhaps this can be interpreted as a recommendation?

We model a collection of web pages as a directed graph $G = (V, E)$. Each page is a vertex, and $(u, v) \in E$ means that page u contains a hyperlink to page v. Multiple links from the same page to the same target count as one edge here. A crawler can acquire such a graph by downloading pages and extracting their links. The graph contains only the pages the crawler could discover at that time.

Pause and Think – Quick Check

If you wanted to find an important web page, would you count its incoming or its outgoing links? Could a page author easily increase either of these numbers?

Counting incoming links is a reasonable first attempt. But should a recommendation from an important page count as much as one from a page that nobody knows? And should a page recommending a thousand other pages give each of them as much support as a page recommending only one? PageRank combines two assumptions: recommendations from important pages should carry more weight, and a page should divide its weight among its outgoing links.

Background

A mathematical model is an abstraction, just as the graph models of the Königsberg bridges were abstractions in Section 3.2. A hyperlink may be a recommendation, but it may also be criticism, advertising, or an automatically generated link. PageRank measures importance according to its assumptions; it does not measure truth or the relevance of a page to every possible search query.

Three things came together in this application: the link model was useful enough to improve ranking, the data could be acquired, and the resulting mathematical problem could be solved in practical time. A useful model that takes centuries to compute will not help a search engine. Conversely, a fast algorithm for a model that measures the wrong thing will not help either. The success of the early approach depended on both mathematics and engineering. Computing PageRank in advance for the acquired graph made its scores available for many subsequent search queries.

Why This Matters

Knowing how to program is necessary to build such a system, but how would we invent the computation if we did not know the mathematics? Graphs let us express the data, linear algebra lets us formulate the ranking, and mathematical results tell us when the computation makes sense. Learning these concepts gives us tools for recognizing opportunities that would otherwise remain invisible.

8.2 Recommendations and the Random Surfer

We will use the same small web graph throughout the computation. Let $V = \{A, B, C, D\}$ and

$$E = \{(A, B), (A, C), (B, C), (C, A), (D, C)\}.$$

Figure 25 shows this graph. Page C has three incoming links, A and B each have one, and D has none.

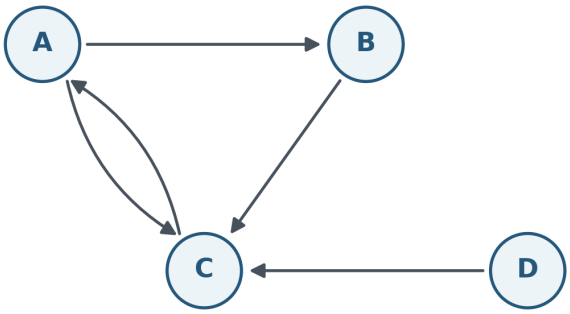


Figure 25: A small web graph. An arrow points from the page containing a link to its target.

Suppose page v has a score r_v. Each page u with an outgoing link to v passes on the fraction $r_u/\text{outdeg}(u)$. Thus, our first attempt is to require

$$r_v = \sum_{u \in Pre(v)} \frac{r_u}{\text{outdeg}(u)}. \tag{6}$$

We use $Pre(v)$ and $\text{outdeg}(u)$ as defined in Section 1.2. For now, assume that every page has at least one outgoing link. Notice the apparent circularity: to determine a page's importance we need the importance of other pages, which may in turn depend on the first page. This is a system of simultaneous equations, not a recursive procedure that follows links until it finds a final answer.
There is another way to read the same equation. Imagine a *random surfer* who, at each step, chooses one outgoing link uniformly at random and follows it. If r_u is the probability of being at page u, then $r_u/\text{outdeg}(u)$ is the probability of being at u and next following its link to v. Summing these contributions gives the probability of arriving at v. We require $r_v \geq 0$ and $\sum_{v \in V} r_v = 1$, because the surfer must be somewhere. Equation (6) describes a distribution that does not change after one step.

Notation

Fix the vertex order (A, B, C, D) and write probabilities as a *column vector* r. We use a matrix P whose column is the *source* and whose row is the *destination*: P_{ij} is the probability of moving from v_j to v_i. If an adjacency matrix uses rows as sources, our matrix therefore uses the transposed orientation. With a row-vector convention one would instead multiply from the right; both conventions work, but we must not mix them.

Definition 8.1. A matrix $P \in \mathbb{R}^{n \times n}$ is *column-stochastic* (dt. spaltenstochastisch) if $P_{ij} \geq 0$ and $\sum_{i=1}^{n} P_{ij} = 1$ for every column j. For a web graph without pages of out-degree zero, we define

$$P_{ij} = \begin{cases} 1/\text{outdeg}(v_j) & \text{if } (v_j, v_i) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

For our example this gives

$$P = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1/2 & 0 & 0 & 0 \\ 1/2 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad r^{(k+1)} = Pr^{(k)}. \tag{7}$$

The superscript (k) counts steps; it is not a power of the vector. Check the first column: page A sends half its probability to B and half to C. Every column sums to one, so multiplying by P preserves the total probability.

Pause and Think – Work It Out

Start with $r^{(0)} = (1/4, 1/4, 1/4, 1/4)^T$ and compute $Pr^{(0)}$. Why does page D lose all its probability? Why do A and B receive different amounts despite having the same in-degree?

8.3 Existence, Uniqueness, and Convergence

We now know how to describe a step. But does there exist a distribution that remains unchanged? Is there only one? And will repeated steps find it? These are three different questions.

Definition 8.2. A *stationary distribution* (dt. stationäre Verteilung) of a column-stochastic matrix P is a vector r with $r_i \geq 0$, $\sum_i r_i = 1$, and $Pr = r$.

Recall that an eigenvector $x \neq 0$ of a matrix satisfies $Px = \lambda x$, where λ is its eigenvalue. A stationary distribution is therefore an eigenvector for the eigenvalue 1, normalized to sum to one. The surfer may still move; it is the distribution over pages that remains unchanged.

Theorem 8.3. *Every finite column-stochastic matrix has eigenvalue* 1 *and at least one stationary distribution. All its eigenvalues have absolute value at most* 1*.*

We state the linear-algebra results in this section without proof. Their assumptions and consequences are what we need for our application. Stochasticity gives existence, but does not by itself give a unique ranking. For example, two disconnected groups that each keep all their probability can retain different amounts depending on the initial distribution.

Definition 8.4. A nonnegative matrix P is *irreducible* (dt. irreduzibel) if its directed transition graph, with an edge $v_j \to v_i$ whenever $P_{ij} > 0$, is strongly connected.

This is exactly the graph property we already know from Section 2: every vertex can reach every other vertex. Equivalently, for every i, j there is some $k \geq 0$ with $(P^k)_{ij} > 0$. Here $P^0 = I$ covers the trivial walk from a vertex to itself.

Background

A transition graph is just a special name for a graph in which the vertices are states and the edges represent a transition between these states.

Theorem 8.5. *An irreducible finite column-stochastic matrix has exactly one stationary distribution, and every entry of this distribution is strictly positive.*

This is a consequence of the Perron–Frobenius theory for nonnegative matrices. But beware: it still does not say that repeated multiplication converges. Consider two pages that link only to

each other. The graph is strongly connected and its unique stationary distribution is $(1/2, 1/2)^T$. Starting from $(1, 0)^T$, however, the distributions alternate forever between $(1, 0)^T$ and $(0, 1)^T$.

The missing condition is *aperiodicity*. In a strongly connected transition graph, it means that the greatest common divisor of the possible positive return lengths to a vertex is 1. In the two-page example, all return lengths are even. A self-loop in a strongly connected transition graph suffices to make it aperiodic, because a return in one step is then possible.

Theorem 8.6. *If a finite column-stochastic matrix is irreducible and aperiodic, then $P^k r^{(0)}$ converges to its unique stationary distribution for every initial probability vector $r^{(0)}$.*

These conditions are sufficient; we need not claim that every reducible graph fails. Our example is reducible because no page can reach D, yet the undamped iteration converges to $(2/5, 1/5, 2/5, 0)^T$. The problem is that an arbitrary web graph does not promise the properties on which a general-purpose ranking algorithm could rely.

8.4 Dangling Pages, Rank Traps, and Teleportation

There are several obstacles to the first model, illustrated in Figure 26:

- A *dangling page* has no outgoing links. The surfer has nowhere to go, and the corresponding column cannot be normalized by its out-degree.
- A *rank trap* is a group that has no links leading out. Once the surfer enters it, the surfer cannot leave. Several such closed groups can also prevent uniqueness.
- A periodic transition graph can make the iteration oscillate even when a unique stationary distribution exists.

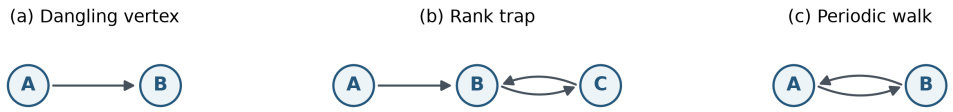


Figure 26: Three problems of following only hyperlinks: a dangling page, a closed group of pages, and periodic motion.

First, we give a dangling page a defined behavior: the surfer chooses any of the $n = |V|$ pages uniformly, including the current page. We replace its zero column by $(1/n, \ldots, 1/n)^T$. From now on, P denotes this repaired column-stochastic matrix. This repair alone does not remove rank traps elsewhere in the graph.

Second, at every step the surfer follows the transitions in P with probability α, and with probability $1 - \alpha$ chooses any page uniformly. This second choice is called *teleportation* and α is called a "damping factor". For our random surfer, this models the behavior that it does not always follow a link, but sometimes enters a new address and jumps to a different page unrelated to the current page. We could introduce an extra link on each page for this behavior, but that would also imply that pages with more links have a reduced probability to teleport. We assume $0 < \alpha < 1$ and use $\alpha = 0.85$ in our example, a conventional choice also used in the early work [3]. Note that α is a model parameter, not a mathematical constant. Potentially, we can measure this from real-world surfers and estimate this probability. Model parameters introduce complexity, because if these are wrong, the model may not align well with the real world.

Definition 8.7. Let $q = (1/n, \ldots, 1/n)^T$ and let $\mathbf{1}$ be the column vector of n ones. The *Google matrix* is

$$M = \alpha P + (1 - \alpha) q \mathbf{1}^T.$$

The *PageRank vector* r is the stationary distribution of M:

$$r = Mr = \alpha P r + (1 - \alpha) q, \qquad \sum_i r_i = 1. \tag{8}$$

Every entry of M is at least $(1-\alpha)/n > 0$. Thus, its transition graph is strongly connected and has self-loops. Theorems 8.5 and 8.6 now guarantee a unique strictly positive PageRank vector and convergence from any initial probability vector. These additional transitions belong to our model; we have not inserted real hyperlinks into the web pages.

Pause and Think – Quick Check

What happens if $\alpha = 0$? What guarantee do we lose if $\alpha = 1$? Explain both answers using the surfer, rather than only manipulating the equation.

One can also choose a nonuniform teleportation distribution q to favor a set of topics or starting pages. This gives *personalized PageRank*. If every $q_i > 0$, the same positivity argument applies. If some entries are zero, that particular argument no longer applies, so we must take care when claiming strict positivity of every score.

8.5 Computing PageRank with the Power Method

We could solve Equation (8) as a system of linear equations. But a web graph may have millions of vertices, and we do not want to store or eliminate a dense matrix of that size. Instead, we repeatedly apply the transition rule. This is called the *power method* (dt. Potenzmethode), because after k steps we have $r^{(k)} = M^k r^{(0)}$. We never need to form M^k explicitly.

Algorithm 20 implements the same multiplication using outgoing adjacency lists. All updates use the old vector r and accumulate in a separate vector s. The quantity z collects the probability at dangling pages.

Algorithm 20 PageRank by Power Iteration

Input: Directed graph $G = (V, E)$, $n = |V| > 0$, $0 < \alpha < 1$, tolerance $\varepsilon > 0$, iteration limit $K \geq 1$
Output: Approximate PageRank vector, or a report that the iteration limit was reached

```
r[v] ← 1/n for every v ∈ V
for k = 1, ..., K do
    z ← Σ_{u∈V: outdeg(u)=0} r[u]
    s[v] ← (1 − α)/n + αz/n for every v ∈ V                ▷ Teleportation
    for u ∈ V with outdeg(u) > 0 do
        for v ∈ Suc(u) do
            s[v] ← s[v] + αr[u]/outdeg(u)
        end for
    end for
    δ ← Σ_{v∈V} |s[v] − r[v]|
    if δ ≤ ε then
        return s
    end if
    r ← s
end for
return r with a report that the iteration limit was reached
```

The stopping test measures the total absolute change, also called the 1-norm of the difference. Small change and small error are not identical. For this damped iteration one can bound the error of the newly computed vector by

$$\|s - r^*\|_1 \leq \frac{\alpha}{1-\alpha} \|s - r\|_1,$$

where r^* is the exact PageRank vector and s is the next iterate after r. We do not prove this bound here; it follows from the fact that one step contracts distances between probability vectors by a factor at most α. To guarantee error at most η in exact arithmetic, use $\varepsilon \leq \eta(1-\alpha)/\alpha$. An iteration limit is useful to detect when the requested accuracy was not reached.

Each iteration takes $O(|V|+|E|)$ time and the graph plus the two vectors require $O(|V|+|E|)$ memory. For k iterations the time is $O(k(|V|+|E|))$. Although M is dense because of teleportation, the algorithm only traverses real edges and adds a uniform contribution. This is the crucial computational observation: the convenient mathematical description does not force us to store a dense matrix. Increasing α gives the link structure more influence, but also weakens the convergence bound and may require more iterations.

8.5.1 The Relationship to Transitive Closure

Does repeated multiplication look familiar? In Section 4.1, Boolean matrix powers described reachability after a fixed number of steps. Here, powers of a transition matrix describe probabilities after a fixed number of steps. In both cases, multiplication combines walks through intermediate vertices. For example,

$$(P^2)_{ij} = \sum_{\ell=1}^{n} P_{i\ell} P_{\ell j}$$

adds the probabilities of all two-step walks from v_j to v_i. If we only ask whether this entry is positive, we recover the corresponding Boolean reachability question, with the row/column orientation adjusted. Ordinary powers of an unweighted adjacency matrix instead count walks.

The difference is what we keep. Boolean multiplication combines possibilities with AND and OR; stochastic multiplication multiplies step probabilities and adds contributions of alternative walks. Transitive closure combines reachable pairs over different lengths. PageRank follows a probability distribution for longer and longer walks until it is close to equilibrium. It neither accumulates a Boolean union nor stops after $n-1$ steps: walks may revisit vertices arbitrarily often. Also, PageRank uses matrix–vector products, while the closure construction computed matrix–matrix products. Teleportation makes every pair reachable in one model step, but their long-run probabilities can still differ considerably.

Pause and Think – Work It Out

In our web graph, starting from A, which pages can the surfer reach in exactly two link-following steps? Calculate their probabilities with P^2. Then explain what information a Boolean two-step matrix would retain and what it would discard.

8.5.2 The Computation on Our Web Graph

We now apply Algorithm 20 to Figure 25. There are no dangling pages, so $z = 0$. For $\alpha = 0.85$ the uniform contribution is $(1-\alpha)/4 = 0.0375$ per page. The update equations are

$$\begin{aligned}
r_A^{(k+1)} &= 0.85 r_C^{(k)} + 0.0375, \\
r_B^{(k+1)} &= 0.85 r_A^{(k)}/2 + 0.0375, \\
r_C^{(k+1)} &= 0.85(r_A^{(k)}/2 + r_B^{(k)} + r_D^{(k)}) + 0.0375, \\
r_D^{(k+1)} &= 0.0375.
\end{aligned}$$

Starting from $r^{(0)} = (0.25, 0.25, 0.25, 0.25)^T$, the first multiplication gives

$$r^{(1)} = (0.25,\ 0.14375,\ 0.56875,\ 0.0375)^T.$$

The entries still sum to one. In the next step, A benefits from the increased probability at C, while C receives less from B and D. Thus, individual scores need not approach their limits monotonically.

The resulting ranking is $C > A > B > D$, with scores approximately 0.394149, 0.372527, 0.195824, and 0.037500, respectively. Pages A and B have equal in-degree, but A receives the whole link-following contribution of the highly ranked page C, while B receives only half of A's contribution. Page D has a positive score because of teleportation. Notice also that the undamped model gave A and C equal scores; changing the model can change the ranking.

k	$r_A^{(k)}$	$r_B^{(k)}$	$r_C^{(k)}$	$r_D^{(k)}$
0	0.250000	0.250000	0.250000	0.250000
1	0.250000	0.143750	0.568750	0.037500
2	0.520938	0.143750	0.297813	0.037500
3	0.290641	0.258898	0.412961	0.037500
5	0.388517	0.202620	0.371364	0.037500
10	0.375054	0.194937	0.392509	0.037500
20	0.372531	0.195814	0.394155	0.037500
∞	0.372527	0.195824	0.394149	0.037500

Table 1: Power iteration for the example, with $\alpha = 0.85$ and uniform initialization. Values are rounded; the last row gives the stationary distribution numerically.

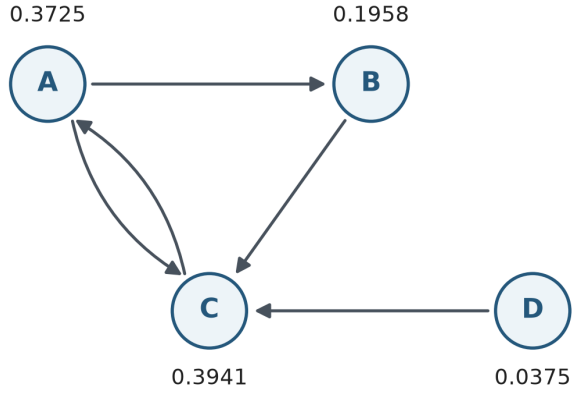


Figure 27: The same web graph with its PageRank scores for $\alpha = 0.85$.

Pause and Think – Programming Exercise

Implement Algorithm 20 using adjacency lists. Reproduce Table 1, then remove the edge (D, C) to create a dangling page. Check that every iterate still sums to one. What happens if you forget to distribute the dangling probability? Finally, compare the results for $\alpha = 0.5$ and $\alpha = 0.95$.

8.6 Other Meanings of Centrality

PageRank assigns importance according to a particular flow of recommendations. But what if our vertices represent people, routers, or locations instead of web pages? A useful centrality measure must reflect the question we want to answer. We now compare local connectivity, short distances, mediation along shortest paths, and connections to important neighbors. None is a universal definition of importance; its assumptions and computational cost must fit the application.

8.6.1 Degree Centrality

The simplest choice is the degree that we already know. For a simple undirected graph with $n > 1$, the normalized *degree centrality* is

$$C_D(v) = \frac{\deg(v)}{n-1}.$$

The denominator is the maximum possible degree. For a directed graph without self-loops, we can similarly define in-degree and out-degree centrality using $\text{indeg}(v)/(n-1)$ and $\text{outdeg}(v)/(n-1)$. The exclusion of self-loops matters for these upper bounds.

Degree centrality is local: it only looks at immediate neighbors. In a contact network it describes the number of direct contacts; in a web graph in-degree counts incoming links, while out-degree counts outgoing links. It is cheap to compute for all vertices in $O(|V| + |E|)$ time from adjacency lists. However, our PageRank example already showed that equal in-degree need not imply equal importance under another model.

8.6.2 Closeness Centrality

Perhaps an important vertex is one that others can reach quickly. Let $d(u, v)$ denote the shortest-path distance from u to v, as studied in Section 5. For a connected, unweighted, undirected graph with $n > 1$, define

$$C_C(v) = \frac{n-1}{\sum_{u \in V \setminus \{v\}} d(u, v)}.$$

This normalized *closeness centrality* (dt. Nähezentralität) is the reciprocal of the average distance to all other vertices. A larger score means shorter distances. The unnormalized version omits the numerator $n - 1$ and gives the same ranking within a fixed graph.

For directed graphs we must choose a direction: incoming closeness uses $d(u, v)$, as above, while outgoing closeness uses $d(v, u)$. A location that is easy for others to reach need not be a location from which others are easy to reach. Incoming closeness might model accessibility of a service, and outgoing closeness might model how quickly a sender can reach recipients.

If vertices cannot reach each other, some distances are infinite and the formula needs a convention. Simply omitting unreachable vertices can misleadingly reward tiny components. One alternative is *harmonic centrality*,

$$C_H(v) = \frac{1}{n-1} \sum_{u \neq v} \frac{1}{d(u, v)}, \qquad 1/\infty := 0.$$

Unreachable vertices then contribute zero. This is a different measure, not just another way to evaluate the same formula. On weighted graphs, distances must describe meaningful travel costs, with positive distances between distinct vertices for these reciprocals to be defined.

8.6.3 Betweenness Centrality

A vertex may have few neighbors but connect otherwise separated parts of a network. Figure 28 shows two triangles connected through D. Vertex D has degree 2, whereas C and E have degree 3. Still, all paths between the two triangles must pass through D.

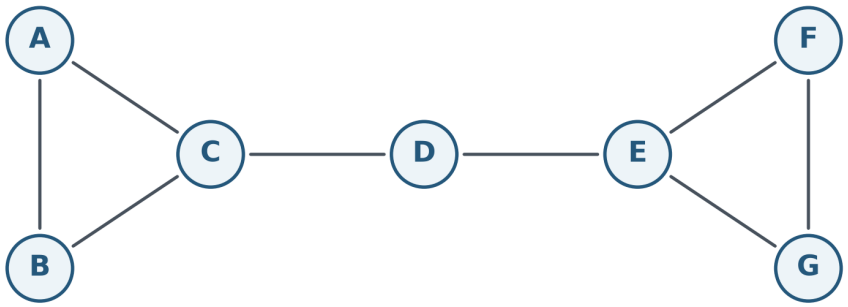


Figure 28: Two groups connected through a vertex of low degree. The interpretation of importance depends on the centrality measure.

Definition 8.8. Let σ_{st} be the number of shortest paths from s to t, and let $\sigma_{st}(v)$ be the number of those paths containing v as an internal vertex. The unnormalized *betweenness centrality* of v in

an undirected graph is

$$C_B(v) = \sum_{\{s,t\} \in [V \setminus \{v\}]^2} \frac{\sigma_{st}(v)}{\sigma_{st}}.$$

Pairs with no path contribute zero. Each unordered pair is counted once, and endpoints do not count as internal vertices.

For $n > 2$, dividing by $\binom{n-1}{2}$ normalizes this measure to $[0, 1]$. In a directed graph, we instead sum over ordered pairs of distinct endpoints and normalize by $(n-1)(n-2)$. If two equally short paths exist and only one uses v, their pair contributes $1/2$, not 1.

In Figure 28, there are $3 \cdot 3 = 9$ pairs with one endpoint in each triangle. Each has a unique shortest path through D, so $C_B(D) = 9$ and its normalized value is $9/15 = 0.6$. By comparison, $C_B(C) = C_B(E) = 8$, and the remaining vertices have betweenness zero. Also, the distances from D to the other vertices sum to 10, giving $C_C(D) = 6/10$, while those from C sum to 11, giving $C_C(C) = 6/11$. The vertex of smaller degree is both closer on average and on more shortest paths.

Betweenness can identify intermediaries between social groups or possible bottlenecks in a communication network. But it assumes that shortest paths are relevant to the process being modeled. If traffic follows different routes, a high betweenness score need not predict actual load.

Pause and Think – Work It Out

Verify the value $C_B(C) = 8$ by separating the graph after removing C. Then add the edge $\{B, F\}$ to the original graph. Before calculating anything, predict which vertices will lose betweenness. Does their degree also have to decrease?

Repeated BFS computes all unweighted distances in $O(|V|(|V| + |E|))$ time. Betweenness additionally needs shortest-path counts and contributions; one predecessor per vertex loses alternative shortest paths. Brandes' algorithm computes betweenness in $O(|V||E|)$ time for connected unweighted graphs [5]. This cost matters when choosing a measure for a large network.

8.6.4 Eigenvector Centrality

Finally, return to the idea that a connection to an important vertex should count more. For a connected undirected graph with at least two vertices and adjacency matrix A, *eigenvector centrality* assigns a nonnegative score vector x satisfying

$$\lambda x_v = \sum_{u \in N(v)} x_u, \qquad Ax = \lambda x.$$

Choose the largest eigenvalue and normalize, for example by $\sum_v x_v = 1$. Perron–Frobenius theory guarantees a unique strictly positive normalized vector under these assumptions. An arbitrary eigenvector may have negative entries and would not serve this purpose. On bipartite graphs, naive power iteration can still oscillate: existence and convergence are separate questions.

For directed graphs, incoming eigenvector centrality sums over $Pre(v)$, using A^T when rows represent sources. Disconnected or reducible graphs need not have unique, strictly positive scores.

Plain eigenvector centrality passes a neighbor's whole score along each edge. PageRank divides it among outgoing links and adds teleportation. Thus, being connected to important people and receiving a large share of circulating attention can yield different rankings.

Pause and Think – Quick Check

You are asked to find an "important" person in a social network. Would you choose somebody with many direct contacts, somebody at short distance from everyone, somebody connecting different groups, or somebody connected to other important people? State an application for which each answer would make sense.

9 Flow Networks

How much can we transport through a network? Imagine that a company sends goods from a factory to a distribution center. Several routes are available, but each road has a limited capacity. Sending everything along a shortest path may not work: the shortest route might be able to carry only a small part of the shipment. We now want to use several routes at the same time.

Flow problems became an important topic in operations research in the 1950s. Ford and Fulkerson published their work on maximum flow in 1956, motivated by questions about transportation through networks.[4] Today, the same model can describe communication bandwidth, transportation, or the assignment of tasks to people. The last application may seem surprising. What does assigning a person to a task have to do with transporting something? We will find out by building the model.

Background

Think of a flow as a rate, for instance units per hour, in a steady state. An intermediate vertex does not permanently accumulate goods. Whatever enters must leave again. The pipe analogy is useful for capacities and conservation, but our model does not describe water pressure, travel times, or temporary storage.

9.1 Maximum Flow Problem

9.1.1 From a Transportation Network to a Flow

We already know directed graphs and weight functions. Here the weight of an edge describes its *capacity* (dt. Kapazität), the largest amount of flow that the edge can carry.

Definition 9.1. A *flow network* is a 4-tuple $N = (G, \omega, s, t)$, where $G = (V, E)$ is a finite directed graph, $\omega : E \to \mathbb{R}_{\geq 0}$ assigns a finite capacity to each edge, and $s, t \in V$ with $s \neq t$ are the *source* (dt. Quelle) and the *sink* (dt. Senke).

Notice the domain of ω: it is E, because the restrictions belong to the edges. We write $\omega(u, v)$ instead of $\omega((u, v))$ for the capacity of the edge (u, v). The vertices s and t are distinguished vertices in the graph, not additional objects outside it.

Before defining a flow, we make one simplification to our graphs. A directed graph may contain both (u, v) and (v, u). Later we will introduce a reverse edge with a special meaning. If an original reverse edge were already present, we would have to distinguish two different reasons for using the same direction. We can avoid this distraction for now.

Definition 9.2. An *oriented graph* is a directed graph $G = (V, E)$ with no self-loops and no pair of oppositely directed edges. Formally,

$$(v, v) \notin E \quad \text{for all } v \in V, \qquad (u, v) \in E \implies (v, u) \notin E.$$

Equivalently, it can be obtained from a simple undirected graph by choosing one direction for each edge.

An oriented graph may still contain directed cycles! For the following definitions, our original network graph is oriented. For maximum flow we also assume that no edge enters s and no edge leaves t. These assumptions keep the notation small. Opposite original edges can be handled by replacing one of them by a two-edge route through a fresh vertex, with the same capacity on both edges.

Pause and Think – Quick Check

Is the directed cycle $a \to b \to c \to a$ an oriented graph? What changes if we add the edge $b \to a$?

[4] L. R. Ford, Jr. and D. R. Fulkerson, *Maximal Flow Through a Network*, Canadian Journal of Mathematics 8 (1956), pp. 399–404. https://doi.org/10.4153/CJM-1956-045-5

The network describes what is possible. A flow describes one particular choice of how much to send along each edge. Thus, capacities are input data, while flow values are decisions.

Definition 9.3. A *feasible flow* (dt. zulässiger Fluss) in N is a function $f : E \to \mathbb{R}_{\geq 0}$ satisfying the following two conditions:

$$0 \leq f(u,v) \leq \omega(u,v) \qquad \text{for every } (u,v) \in E, \tag{9}$$

$$\sum_{u \in Pre(v)} f(u,v) = \sum_{w \in Suc(v)} f(v,w) \qquad \text{for every } v \in V \setminus \{s,t\}. \tag{10}$$

The first condition is the *capacity constraint*. The second is *flow conservation* (dt. Flusserhaltung): at every intermediate vertex, incoming flow equals outgoing flow.

Recall from Section 1.2 that $Pre(v)$ and $Suc(v)$ contain the predecessors and successors of v. The sums therefore include only existing edges. An empty sum has value zero.

Definition 9.4. The *value* of a flow is the net amount leaving the source:

$$|f| = \sum_{v \in Suc(s)} f(s,v) - \sum_{u \in Pre(s)} f(u,s).$$

Under our assumption that no edge enters s, this is simply the first sum. The *maximum flow problem* asks for a feasible flow with the largest possible value.

The notation $|f|$ does not mean that we count edges or add all edge flows. Sending one unit through three edges still delivers only one unit. Conservation implies that the value also equals the net amount entering t: when we sum the balances over all vertices, each edge appears once as outgoing and once as incoming and cancels.

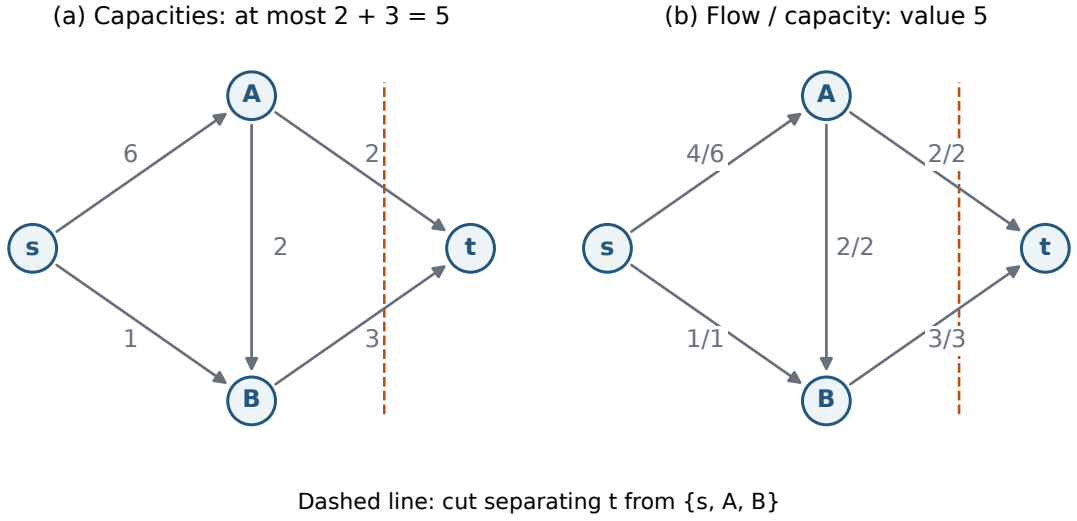


Figure 29: The network from the introductory flow example in the slides. Capacities leaving s sum to 7, but the sink can receive at most 5. The right-hand drawing gives a feasible flow of value 5; labels there mean flow/capacity.

Consider Figure 29. We can send four units from s to A, of which two go directly to t and two continue through B. Another unit goes directly from s to B, so three units leave B for t. Check conservation at both intermediate vertices. We deliver five units, even though the sum of the flow values on all five edges is twelve.

9.1.2 Cuts Give an Upper Bound

How do we know that five units are the best we can do? Every unit must eventually enter t. The two edges entering t have total capacity $2+3=5$. No choice of routes can overcome that limitation. We can generalize this observation by placing an imaginary boundary anywhere between s and t.

Definition 9.5. An *s-t cut* is a partition $(S, V \setminus S)$ of the vertices with $s \in S$ and $t \notin S$. Its *capacity* is

$$\omega(S, V \setminus S) = \sum_{\substack{(u,v)\in E \\ u\in S,\ v\notin S}} \omega(u,v).$$

Only edges directed from the source side to the sink side contribute to this capacity.

A cut does not require either side to be connected. We partition vertices; the edges crossing between the parts then follow from that choice. Summing conservation over the vertices in S except the source gives

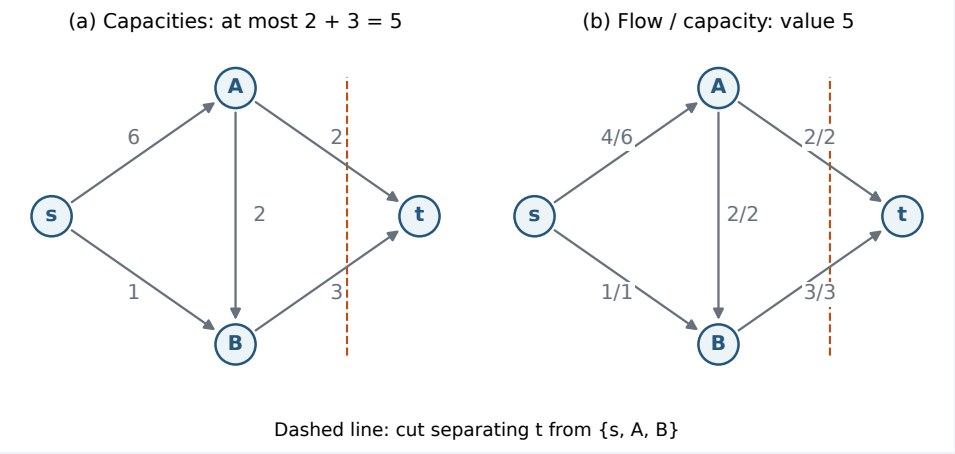
$$|f| = \sum_{\substack{(u,v)\in E \\ u\in S,\ v\notin S}} f(u,v) - \sum_{\substack{(u,v)\in E \\ u\notin S,\ v\in S}} f(u,v) \ \le\ \omega(S, V \setminus S). \tag{11}$$

All flow internal to S cancels. Flow crossing back into S must be subtracted, because it does not contribute to the net delivery. Thus, *every cut capacity is an upper bound on every feasible flow value.*

In Figure 29, choose $S = \{s, A, B\}$. Its capacity is five, and we have found a flow of value five. Together these two objects certify optimality. A flow alone gives a lower bound on what is achievable; a cut gives an upper bound. When they agree, there is nothing left to improve.

Pause and Think – Work It Out

For the same network, calculate the capacities of the cuts with source sides $\{s\}$, $\{s, A\}$, and $\{s, B\}$. Which of them also certifies that the displayed flow is maximum?



9.1.3 The Residual Graph: What Can We Still Change?

We could try to construct a flow by finding an s-t path, sending as much as possible along it, and repeating. But what if a path uses an edge that we later need for another route? A useful algorithm must be allowed to revise its earlier decisions.

Take one original edge (u, v) with capacity five, currently carrying two units. There are exactly two possible changes: increase its flow by at most three, or decrease its flow by at most two. We represent the first change by an arrow $u \to v$ and the second by an arrow $v \to u$.



Figure 30: One original edge gives two possible adjustments. A forward residual edge permits adding flow; a backward residual edge permits cancelling flow. Residual labels give the maximum allowed adjustment.

Definition 9.6. For a feasible flow f in an oriented network, its *residual graph* (dt. Residualgraph, Restnetz) is $G_f = (V, E_f)$, where

$$E_f = \{(u, v) \mid (u, v) \in E,\ f(u, v) < \omega(u, v)\} \\ \cup \{(v, u) \mid (u, v) \in E,\ f(u, v) > 0\}.$$

Its *residual capacity* $r_f : E_f \to \mathbb{R}_{>0}$ is defined by

$$r_f(u, v) = \begin{cases} \omega(u, v) - f(u, v), & \text{if } (u, v) \in E, \\ f(v, u), & \text{if } (v, u) \in E. \end{cases}$$

We call these the *forward* and *backward* cases, respectively. Edges with zero residual capacity are omitted.

The two cases are unambiguous because the original graph has no opposite edges. The residual graph, however, can contain both directions and is therefore not necessarily oriented. We use a different graph for a different question: G describes transportation possibilities, while G_f describes feasible adjustments to the current flow.

Notation

We store f only on original edges, and every stored value remains nonnegative. A backward residual edge is not a new transportation link and does not carry a negative physical flow. Traversing $v \to u$ in the residual graph tells us to *subtract* from the stored value $f(u, v)$.

Pause and Think – Quick Check

For an edge of capacity five, draw the residual edges when its flow is zero, two, and five. In which case do both residual directions exist?

Now suppose that we find a directed s-t path P in G_f. We can combine all its adjustments, provided that none exceeds its residual capacity.

Definition 9.7. An *augmenting path* is a directed s-t path in the residual graph. As before, a path does not repeat vertices. Its *bottleneck capacity* is

$$\Delta = \min_{(u,v) \text{ on } P} r_f(u, v).$$

To *augment* along P, for each residual edge (u, v) on the path we update the corresponding original edge:

$$\begin{cases} f(u, v) \leftarrow f(u, v) + \Delta, & \text{if } (u, v) \in E, \\ f(v, u) \leftarrow f(v, u) - \Delta, & \text{if } (v, u) \in E. \end{cases}$$

All other original edge flows remain unchanged.

Why does this preserve conservation? A residual step $u \to v$ increases the net outgoing flow at u by Δ and decreases it at v by Δ, whether we add a forward flow or cancel a backward one. At an intermediate path vertex, the incoming and outgoing adjustments cancel. Only s gains net outgoing flow and t gains net incoming flow. Thus, the new flow is feasible and its value increases by Δ. The minimum in the definition of Δ guarantees the capacity bounds, including nonnegativity when we subtract.

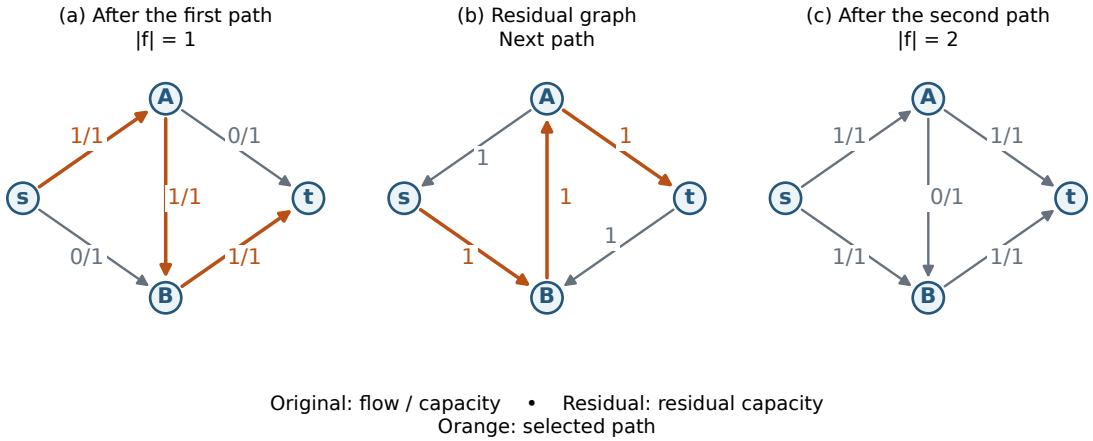


Figure 31: All original capacities are one. After choosing s, A, B, t, no forward-only augmentation is possible. The residual path s, B, A, t cancels $A \to B$ and produces a flow of value two.

Figure 31 gives a small example where cancellation is essential. First send one unit along (s, A, B, t). The edges $s \to A$ and $B \to t$ are now full. An algorithm considering only forward edges gets stuck at value one. But the residual path (s, B, A, t) is available. Its middle step cancels the unit on $A \to B$, while its other steps add one unit on $s \to B$ and $A \to t$. We obtain one unit on each of the routes (s, A, t) and (s, B, t).

Nothing has to travel physically backwards. We are computing a plan and replacing an earlier plan by a better one. This interpretation will also help us understand the residual graph when we use flows to represent assignments.

9.1.4 Ford-Fulkerson Algorithm

We now have all the ingredients. Start with the zero flow, find an augmenting path, and apply its adjustments. Repeat until no augmenting path exists. Algorithm 21 describes the Ford-Fulkerson method. It deliberately leaves open *which* path to choose.

We have argued that every augmentation is feasible and increases the value. But a locally unimprovable solution is not automatically globally optimal. Why is the absence of an augmenting path enough here?

Theorem 9.8 (Max-Flow Min-Cut)**.** *The maximum value of a feasible flow equals the minimum capacity of an s-t cut. In particular, a feasible flow is maximum if and only if its residual graph contains no s-t path.*

Proof. If an augmenting path exists, we can increase the value, so the current flow is not maximum. Otherwise, let S be all vertices reachable from s in G_f. Since t is not reachable, $(S, V \setminus S)$ is an s-t cut.

Every original edge from S to $V \setminus S$ must be saturated. If it had spare capacity, its forward residual edge would make its endpoint reachable. Every original edge from $V \setminus S$ to S must carry zero flow. If it carried positive flow, its backward residual edge would again lead from S to an allegedly unreachable vertex.

Substitute these facts in Equation (11): the outgoing terms equal their capacities and the incoming terms are zero. Hence $|f| = \omega(S, V \setminus S)$. No feasible flow can exceed any cut capacity, so

Algorithm 21 Ford-Fulkerson Method

Input: An oriented network $N = (G, \omega, s, t)$ with integral capacities
Output: A maximum flow f

$f(e) \leftarrow 0$ for every $e \in E$
Construct G_f and r_f
while there exists an s-t path P in G_f **do**
    $\Delta \leftarrow \min_{(u,v) \text{ on } P} r_f(u, v)$
    for each residual edge (u, v) on P **do**
        if $(u, v) \in E$ **then**
            $f(u, v) \leftarrow f(u, v) + \Delta$
        else
            $f(v, u) \leftarrow f(v, u) - \Delta$
        end if
    end for
    Update G_f and r_f using the new flow
end while
return f

this flow is maximum and this cut is minimum. Applying the argument to a maximum flow proves the equality. □

The proof also gives us a practical result: one final graph search produces an optimality certificate. The source side consists of the reachable vertices, even if it looks quite different from a vertical line through the drawing. For integer capacities, the following argument also guarantees that the algorithm reaches such a flow.[5]

Theorem 9.9 (Integrality)**.** *If every capacity is an integer, Ford-Fulkerson terminates and produces an integral maximum flow.*

Proof. Initially all flows are zero. If all current flows are integers, then all residual capacities are integers. Their positive minimum Δ is an integer, so augmenting preserves integrality and increases the value by at least one. The value is bounded above by the finite total capacity leaving s. Thus only finitely many augmentations are possible, and the terminal flow is maximum by Theorem 9.8. □

This is an existence statement as well: integral capacities guarantee that *an* integral optimum exists. They do not say that every feasible flow, or every optimum, must be integral.

If F is the integral maximum flow value, there are at most F augmentations. A search and residual update using adjacency lists takes $O(|V|+|E|)$ time, giving $O((|V|+|E|)(F+1))$ including the final search. With isolated vertices removed, the usual shorthand is $O(|E|F)$ for $F \geq 1$. The dependence on a numerical value matters: representing a large capacity in binary takes far fewer than that many bits. This is a *pseudo-polynomial* bound, not a polynomial bound in the input length.

[5]For real capacities, existence of a maximum follows from a fact from analysis: a continuous function attains a maximum on a nonempty compact set. The feasible flows form a closed and bounded subset of the finite-dimensional box $\prod_{e \in E}[0, \omega(e)]$.

9.1.5 Why the Path Choice Matters

Consider the network with capacities

$$\omega(s, A) = \omega(A, t) = \omega(s, B) = \omega(B, t) = 100, \qquad \omega(A, B) = 1.$$

The maximum flow is clearly 200: send 100 along each of (s, A, t) and (s, B, t). But suppose our path finder first chooses (s, A, B, t) and then (s, B, A, t) in the residual graph.

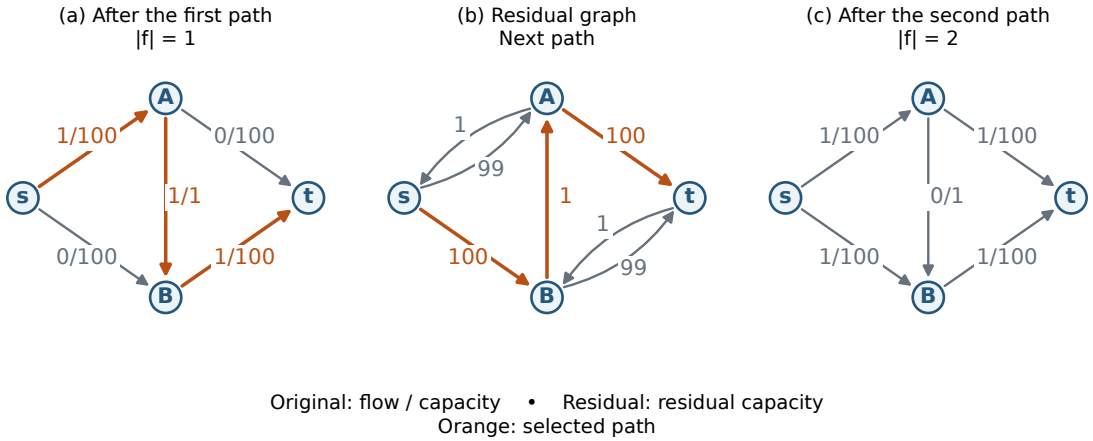


Figure 32: A legal but unfortunate path choice. Alternately using and cancelling the middle edge increases the flow by only one unit per augmentation. Large outer capacities do not help this choice of paths.

Step	Residual path	$\lvert f\rvert$	$f(s, A)$	$f(A, t)$	$f(s, B)$	$f(B, t)$	$f(A, B)$
0	–	0	0	0	0	0	0
1	(s, A, B, t)	1	1	0	0	1	1
2	(s, B, A, t)	2	1	1	1	1	0
3	(s, A, B, t)	3	2	1	1	2	1
4	(s, B, A, t)	4	2	2	2	2	0

After $2k$ augmentations, all four outer edges carry k units and the middle edge carries zero. After 200 augmentations we finally reach the maximum. Replacing 100 by a larger integer makes the example arbitrarily slow while leaving its five-edge structure unchanged.

Background

Ford-Fulkerson does not return a wrong answer on this example; it makes poor progress. Rational capacities also give termination, since we can scale them to integers. With irrational capacities, some networks and path choices can cause infinitely many augmentations, and the limiting value need not be maximum. That is a separate failure mode, not what happens in the integer example above.

9.1.6 Edmonds-Karp Algorithm

How could we prevent this behavior without inventing a completely different method? Edmonds-Karp uses Ford-Fulkerson with one precise rule: always choose an augmenting path with the *fewest edges*. A breadth-first search (BFS) in the current residual graph finds such a path. Residual capacities decide whether an edge is available and how much we can augment; they are not distances for the BFS.

On the same 100–1 network, BFS chooses (s, A, t) or (s, B, t): each has two edges, while the paths using the middle edge have three. The first augmentation sends 100 units. The second sends 100 on the other two-edge path. Two augmentations solve exactly the instance for which the alternating choices needed 200.

Algorithm 22 Edmonds-Karp Algorithm

Input: An oriented network $N = (G, \omega, s, t)$
Output: A maximum flow f

```
f(e) ← 0 for every e ∈ E
Construct G_f and r_f
while BFS from s reaches t in G_f do
    Reconstruct a shortest s-t path P from the BFS predecessors
    Δ ← min_{(u,v) on P} r_f(u, v)
    Augment f along P as in Algorithm 21
    Update G_f and r_f
end while
return f
```

Theorem 9.10. *Edmonds-Karp performs $O(|V||E|)$ augmentations and has running time $O(|V||E|^2)$ after removing isolated vertices. The bound does not depend on the capacity values.*

Proof. We sketch the argument. Let $d_f(v)$ be the BFS distance from s to v, with infinity for unreachable vertices. After an augmentation along a shortest path, these distances never decrease. Existing residual edges already satisfy $d_f(v) \leq d_f(u) + 1$ for every reachable tail u. A newly appearing reverse edge points backwards along the chosen path, from a vertex at level $i+1$ to one at level i, and satisfies that inequality too. Hence any new residual path to a vertex has at least its old distance.

Each augmentation saturates at least one residual edge (u, v) on its shortest path, where $d_f(v) = d_f(u)+1$. Before the same residual direction can be saturated again, some later augmentation must use (v, u) to restore its capacity. At that later time, shortest-path levels satisfy $d(v) + 1 = d(u)$. Since distances have not decreased, u must then be at least two levels farther from s than at the previous saturation of (u, v).

Finite BFS distances are at most $|V| - 1$. Each residual direction can therefore be saturated only $O(|V|)$ times, and there are at most $2|E|$ directions. Charging each augmentation to one saturated edge gives $O(|V||E|)$ augmentations. Each BFS and update takes $O(|V| + |E|)$ time, or $O(|E|)$ after removing isolated vertices, yielding the claimed bound. □

Even with real capacities, the number of augmentations is bounded in this way, assuming exact arithmetic. In software, floating-point rounding can create tiny apparent residual capacities; this is a numerical issue that the mathematical proof does not address.

Implementation Note

A useful adjacency-list representation stores two linked records for each original edge: one forward record with initial residual capacity $\omega(u, v)$ and one reverse record with initial capacity zero. Each record stores the index of its partner. Augmenting subtracts Δ from the selected record and adds Δ to its partner. BFS follows only positive residual capacities and stores edge records as predecessors. With this representation, opposite original edges can also be kept as separate pairs of records.

Pause and Think – Programming Exercise

Implement both arbitrary-path Ford-Fulkerson and Edmonds-Karp. Reproduce the alternating choices on the 100–1 network, then increase the four outer capacities. Count augmentations and compare the running times. At the end, verify capacity constraints and conservation, and use a final residual search to compute a cut whose capacity equals the returned flow value.

9.2 Bipartite Matching

Suppose we want to assign students to projects. Each student can work on at most one project, and each project has room for one student. Some assignments are possible, others are not: perhaps

a project requires a particular skill. How many students can we assign? At first, this does not look like a question about transporting something through a network. Let us see how it becomes one.

Definition 9.11. An undirected graph $G = (V, E)$ is *bipartite* if V can be partitioned into two disjoint sets A and B such that every edge has one endpoint in each set. A *matching* is a set $M \subseteq E$ in which no two edges share an endpoint. A matching is *maximum* if its size is as large as possible. It is *perfect* if every vertex belongs to one of its edges.

For our example, A contains the students, B contains the projects, and $\{a, b\} \in E$ means that assigning student a to project b is possible. A matching describes a set of assignments that do not conflict with each other.

Background

Do not confuse *maximum* with *maximal*. A maximal matching cannot be extended by simply adding another edge. A maximum matching has the largest possible number of edges, even if we would have to replace earlier choices to obtain it.

Consider $A = \{a_1, a_2\}$, $B = \{b_1, b_2\}$, and the three edges $\{a_1, b_1\}$, $\{a_1, b_2\}$, and $\{a_2, b_1\}$. If we select $\{a_1, b_1\}$ first, neither of the remaining edges can be added. We have a maximal matching of size one. But $\{\{a_1, b_2\}, \{a_2, b_1\}\}$ is a matching of size two! We have to undo the first assignment to obtain the better solution. Does this sound familiar?

9.2.1 From Matching to Maximum Flow

We construct a network by adding two new vertices s and t. Direct each original edge from A to B, add an edge from s to every vertex in A, and add an edge from every vertex in B to t. Give *every* edge capacity one. More formally, the new directed graph has

$$V' = A \cup B \cup \{s, t\}, \qquad E' = \{(s, a) \mid a \in A\} \cup \{(a, b) \mid \{a, b\} \in E\} \cup \{(b, t) \mid b \in B\}.$$

An integral unit of flow takes a path $s \to a \to b \to t$ and represents the assignment of a to b. Why can we not assign a student twice? The edge (s, a) has capacity one. Similarly, the edge (b, t) prevents a project from receiving two students.

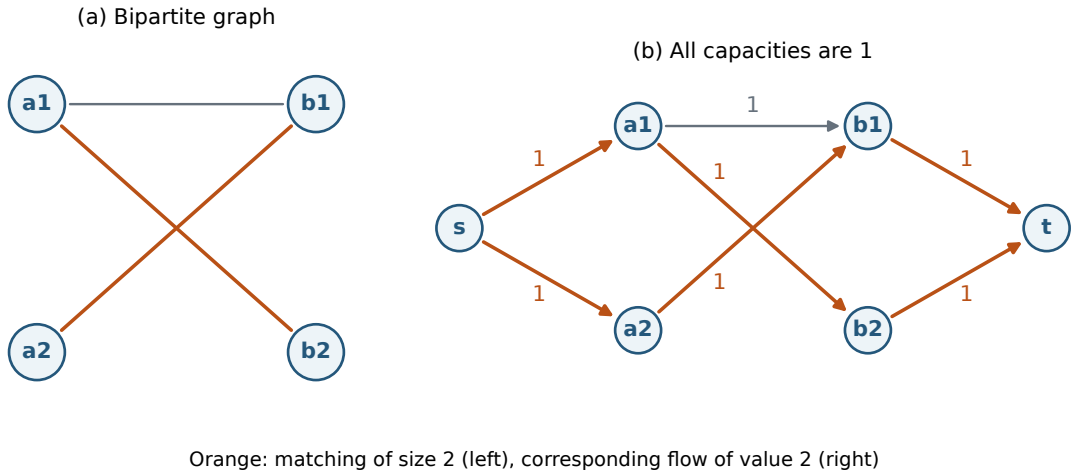


Figure 33: A bipartite graph and its flow network. Orange edges show a matching of size two and the corresponding flow. Every network edge has capacity one; the crossing of two drawn edges is not a vertex.

Theorem 9.12. *The size of a maximum matching in a bipartite graph equals the value of a maximum flow in the network constructed above.*

Proof. Given a matching of size k, send one unit along $s \to a \to b \to t$ for each matched pair. Since no endpoints are shared, no capacity is exceeded, and conservation holds at every student and project. This is a feasible flow of value k.

Conversely, choose an integral maximum flow, which exists because all capacities are integers. Select exactly those original edges whose corresponding directed edge carries one unit. Conservation and the capacity-one edges at the source and sink ensure that each vertex is selected at most once. The selected edges therefore form a matching, and their number is the value of the flow. The two constructions establish the claim. □

In our small example, the initial assignment a_1 to b_1 sends flow along $s \to a_1 \to b_1 \to t$. The residual graph then contains the path

$$s \to a_2 \to b_1 \to a_1 \to b_2 \to t.$$

The backward residual edge $b_1 \to a_1$ cancels the old assignment. Augmenting along this path assigns a_2 to b_1 and a_1 to b_2. Thus, the residual graph expresses a useful operation here: *reassign somebody to make room for another assignment.*

Pause and Think – Work It Out

Suppose a project can accept three students, but each student may still join only one project. Which capacities would you change? Why is changing only the edges between students and projects insufficient?

9.2.2 Stable Marriage Problem

So far, every possible assignment has been equally good. What changes if students prefer some projects over others, and projects also have preferences among students? Assigning everybody does not necessarily produce a satisfactory result. Two participants might prefer each other to their assigned partners and have a reason to leave their assignments.

The classical *stable marriage problem* considers two disjoint sets A and B with $|A| = |B| = n$. Every participant gives a strict ranking of *all* participants on the other side. Thus, there are no ties, every possible partner is acceptable, and we seek a perfect matching. These assumptions matter for the guarantees below.

Notation

We write $b \succ_a b'$ when participant a prefers b to b'. We describe a perfect matching by a bijection $\mu : A \to B$. Thus, $\mu(a)$ is the partner of a, and $\mu^{-1}(b)$ is the partner of b.

Definition 9.13. A pair $(a, b) \in A \times B$ is a *blocking pair* for a matching μ if

$$b \succ_a \mu(a) \quad \text{and} \quad a \succ_b \mu^{-1}(b).$$

A matching is *stable* if it has no blocking pair.

Both participants have to prefer the change. One participant wishing for a better partner is not enough to make the matching unstable. For example, suppose Alice and Carol both prefer Bob to Dave, while Bob and Dave both prefer Alice to Carol. Matching Alice with Dave and Carol with Bob is unstable: Alice and Bob form a blocking pair. Matching Alice with Bob and Carol with Dave is stable, although Carol and Dave both receive their second choice.

Pause and Think – Quick Check

Why does a maximum matching algorithm alone not solve the stable marriage problem? Which information is absent from the capacity-one network?

The Gale–Shapley algorithm. David Gale and Lloyd Shapley introduced the procedure in 1962.[6] The idea is to allow tentative assignments. One side proposes, and the other side keeps its best offer so far. A tentative partner can be replaced if a better proposal arrives. A rejected participant continues down their preference list. Let A be the proposing side.

Algorithm 23 Gale–Shapley with A Proposing

Input: Disjoint sets A, B of equal size; complete, strict preference lists
Output: A stable perfect matching M

```
M ← ∅; mark every participant free
For each a ∈ A, place a pointer at the start of its preference list
while some a ∈ A is free do
    b ← the next participant on a's list; advance a's pointer
    if b is free then
        Add (a, b) to M; mark a and b engaged
    else if b prefers a to its current partner a′ then
        Replace (a′, b) in M by (a, b)
        Mark a′ free and a engaged
    else
        b rejects a; a remains free
    end if
end while
return M
```

Notice the asymmetry. Proposers move down their lists, while receivers can only improve their tentative partners. A proposal is never repeated. There are n^2 possible proposals, so the algorithm terminates after at most n^2 iterations. A free proposer cannot have exhausted its list: then all n receivers would have received a proposal, and each would still hold one. This would engage all n proposers, contradicting that one is free. Therefore, termination produces a perfect matching.

Implementation Note

To obtain $\mathcal{O}(n^2)$ running time, store a pointer into each proposer's list, maintain a queue of free proposers, and prepare a rank array for each receiver. A receiver can then compare two proposers in constant time. Searching through a preference list for each comparison would give an unnecessary extra factor of n.

Theorem 9.14. *Gale–Shapley produces a stable matching.*

Proof. Suppose (a, b) were a blocking pair in the final matching. Since a prefers b to its final partner, a must have proposed to b earlier. Participant b either rejected a immediately or replaced a later. In either case, b then held a partner preferred to a. Since a receiver's partner only improves, b cannot prefer a to its final partner. This contradicts that (a, b) is a blocking pair. □

Who benefits from proposing? Consider the preferences in Table 2. Each row lists the other side from most to least preferred.

	First	Second	Third		First	Second	Third
a_1	b_1	b_3	b_2	b_1	a_3	a_2	a_1
a_2	b_2	b_1	b_3	b_2	a_1	a_3	a_2
a_3	b_2	b_3	b_1	b_3	a_2	a_1	a_3

Table 2: Complete, strict preferences for three participants on each side.

Choose the lowest-index free proposer at each step. With A proposing, we obtain the following trace. Held pairs are tentative until the algorithm ends.

[6] D. Gale and L. S. Shapley, *College Admissions and the Stability of Marriage*, American Mathematical Monthly 69(1), pp. 9–15. https://doi.org/10.1080/00029890.1962.11989827

Step	Proposal	Receiver's decision
1	$a_1 \to b_1$	Hold a_1.
2	$a_2 \to b_2$	Hold a_2.
3	$a_3 \to b_2$	Replace a_2 by a_3; a_2 becomes free.
4	$a_2 \to b_1$	Replace a_1 by a_2; a_1 becomes free.
5	$a_1 \to b_3$	Hold a_1.

The resulting matching is

$$M_A = \{(a_1, b_3), (a_2, b_1), (a_3, b_2)\}.$$

Now exchange the roles. Participant b_1 proposes to a_3, b_2 proposes to a_1, and b_3 proposes to a_2. These are three different receivers, so all proposals are held and the algorithm finishes immediately:

$$M_B = \{(a_1, b_2), (a_2, b_3), (a_3, b_1)\}.$$

Both matchings are stable. But the participants do not judge them equally. For comparison, let us award three points for a first choice, two for a second choice, and one for a third choice.

Proposing side	a_1	a_2	a_3	b_1	b_2	b_3	A total	B total	Total
A	2	2	3	2	2	2	7	6	13
B	1	1	1	3	3	3	3	9	12

Table 3: Preference points of the two Gale–Shapley outcomes.

Here, every participant is better off when their own side proposes. The difference is a property of the algorithm, not an accident of choosing the next free proposer.

Theorem 9.15 (Proposer optimality)**.** *Under complete, strict preferences, Gale–Shapley gives each proposer their most preferred partner among all stable matchings. Each receiver obtains their least preferred partner among all stable matchings.*

The qualification *among all stable matchings* is essential. Proposer optimality does not promise everybody their first choice. In M_A, for example, a_1 and a_2 receive their second choices, but neither can receive their first choice in any stable matching.

Proof. The key observation is that a rejected pair cannot occur in a stable matching. Suppose otherwise, and consider the first rejection of a pair (a, b) that belongs to some stable matching M. At this rejection, b holds a proposer a' preferred to a. Before proposing to b, participant a' was rejected by everybody it prefers to b. By our choice of the first rejection, none of these earlier rejected pairs can belong to a stable matching. Hence a' prefers b to its partner in M. Since b prefers a' to its partner a in M, the pair (a', b) blocks M, a contradiction.

A proposer has been rejected by everybody it prefers to its final partner. None of those rejected pairs can occur in a stable matching, proving proposer optimality. Finally, suppose a receiver b had a worse partner in another stable matching M. Its Gale–Shapley partner a also prefers b to its partner in M, by proposer optimality and strict preferences. The pair (a, b) would block M. Thus, receivers obtain their worst stable partners. □

Background

Stability does not mean fairness. It rules out mutually attractive deviations, but it does not balance the interests of the two sides or maximize total preference points. The points in Table 3 are an additional modeling choice: a ranking alone does not say that the difference between first and second choice is worth exactly one point. The table makes the asymmetry visible under a simple, explicitly chosen scoring rule.

Pause and Think – Work It Out

There are only $3! = 6$ perfect matchings in this example. List them and find a blocking pair for each matching other than M_A and M_B. Which stable matching maximizes the total points? Would you necessarily call that outcome fair?

If an application needs a different objective, we have to express it explicitly. Maximizing the sum of assignment scores is a weighted matching problem, which can be modeled using minimum-cost flow. Requiring stability *as well* needs additional constraints; the ordinary matching network does not enforce the absence of blocking pairs. This distinction between an efficient specialized algorithm and a more flexible model will accompany us in the next part of this chapter.

10 Minimum-Cost Flow

So far we have asked how much we can transport. Suppose that we already know how much must arrive, but different routes have different costs. A fast connection might be expensive, a cheap connection might have little capacity, and several shipments might compete for the same connection. Which feasible flow should we choose?

We can keep most of our formalism. The network $N = (G, \omega, s, t)$ still describes the connections and their capacities. We now add a cost function $c : E \to \mathbb{R}$ and a required amount $d \geq 0$. The number $c(u, v)$ is the cost of sending *one unit* along (u, v). It is not the cost of opening that connection, and it does not replace its capacity.

Definition 10.1. For a flow f, its *total cost* is

$$C(f) = \sum_{(u,v)\in E} c(u, v)f(u, v).$$

The *minimum-cost flow problem* asks for a feasible flow of value d that minimizes $C(f)$, or reports that no such flow exists.

The required value matters. If all costs are positive and we merely ask for a cheapest feasible flow, sending nothing is an excellent solution! In the *minimum-cost maximum-flow problem*, we first determine the maximum value d, and then minimize the cost among flows of that value. These are two objectives in a specified order: an inexpensive flow of smaller value does not qualify.

10.1 Supplies and Demands

Often there is more than one supplier and more than one customer. Rather than giving each vertex a special name, we can assign it a number. A positive number means that the vertex supplies goods, a negative number means that it consumes goods, and zero means that it only passes goods on.

Definition 10.2. A *balance function* $b : V \to \mathbb{R}$ specifies the required net outgoing flow at each vertex. A feasible flow satisfies

$$0 \leq f(u, v) \leq \omega(u, v) \qquad \text{for all } (u, v) \in E,$$

$$\sum_{w:(v,w)\in E} f(v, w) - \sum_{u:(u,v)\in E} f(u, v) = b(v) \qquad \text{for all } v \in V.$$

The minimum-cost flow problem with these balances minimizes $C(f)$ over all such flows.

Read the second equation as "what leaves minus what arrives". Thus, a factory with $b(v) = 6$ sends out six units more than it receives. A customer with $b(v) = -5$ receives five units more than it sends out. Our single-source problem is the special case $b(s) = d$, $b(t) = -d$, and $b(v) = 0$ elsewhere. If $b(v) = 0$ at *every* vertex, we call the flow a *circulation*.

Pause and Think – Quick Check

Sum the balance equations over all vertices. Why must $\sum_{v \in V} b(v) = 0$? Does this condition alone guarantee that a feasible flow exists?

Every edge contributes once positively and once negatively, so the total supply must equal the total demand. However, capacities or missing connections can still prevent a shipment. First check that the total balance is zero; otherwise report infeasibility. To find a feasible starting flow, introduce a new source s_0 and sink t_0. For each $b(v) > 0$, add (s_0, v) with capacity $b(v)$. For each $b(v) < 0$, add (v, t_0) with capacity $-b(v)$. Keep the original edges and ignore their costs for this step. Compute a maximum flow from s_0 to t_0. A feasible flow exists exactly when its value is $\sum_{v:b(v)>0} b(v)$. All edges from s_0 and into t_0 are then saturated. Removing the new vertices leaves a flow with precisely the required balances.

10.2 Residual Costs: Undoing a Decision Has a Price

The residual capacities have the same meaning as before: how much can we still add, and how much can we undo? Costs need to describe these two operations as well. Adding one unit to an original edge costs $c(u, v)$. Removing one unit from it saves exactly that amount. Consequently, a backward residual edge has the *negative* of the original cost.

For every original edge (u, v), the residual graph contains

residual edge	condition	capacity	cost
(u, v)	$f(u, v) < \omega(u, v)$	$\omega(u, v) - f(u, v)$	$c(u, v)$
(v, u)	$f(u, v) > 0$	$f(u, v)$	$-c(u, v)$.

We write c_f for these residual costs and r_f for the residual capacities. As before, the assumption that the original graph is oriented makes each residual edge unambiguous.

Pause and Think – Quick Check

An edge has capacity 8, cost 5, and currently carries 3 units. What residual edges exist, with which capacities and costs? What changes if we undo two units?

An augmenting s-t path changes the amount delivered. Here that amount is already fixed. Instead, we can change the flow along a *directed cycle* in the residual graph. At every vertex of the cycle, one residual edge enters and one leaves. The changes cancel in the balance equation, even where a residual edge represents undoing an original edge.

If Q is such a cycle, let

$$c_f(Q) = \sum_{(u,v) \in Q} c_f(u, v), \qquad \Delta = \min_{(u,v) \in Q} r_f(u, v).$$

Augmenting by Δ around Q changes the cost by $\Delta c_f(Q)$. Thus, a cycle with negative total cost improves the flow while preserving every supply and demand.

Consider Figure 34. Both routes have capacity 3, and we must deliver $d = 3$. Initially we send everything along $s \to A \to t$, at a total cost of $3 \cdot 4 = 12$. The residual cycle

$$s \to B \to t \to A \to s$$

has cost $1 + 0 + 0 - 4 = -3$ and residual bottleneck 3. We add three units on the cheap route and undo three units on the expensive route. The amount delivered stays 3, but its cost falls by $3 \cdot 3 = 9$, from 12 to 3.

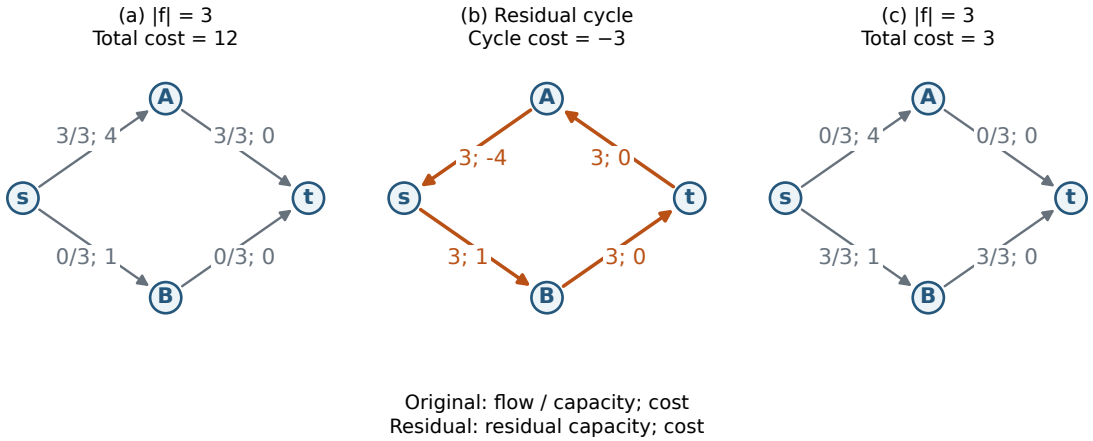


Figure 34: Improving a flow of fixed value by a negative residual cycle. The backward residual edges undo the expensive shipment.

A negative residual edge alone is not enough to improve the flow. We must combine changes so that the balance equations still hold. Likewise, the negative cycle does not represent goods physically travelling backwards around the network: it describes a revision of our shipment plan.

10.3 Klein's Cycle-Canceling Algorithm

We now have almost the same framework as Ford-Fulkerson. Start with a feasible flow, find an improvement in the residual graph, and augment by its bottleneck. For maximum flow, the improvement was a path that increased the flow value. For minimum cost, it is a cycle that decreases the cost.

Algorithm 24 Klein's cycle-canceling algorithm

Input: Oriented graph G, finite integral capacities ω, integral costs c and balances b
Output: A minimum-cost feasible flow, or a report of infeasibility

Check $\sum_v b(v) = 0$; if it fails, return infeasible
Find an integral feasible flow f using the auxiliary maximum-flow construction
if no feasible flow exists **then**
    return infeasible
end if
Construct the residual graph, capacities r_f, and costs c_f
while a directed cycle Q with $c_f(Q) < 0$ exists in the residual graph **do**
    $\Delta \leftarrow \min_{(u,v)\in Q} r_f(u, v)$
    Increase f by Δ on original edges used forward by Q
    Decrease f by Δ on original edges used backward by Q
    Update the residual graph, capacities, and costs
end while
return f

This method is associated with Morton Klein's 1967 paper, *A Primal Method for Minimal Cost Flows with Applications to the Assignment and Transportation Problems*, Management Science

14(3), 205–220.[7]

Theorem 10.3 (Negative-cycle optimality criterion)**.** *A feasible flow has minimum cost for its prescribed balances if and only if its residual graph has no directed cycle of negative total cost.*

Proof. If a negative residual cycle exists, augmenting around it gives a feasible flow of smaller cost. The current flow therefore cannot be optimal.

For the converse, suppose that a cheaper feasible flow g exists. Compare g with f on each original edge. Where g is larger, put the difference on the forward residual edge; where it is smaller, put the difference on the backward residual edge. The resulting residual flow respects the residual capacities. Since f and g have the same balances, their difference has zero balance at every vertex: it is a circulation.

A nonzero nonnegative circulation can be decomposed into directed cycles. To see why, follow edges carrying positive flow. Conservation prevents the walk from getting stuck, and finiteness forces a repeated vertex. This gives a cycle. Subtract its smallest edge flow, which removes at least one positive edge, and repeat. Hence the cost difference $C(g) - C(f)$ is a sum of cycle costs multiplied by positive amounts. Since that difference is negative, at least one of the cycles has negative cost. This contradicts the assumed absence of negative residual cycles. □

How do we find these cycles? We already have the main tool: Bellman-Moore-Ford. Residual costs may be negative, so ordinary Dijkstra is unsuitable here. We need to detect a cycle *anywhere* in the residual graph, including components unreachable from the original source. Initialize every distance to zero, as if an extra source had a zero-cost edge to every vertex. Relax all residual edges for $|V|$ rounds. If a distance changes in the last round, a negative cycle exists. Store predecessors; from a vertex updated in that round, follow predecessors $|V|$ times to enter a cycle, and then continue until a vertex repeats. The predecessor walk runs backwards, so reverse it to obtain the directed cycle. This costs $O(|V||E|)$ per search, since there are at most $2|E|$ residual edges.

Background

For integer capacities, integer balances, and integer costs, choose an integral feasible starting flow. Every augmentation remains integral and decreases the total cost by at least one. With $B = \sum_{e \in E} \omega(e)|c(e)|$, every feasible cost lies between $-B$ and B, so there are at most $2B$ cancellations. This is a *pseudo-polynomial* bound: B depends on numerical values, not just the number of bits needed to encode them. Rational data can be scaled to integers, but scaling may produce large numbers. This argument does not give a termination guarantee for arbitrary real data. More sophisticated minimum-cost flow algorithms offer stronger bounds; the basic cycle-canceling method is useful here because its improvement step is easy to understand.

Pause and Think – Work It Out

In the two-route example, reduce the capacity of (s, B) to 2, leaving the other capacities at 3 and the demand at 3. Start with the expensive shipment again. Find the negative residual cycle, perform one cancellation, and explain why the new cost is 6. Why can we no longer send all three units on the cheaper route?

10.4 A More General Model

Minimum-cost flow can express choices that maximum flow alone cannot distinguish. For example, after finding the largest possible bipartite matching, we may want a matching of that size with the smallest total assignment cost. Add a cost to each person–task edge and prescribe the previously computed matching size as the flow value. This optimizes the sum of those costs. If costs encode preference ranks, the resulting matching need not be stable: blocking pairs are a different requirement.

Shortest paths also fit this model. For a graph with nonnegative edge lengths and an s-t path, give each edge capacity one, use its length as the unit cost, and require one unit from s to t. An

[7] https://doi.org/10.1287/mnsc.14.3.205

integral optimum contains an s-t path; any additional cycles can be removed without increasing cost. The remaining path is a shortest one. Thus, several problems we have studied can be expressed using the same flow model.

We can also solve a maximum-flow problem using a minimum-cost flow algorithm, without first knowing the maximum value. Give every original edge cost zero. Add a return edge (t, s) of capacity

$$U = \sum_{v:(s,v)\in E} \omega(s, v)$$

and cost -1. Set every balance to zero, and compute a minimum-cost circulation. The zero flow is already feasible. Every unit on the return edge must be balanced by one net unit sent from s to t through the original network. The total cost is precisely minus the flow on the return edge. Minimizing it therefore maximizes the original flow value. Removing the return edge gives the desired maximum flow.

If a direct edge (s, t) would violate our oriented-graph convention after adding (t, s), use a fresh vertex r and the return path $t \to r \to s$ instead. Give both edges capacity U, one cost -1 and the other cost zero. Conservation at r forces the same amount through both. The same subdivision also works for a more general input containing a return edge already.

Pause and Think – Quick Check

Why would giving the return edge cost zero fail to encode the maximum-flow objective? What would a minimum-cost algorithm be allowed to return?

This is a *reduction*: we translate an instance and then translate its solution back. It explains in a precise sense why minimum-cost flow is a more general model. A specialized maximum-flow algorithm often uses less machinery and can be faster for that special case. But "more general" is not itself a theorem about running time. The chosen algorithms, representations, and instances still matter.

10.5 A Glimpse of Linear Programming

Look once more at the minimum-cost flow equations. There is one unknown $f(u, v)$ for every edge. The graph tells us which unknowns occur in each balance equation, but the equations themselves use only sums and multiplication by constants:

$$\begin{aligned}
\text{minimize} \quad & \sum_{(u,v)\in E} c(u, v) f(u, v) \\
\text{subject to} \quad & \sum_{w:(v,w)\in E} f(v, w) - \sum_{u:(u,v)\in E} f(u, v) = b(v) \quad \text{for all } v \in V, \\
& 0 \le f(u, v) \le \omega(u, v) \qquad\qquad\qquad\qquad \text{for all } (u, v) \in E.
\end{aligned}$$

The variables describe our decisions. The constraints describe which decisions are allowed. The objective compares the allowed decisions.

Definition 10.4. A *linear program* (LP) minimizes or maximizes a linear expression in real-valued decision variables, subject to linear equalities and inequalities.

Here, "linear" means that each variable is multiplied only by a constant and these terms are added. A term such as $4f(s, A)$ is linear; a product of two decision variables or a squared decision variable is not. Linear programming permits many constraints beyond graph balances. For example, a shared resource restriction $2x_1 + 5x_2 \le 40$ need not correspond to a network edge at all. Thus, minimum-cost flow is a special case of linear programming, which also models problems outside graph theory.

We will stop at this modeling connection; solving general linear programs would be a topic of its own. Polynomial-time algorithms for rational LPs exist. General solvers provide wider modeling freedom, while specialized flow algorithms can exploit the network structure. Requiring variables

to be integers gives *integer linear programming*, which is NP-hard in general. This is a meaningful change: fractional production quantities may be acceptable, but assigning half a person to each of two indivisible jobs may not be.

Background

Network flow has a special integrality property. With integer capacities and integer balances, a feasible minimum-cost flow problem has an integral optimum, even though its variables are allowed to be real numbers. Thus, we need no integer constraints to obtain an integral optimal flow. This property does not say that every feasible or optimal flow must be integral, and it need not survive additional arbitrary linear constraints.

10.5.1 The Same Example in Excel Solver

We can now hand our small example to a general solver. The accompanying workbook `5_network_flows_solver.xlsx` contains a `MinCostFlow` sheet with this model and a `MaxFlow` sheet for the 100–1 example. Both contain a checked optimal solution and instructions for setting up Solver manually. To build the minimum-cost model yourself, create the following table in cells `A1:E5`. The entries in column E are the decision variables; initially enter zero there.

row	A: From	B: To	C: Capacity	D: Unit cost	E: Flow
2	s	A	3	4	0
3	A	t	3	0	0
4	s	B	3	1	0
5	B	t	3	0	0

Enter `=SUMPRODUCT(D2:D5,E2:E5)` in `E7` for the total cost. Next, enter this balance table in columns G–I. Column H calculates net outgoing flow, and column I specifies the required balance.

row	G: Vertex	H: Net outgoing flow	I: Required
2	s	`=E2+E4`	3
3	A	`=E3-E2`	0
4	B	`=E5-E4`	0
5	t	`=-E3-E5`	-3

In Solver, minimize `E7` by changing `E2:E5`. Add the constraints `E2:E5 >= 0`, `E2:E5 <= C2:C5`, and `H2:H5 = I2:I5`. Choose the *Simplex LP* solving method. If Solver is not available, enable the Solver add-in first. The formula above uses English function names; a localized installation may use a translated name such as `SUMMENPRODUKT` and semicolons instead of commas.

The expected result in `E2:E5` is $(0, 0, 3, 3)$ with total cost 3. There is no integer constraint in this model. Check the balance cells as well as the objective: a cheap shipment that fails to deliver the required amount is not a solution. The zero starting entries deliberately fail the demand constraints; Solver finds a feasible optimum from the model.

Pause and Think – Work It Out

Change cell `C4` to 2 and solve again. Compare the result with your negative-cycle calculation. Then increase the required shipment to 7 by setting the source and sink balances to 7 and -7. Explain the solver's infeasibility result using only the capacities leaving s.

11 Graph Coloring

How many colors do you need to color a map? Naturally, you could give every region its own color. But that seems wasteful. Regions that are far apart can share a color without causing any confusion. We only want regions that share a boundary to have different colors. So, how many colors are really necessary?

In 1852, Francis Guthrie asked whether four colors would always suffice for a map. The question is easy to explain, and you can try it yourself with a pencil and a few colors. A proof, however, has to account for *every* possible map. The problem remained open for more than a century. In 1976, Kenneth Appel and Wolfgang Haken established the four-color theorem using extensive computer calculations.[8] We will return to the theorem after expressing the problem as a graph.

The idea has applications well beyond maps. Imagine that two exams have students in common. They must be held at different times. Two computations need exclusive access to the same resource? They cannot run simultaneously. In both cases, we want to assign something to objects while keeping certain pairs apart. A color will be our name for this assignment; it need not be a color on a screen.

11.1 Vertex Coloring and the Chromatic Number

Let's take a simple undirected graph $G = (V, E)$. We give each vertex a color, subject to one rule: adjacent vertices must receive different colors. Vertices that are not adjacent may receive the same color, but they don't have to. This last point is worth keeping in mind. An edge forces a difference; a missing edge does not force equality.

Definition 11.1. A *proper vertex coloring* (dt. zulässige Knotenfärbung) of a graph $G = (V, E)$ with a palette of k colors is a function

$$f : V \to \{1, 2, \ldots, k\} \qquad \text{such that} \qquad \forall \{u, v\} \in E : \ f(u) \neq f(v).$$

A graph is *k-colorable* if such a function exists. The palette contains k available colors; the coloring may use fewer than k of them.

Why use numbers instead of red, green, and blue? Because the actual names of the colors do not matter. If we replace every occurrence of color 1 by blue and every occurrence of color 2 by orange, the coloring remains valid. What matters is which vertices share a color. From now on, "coloring" means proper vertex coloring unless stated otherwise.

Finding *some* coloring is easy: give every vertex its own color. The interesting question is how many colors we can save.

Definition 11.2. The *chromatic number* (dt. chromatische Zahl) $\chi(G)$ is the smallest number of colors needed for a proper vertex coloring of G. A coloring that uses exactly $\chi(G)$ colors is an *optimal coloring*. We set $\chi(G) = 0$ for the graph with no vertices.

A nonempty graph without edges needs just one color. The graph K_2 needs two. In a triangle, the third vertex is adjacent to both of the others, so it requires a third color. More generally, $\chi(K_n) = n$, because every pair of vertices must have different colors.

Pause and Think – Quick Check

Suppose somebody shows you a valid coloring with five colors. Have they shown that $\chi(G) = 5$? What additional information would convince you that fewer colors are impossible?

A coloring with five colors shows that $\chi(G) \leq 5$. To establish equality, we also need a reason why at least five colors are necessary. For instance, perhaps the graph contains five vertices that are all pairwise adjacent.

[8] For the history and a discussion of the theorem, see Robin Thomas, *The Four Color Theorem*, `https://thomas.math.gatech.edu/FC/fourcolor.html`.

Definition 11.3. A *clique* is a set of vertices that are pairwise adjacent. The *clique number* $\omega(G)$ is the largest number of vertices in a clique of G.

All vertices in a clique need different colors, even if the clique is only a small part of a larger graph. Thus, for every nonempty simple graph,

$$\omega(G) \leq \chi(G) \leq |V|.$$

The two inequalities serve different purposes: a clique gives a *lower bound*, while a coloring gives an *upper bound*. When the two bounds meet, we know the optimum. They need not meet, however. The cycle C_5 has no triangle, so its largest clique has only two vertices. Still, it needs three colors.

11.1.1 When Two Colors Suffice

Why does C_5 need three colors? Start at one vertex and alternate colors 1 and 2 around the cycle. The five vertices receive $1, 2, 1, 2, 1$. But the last vertex is adjacent to the first, and both have color 1. We cannot close the cycle. Giving the last vertex color 3 resolves the conflict. On C_4, the alternating sequence $1, 2, 1, 2$ already works. The issue is whether the cycle has odd or even length.

Recall that a bipartite graph has two parts with no edges inside either part. Color one part with color 1 and the other with color 2. Conversely, a proper coloring with at most two colors gives exactly such a partition. Together with the characterization from Section 1.4, this gives

$$G \text{ is bipartite} \iff \chi(G) \leq 2 \iff G \text{ has no odd cycle.}$$

If the graph has at least one edge, a bipartite graph needs exactly two colors. Without that qualification, saying "bipartite means $\chi(G) = 2$" would miss the edgeless graphs.

We can turn this observation into an algorithm. Start a BFS or DFS at any uncolored vertex, give it color 1, and give each newly discovered neighbor the opposite color. Whenever an edge leads to an already colored vertex, check that its endpoints have different colors. A conflict means that the graph is not bipartite. If a component is finished, start again in another uncolored component. Using adjacency lists, the whole test takes $O(|V| + |E|)$ time.

Pause and Think – Quick Check

A star has one central vertex and eight leaves. Its maximum degree is 8. How many colors does it need? Why does having many neighbors not necessarily require many colors?

11.1.2 Back to Maps

To turn a map into a graph, represent each region by a vertex and connect two vertices when their regions share a boundary segment. Merely touching at a point does not count. For an ordinary map with connected regions in the plane, this produces a planar adjacency graph. A graph is *planar* if it can be drawn in the plane without edges crossing, except at shared endpoints. A particular drawing may have crossings even when the graph is planar; perhaps the edges can be drawn differently.

Theorem 11.4 (Four-Color Theorem)**.** *Every simple planar graph G satisfies $\chi(G) \leq 4$.*

The theorem says "at most four", not that every map needs four colors. Nor does it apply to arbitrary graphs: K_5 needs five colors and is not planar. The assumptions about maps also matter. If one political country consists of disconnected regions that must all receive the same color, treating the entire country as one vertex need not produce a planar graph.

11.2 Planning a Sports Competition

Suppose $n \geq 2$ teams participate in a sports competition. Every team must play every other team exactly once, and no team may play more than once on the same day. We want the competition to finish in as few days as possible. Assume that enough playing fields are available for games involving different teams to take place on the same day.

Pause and Think – Work It Out

Express this as a graph coloring problem. What should a vertex represent? When should two vertices be adjacent? What does a color mean? A description in words is enough, but make sure that you can turn a coloring back into an actual schedule.

Perhaps your first idea was to create one vertex for each team. That seems natural: the teams are the objects mentioned in the problem, and an edge can represent a game between them. Let's call this *Model A*. Since every pair of teams must play, the resulting graph is K_n.

Now color it. We already know that K_n needs n colors. Does that mean that we need n days? Well, what does it mean to assign a day to a *team*? Suppose team A receives day 1 and team B receives day 2. On which day do they play each other? We haven't assigned a day to their game at all. If the team colors mean the days on which teams are available, the two teams even receive different days!

The difficulty is not in coloring K_n. We can do that perfectly well. The difficulty is that this coloring does not answer the scheduling question. The graph records who plays whom, but vertex colors assign values to teams, whereas we need to assign days to games. A number returned by an algorithm is not yet a solution to the original problem.

Let's try *Model B*. This time, create a vertex for every *game*. Two game vertices are adjacent exactly when the games involve a common team. For example, the game AB conflicts with AC, since team A cannot play both on one day. But AB does not conflict with CD. These games can take place on the same day.

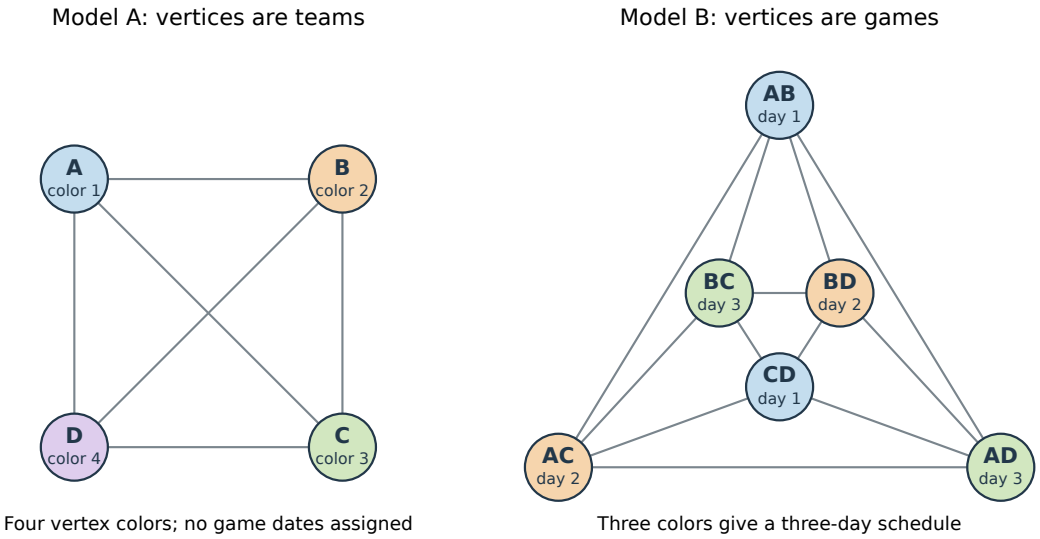


Figure 35: Two models for four teams. Model A represents teams as vertices and games as edges. Model B represents games as vertices and conflicts as edges; its vertex colors specify days. The numbers identify colors even without a color printout.

Now a color has a clear meaning: it is the day on which a game takes place. Adjacent vertices must have different colors, so games with a common team take place on different days. Vertices of the same color form the games of one day. Figure 35 shows the two models for four teams, and the coloring of Model B gives the following schedule:

Day (color)	First game	Second game
1	AB	CD
2	AC	BD
3	AD	BC

All six games occur exactly once, and each team appears once per day. Can we do better than three days? No: team A has three games and may play only once per day. We have both a schedule

using three days and a reason why fewer days are impossible. In graph terms, the vertices AB, AC, and AD form a clique of size three, and the coloring uses three colors.

Notice that the relationship works in both directions. Every proper coloring of Model B gives a valid schedule by interpreting colors as days. Every valid schedule gives a proper coloring by assigning each game its day. Therefore, minimizing the number of colors really does minimize the number of days. An arbitrary proper coloring only gives a feasible schedule; an *optimal* coloring gives a shortest schedule.

Background

Model A can also be used, if we change what we color. Assign a color to each *edge*, and require edges incident to the same vertex to have different colors. This is a *proper edge coloring*. Since edges represent games, their colors now represent days. Model B makes this an ordinary vertex coloring problem by turning the edges of Model A into vertices. The graph obtained by this construction is called the *line graph* of Model A.

So, teams as vertices was not a useless idea. It was the combination of that representation with *vertex* coloring that failed to express our constraint. We encountered a similar issue with the Seven Bridges of Königsberg: choosing a graph and choosing the question to ask about that graph have to fit together.

For n teams, Model B has $\binom{n}{2} = n(n-1)/2$ vertices. Each team participates in $n-1$ games, giving a lower bound of $n-1$ days. When n is odd, at least one team must sit out each day, so at most $(n-1)/2$ games fit into a day. Dividing the total number of games by that daily capacity gives the stronger lower bound n.

These bounds can be attained by round-robin schedules: for even n, arrange the teams in two rows of $n/2$ positions and pair teams in the same column. Keep the top-left team fixed. Read the remaining positions along the top row from left to right, then along the bottom row from right to left, and move every team one position forward in this cyclic list between rounds. After $n-1$ rounds, every pair has played once. For odd n, add a dummy team; playing the dummy means having a day off. The minimum is therefore $n-1$ days for even n and n days for odd $n \geq 3$. The table above illustrates the three rounds needed for four teams.

Pause and Think – Work It Out

What changes if there is only one playing field and only one game can be held each day? What changes if there are two fields and at most two games per day? Does the original conflict graph express these additional restrictions?

11.2.1 The Same Idea in Other Applications

The useful pattern is to identify objects that need a resource, then connect pairs that cannot share it. For *exam scheduling*, vertices are exams, an edge indicates at least one common student, and colors are time slots. The resulting schedule avoids simultaneous exams for any student. Room capacities or exams of different durations would require additional information.

In a simple model of *register allocation*, vertices represent values held by a program. An edge joins values that are simultaneously live and must be kept separately. Colors represent registers. Values whose lifetimes do not overlap can reuse a register. If the available registers do not suffice, a compiler may have to keep some values in memory instead.

Both examples ask us to reuse a limited resource without creating conflicts. The graph does not know what an exam, a game, or a register is. That interpretation is our responsibility when we construct the model and when we read its solution.

11.3 Greedy Coloring

How would you color a graph yourself? You might pick a vertex, give it the first color, then move to another vertex and see which colors are still available. Whenever possible, reuse a color. Only

introduce a new color when all existing ones are blocked by neighbors. This is the idea of *greedy coloring*, also called *first-fit coloring*.

We need an order in which to visit the vertices. Once that order is fixed, Algorithm 25 makes every choice by taking the smallest positive color number that is not already used by a colored neighbor.

Algorithm 25 Greedy Vertex Coloring (First Fit)

Input: A simple undirected graph $G = (V, E)$ and an order $(v_1, \ldots, v_m)$ containing every vertex exactly once
Output: A proper coloring f of G

```
f ← an empty map
for i ← 1 to m do
    F ← {f(u) | u ∈ N(v_i) and u ∈ dom(f)}
    c ← 1
    while c ∈ F do
        c ← c + 1
    end while
    f(v_i) ← c
end for
return f
```

Here, F is the set of currently forbidden colors and $\mathrm{dom}(f)$ is the set of vertices already assigned a color. Only *already colored* neighbors contribute to F. The algorithm always produces a proper coloring: when the later endpoint of an edge receives its color, it avoids the color of the earlier endpoint. But does it always use as few colors as possible?

11.3.1 One Graph, Two Orders

Consider the graph in Figure 36. There are four vertices $a_1, \ldots, a_4$ in the top row and four vertices $b_1, \ldots, b_4$ in the bottom row. Every a_i is adjacent to every b_j *except* b_i. There are no edges within either row. Thus,

$$E = \{\{a_i, b_j\} \mid 1 \le i, j \le 4,\ i \ne j\}.$$

This is a bipartite graph with edges, so we already know that its chromatic number is two. Let's see what greedy coloring does.

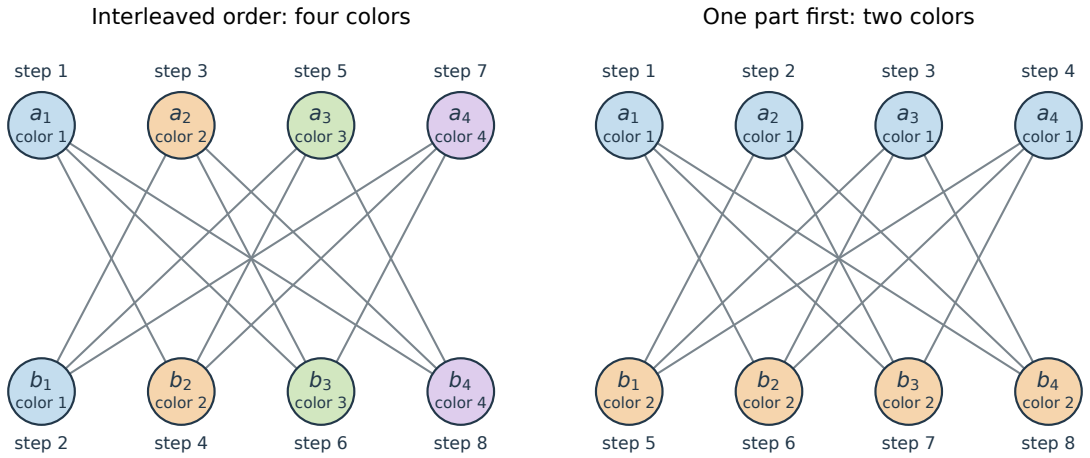


Figure 36: The same graph colored greedily in two different orders. Interleaving corresponding vertices uses four colors; visiting one whole row first uses two. Color numbers are shown explicitly. Edge crossings are not additional vertices.

First take the order $(a_1, b_1, a_2, b_2, a_3, b_3, a_4, b_4)$. Vertex a_1 receives color 1. So does b_1, because these two vertices are not adjacent. That looks economical. However, a_2 is adjacent to b_1, so it needs color 2. Vertex b_2 is adjacent to a_1, but not to a_2, and also receives color 2.

Now a_3 sees colors 1 and 2 at b_1 and b_2, so it needs color 3. The same happens to b_3 through a_1 and a_2. Finally, a_4 and b_4 each see all three earlier colors and require color 4. Every decision was legal, yet the result uses twice as many colors as necessary.

Compare this with the order $(a_1, a_2, a_3, a_4, b_1, b_2, b_3, b_4)$. No top-row vertex has a previously colored neighbor, so all four receive color 1. Every bottom-row vertex then sees only color 1 among its neighbors and receives color 2. The full sequence of choices is:

	Interleaved order			One row first		
Step	Vertex	Forbidden	Color	Vertex	Forbidden	Color
1	a_1	$\emptyset$	1	a_1	$\emptyset$	1
2	b_1	$\emptyset$	1	a_2	$\emptyset$	1
3	a_2	$\{1\}$	2	a_3	$\emptyset$	1
4	b_2	$\{1\}$	2	a_4	$\emptyset$	1
5	a_3	$\{1, 2\}$	3	b_1	$\{1\}$	2
6	b_3	$\{1, 2\}$	3	b_2	$\{1\}$	2
7	a_4	$\{1, 2, 3\}$	4	b_3	$\{1\}$	2
8	b_4	$\{1, 2, 3\}$	4	b_4	$\{1\}$	2

Pause and Think – Quick Check

After the interleaved run, can we simply merge two of its color classes to reduce the number of colors? Or do we have to change which vertices share a color? Check the edge between a_1 and b_2 before answering.

The early decision to give a_1 and b_1 the same color is allowed, but it mixes the two parts of the bipartite graph. The algorithm does not reconsider earlier choices. This is where it loses the opportunity to color the graph optimally. "Greedy" describes how an algorithm makes decisions; by itself it does not promise optimality.

11.3.2 What Does Greedy Coloring Guarantee?

A vertex of degree d has at most d forbidden colors when we reach it. Among colors $1, \ldots, d+1$, at least one is therefore available. Since no degree exceeds $\Delta(G)$, greedy coloring never uses more than $\Delta(G) + 1$ colors. Consequently,

$$\omega(G) \le \chi(G) \le \Delta(G) + 1 \le |V|$$

for a nonempty simple graph. For the eight-vertex example, $\Delta(G) = 3$, and the bad order reaches the bound of four. For the star with eight leaves, the bound permits nine colors even though two suffice. An upper bound need not be close to the answer.

An order giving an optimal greedy coloring always exists. To see the idea, imagine that we already have an optimal coloring and list its color classes one after another. Vertices within a class are not adjacent. Greedy coloring needs at most one new color for each class, so it cannot use more colors than that optimal coloring. But this observation does not give us a fast way to *find* the order: we assumed that we already knew an optimal coloring!

Background

The largest number of colors that first-fit coloring can use over all vertex orders is called the *Grundy number* $\Gamma(G)$. Thus, $\chi(G) \le \Gamma(G) \le \Delta(G) + 1$. Our example has $\chi(G) = 2$ and $\Gamma(G) = 4$. The graph stays the same; only the order changes.

In practice, one may try vertices of large degree first, or repeatedly select an uncolored vertex whose colored neighbors already use the most distinct colors. The latter idea is called *saturation degree ordering* and underlies DSATUR. Such choices aim to handle constrained vertices early, but a greedy choice still need not be optimal. In our example all degrees are equal, so sorting by degree alone would leave every choice to the tie-breaking rule.

11.4 Finding a Coloring and Finding the Optimum

It is easy to check a proposed coloring: inspect every edge and compare its endpoint colors. With adjacency lists, including a check that every vertex has been assigned a color, this takes $O(|V|+|E|)$ time. Finding the *smallest* palette is a different task. For general graphs, determining $\chi(G)$ is NP-hard, and deciding whether a graph is k-colorable is NP-complete for each fixed $k \geq 3$. This does not mean that every instance is difficult; we have already solved complete graphs, bipartite graphs, and our structured scheduling example.

An exact approach is to ask first whether a coloring with k colors exists. Assign colors to vertices one at a time. If all k colors conflict with already colored neighbors, undo an earlier assignment and try another color. This is *backtracking*. Exhausting every possibility shows that k colors do not suffice. Starting at a lower bound and increasing k until a coloring is found gives the chromatic number.

There are $k^{|V|}$ assignments to a palette of k colors before eliminating invalid choices, so exhaustive search quickly becomes expensive. Backtracking can reject conflicts early, and bounds can exclude branches that cannot improve the best coloring found so far. These improvements are useful, but do not remove the general difficulty.

Greedy coloring takes the other approach: obtain a feasible answer quickly. For the given vertex order, scanning neighbor colors and using a set with constant-time membership tests gives an expected $O(|V|+|E|)$ implementation. The vertex ordering may require additional work. If the algorithm uses q colors, then q is an upper bound on $\chi(G)$, not automatically its value. A clique of size q would certify optimality; without a matching lower bound, there may still be a better coloring.

Pause and Think – Work It Out

A scheduling program finds a five-day plan, while the conflict graph contains a clique of size four. What can you conclude about the shortest possible schedule? If a second run finds a four-day plan, what changes? If it finds no improvement, have you proved that five days are necessary?

For a practical application, we therefore have two questions to keep apart. Does the coloring satisfy the constraints of our model? And do we have a reason why fewer colors are impossible? Before either question, however, comes the one we encountered in the sports competition: does the model actually express the problem we wanted to solve?

Choosing an Algorithm

Choose the problem and its assumptions before choosing an algorithm. Let $n = |V|$, $m = |E|$. Bounds below use adjacency lists, constant-time arithmetic, and binary heaps where needed; the closure and Floyd rows use matrices. Heap bounds refer to implementations storing predecessors, not copying whole paths on every update.

Algorithm	Problem and assumptions	Output	Central idea	Time
BFS	Fewest edges; unweighted graph	Distances, paths	Explore layers	$O(n + m)$
Tarjan	Strong connectivity; directed graph	SCC partition	DFS indices and lowlinks	$O(n + m)$
Topological sort	Precedence; DAG	Compatible total order	Remove zero in-degree vertices	$O(n + m)$
Warshall	Reachability; directed or symmetric adjacency matrix	Closure matrix	Allow one more intermediate vertex	$O(n^3)$
Floyd	All-pairs distances; no negative cycles	Distance matrix	Same recurrence with $\min, +$	$O(n^3)$
Bellman–Moore–Ford	Single-source distances; negative edges allowed	Distances or detection of a reachable negative cycle	Repeat relaxation	$O(nm + n)$
Dijkstra	Single-source distances; non-negative weights	Distances, shortest-path tree	Finalize smallest distance	$O((n + m) \log n)$
A*	One target; non-negative weights, consistent h	Optimal target route	Finalize smallest $g + h$	$O((n + m) \log n)$
Prim	MST; connected, undirected; any real weights	Minimum-cost tree	Cheapest edge leaving the growing tree	$O(m \log n)$
Kruskal	Same MST problem	Minimum-cost tree	Sort edges; union-find avoids cycles	$O(m \log m)$
Chu–Liu/ Edmonds	Directed spanning arborescence; every vertex reachable from root	Cheapest rooted arborescence	Select incoming edges; contract cycles	$O(nm)$ for basic contraction
Edmonds–Karp	Maximum flow; non-negative capacities	Flow and min-cut certificate	BFS in residual graph	$O(nm^2)$
Greedy coloring	Vertex conflicts; simple undirected graph	Valid coloring, at most $\Delta + 1$ colors	First available color in chosen order	$O(n + m)$ with marked colors
Minimax	Finite alternating game tree; perfect information, zero sum	Value under optimal play	Back up alternating max/min	$O(N)$ tree nodes; $O(b^d)$

For A*, evaluating h is assumed constant-time; reopening vertices with an inconsistent heuristic has a different bound. Minimax's N counts history-tree nodes, not distinct positions. Greedy coloring guarantees feasibility and an upper bound, not a minimum coloring. A shortest-path tree minimizes distances from its source; an MST minimizes total selected edge weight.

References

[1] László Babai. Graph Isomorphism in Quasipolynomial Time. In *Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC)*, 2016. https://arxiv.org/abs/1512.03547

[2] Jan van den Brand, Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, Sushant Sachdeva, and Aaron Sidford. A Deterministic Almost-Linear Time Algorithm for Minimum-Cost Flow. arXiv:2309.16629, 2023. https://arxiv.org/abs/2309.16629

[3] Sergey Brin and Lawrence Page. The Anatomy of a Large-Scale Hypertextual Web Search Engine. *Computer Networks and ISDN Systems*, 30:107–117, 1998. https://snap.stanford.edu/class/cs224w-readings/Brin98Anatomy.pdf

[4] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Technical Report 1999-66, Stanford InfoLab, 1999. http://ilpubs.stanford.edu:8090/422/

[5] Ulrik Brandes. A Faster Algorithm for Betweenness Centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001. https://www.uni-konstanz.de/algo/publications/b-fabc-01.pdf